

LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE



Oleh:

Nur Hikmah

NIM. 2310817120010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2025**

LEMBAR PENGESAHAN LAPORAN AKHIR PRAKTIKUM MOBILE

Laporan Praktikum Pemrograman Mobile

Modul 1 : Android Basic with Kotlin

Modul 2 : Android Layout

Modul 3 : Build a Scrollable List

Modul 4 : ViewModel and Debugging

Modul 5 : Connect to the Internet

ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nur Hikmah

NIM : 2310817120010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL	8
MODUL 1: ANDROID BASIC WITH KOTLIN	11
SOAL 1	11
A. Source Code	13
B. Output Program.....	16
C. Pembahasan.....	18
MODUL 2: ANDROID LAYOUT	21
SOAL 1	21
A. Source Code	22
B. Output Program.....	28
C. Pembahasan.....	31
MODUL 3: BUILD A SCROLLABLE LIST.....	36
SOAL 1	36
A. Source Code	38
A. Output Program.....	53
B. Pembahasan.....	57
SOAL 2	66
A. Pembahasan.....	66
MODUL 4: VIEWMODEL AND DEBUGGING.....	68

SOAL 1	68
A. Source Code	69
B. Output Program.....	86
C. Pembahasan.....	94
SOAL 2	105
A. Pembahasan.....	105
MODUL 5: CONNECT TO THE INTERNET	107
SOAL 1	107
A. Source Code	108
B. Output Program.....	134
C. Pembahasan.....	139
TAUTAN GIT.....	158

DAFTAR GAMBAR

MODUL 1: ANDROID BASIC WITH KOTLIN

Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1	11
Gambar 2. Tampilan Dadu Setelah di Roll Soal 1 Modul 1	12
Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1	13
Gambar 4. Screenshot Hasil Tampilan Awal Aplikasi Soal 1 Modul 1	16
Gambar 5. Screenshot Hasil Tampilan Dadu Setelah di Roll Soal 1 Modul 1	17
Gambar 6. Screenshot Hasil Tampilan Roll Dadu Double Soal 1 Modul 1	18

MODUL 2: ANDROID LAYOUT

Gambar 7. Tampilan Awal Aplikasi Soal 1 Modul 2	21
Gambar 8. Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2	22
Gambar 9. Screenshot Hasil Tampilan Awal Aplikasi Soal 1 Modul 2	28
Gambar 10. Screenshot Hasil Tampilan Error Ketika Tidak Ada Penginputan (Kosong) Soal 1 Modul 2	29
Gambar 11. Screenshot Hasil Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2	29
Gambar 12. Screenshot Hasil Tampilan Aplikasi Setelah Dijalan dengan Merotasi Device Soal 1 Modul 2	29
Gambar 13. Screenshot Hasil Tampilan Error Ketika Menginputkan 0 Soal 1 Modul 2	30

MODUL 3: BUILD A SCROLLABLE LIST

Gambar 14. Contoh UI List Soal 1 Modul 3	37
Gambar 15. Contoh UI Detail Soal 1 Modul 3	37
Gambar 16. Screenshot Hasil UI List Soal 1 Modul 3	53
Gambar 17. Screenshot Hasil UI List 10 Item Soal 1 Modul 3	54

Gambar 18. Screenshot Hasil Berpindah ke Aplikasi Lain Saat Menekan Tombol Shopee Soal 1 Modul 3	55
Gambar 19. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 3	56

MODUL 4: VIEWMODEL AND DEBUGGING

Gambar 20. Contoh Penggunaan Debugger Soal 1 Modul 4	69
Gambar 21. Screenshot Hasil UI List Soal 1 Modul 4.....	86
Gambar 22. Screenshot Hasil UI List 10 Item Soal 1 Modul 4	87
Gambar 23. Screenshot Hasil Berpindah ke Aplikasi Lain Saat Menekan Tombol Shopee Soal 1 Modul 4	88
Gambar 24. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 4	89
Gambar 25. Screenshot Hasil UI List Item Rotate Soal 1 Modul 4.....	90
Gambar 26. Screenshot Hasil UI Detail Item Rotate Soal 1 Modul 4	90
Gambar 27. Screenshot Hasil UI Detail Item Rotate dan Scrollable Soal 1 Modul 4	91
Gambar 28. Screenshot Hasil Debugging Aplikasi Soal 1 Modul 4.....	91
Gambar 29. Screenshot Hasil Debugging Aplikasi Step Over Soal 1 Modul 4.....	92
Gambar 30. Screenshot Hasil Debugging Aplikasi Step Into Soal 1 Modul 4	92
Gambar 31. Screenshot Hasil Debugging Aplikasi Step Out Soal 1 Modul 4.....	93
Gambar 32. Screenshot Hasil Logging Soal 1 Modul 4	93

MODUL 5: CONNECT TO THE INTERNET

Gambar 33. Screenshot Hasil UI List Soal 1 Modul 5.....	134
Gambar 34. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 5	135
Gambar 35. Screenshot Hasil Berpindah ke Aplikasi Shopee Saat Menekan Tombol Shopee Soal 1 Modul 5	136
Gambar 36. Screenshot Hasil UI List Tambah Favorit Soal 1.....	137

Gambar 37. Screenshot Hasil UI Halaman Favorit Soal 1 Modul 5.....	138
--	-----

DAFTAR TABEL

MODUL 1: ANDROID BASIC WITH KOTLIN

Tabel 1. Source Code Jawaban Soal 1 Modul 1	15
Tabel 2. Source Code Jawaban Soal 1 Modul 1	16

MODUL 2: ANDROID LAYOUT

Tabel 3. Source Code Jawaban Soal 1 Modul 2	23
Tabel 4. Source Code Jawaban Soal 1 Modul 2	24
Tabel 5. Source Code Jawaban Soal 1 Modul 2	27
Tabel 6. Source Code Jawaban Soal 1 Modul 2	27

MODUL 3: BUILD A SCROLLABLE LIST

Tabel 7. Source Code Jawaban Soal 1 Modul 3	39
Tabel 8. Source Code Jawaban Soal 1 Modul 3	40
Tabel 9. Source Code Jawaban Soal 1 Modul 3	41
Tabel 10. Source Code Jawaban Soal 1 Modul 3.....	42
Tabel 11. Source Code Jawaban Soal 1 Modul 3.....	42
Tabel 12. Source Code Jawaban Soal 1 Modul 3.....	43
Tabel 13. Source Code Jawaban Soal 1 Modul 3.....	45
Tabel 14. Source Code Jawaban Soal 1 Modul 3.....	46
Tabel 15. Source Code Jawaban Soal 1 Modul 3.....	47
Tabel 16. Source Code Jawaban Soal 1 Modul 3.....	51
Tabel 17. Source Code Jawaban Soal 1 Modul 3.....	52

MODUL 4: VIEWMODEL AND DEBUGGING

Tabel 18. Source Code Jawaban Soal 1 Modul 4.....	71
Tabel 19. Source Code Jawaban Soal 1 Modul 4.....	72
Tabel 20. Source Code Jawaban Soal 1 Modul 4.....	72

Tabel 21. Source Code Jawaban Soal 1 Modul 4.....	74
Tabel 22. Source Code Jawaban Soal 1 Modul 4.....	74
Tabel 23. Source Code Jawaban Soal 1 Modul 4.....	75
Tabel 24. Source Code Jawaban Soal 1 Modul 4.....	76
Tabel 25. Source Code Jawaban Soal 1 Modul 4.....	76
Tabel 26. Source Code Jawaban Soal 1 Modul 4.....	78
Tabel 27. Source Code Jawaban Soal 1 Modul 4.....	80
Tabel 28. Source Code Jawaban Soal 1 Modul 4.....	80
Tabel 29. Source Code Jawaban Soal 1 Modul 4.....	85
Tabel 30. Source Code Jawaban Soal 1 Modul 4.....	85

MODUL 5: CONNECT TO THE INTERNET

Tabel 31. Source Code Jawaban Soal 1 Modul 5.....	108
Tabel 32. Source Code Jawaban Soal 1 Modul 5.....	108
Tabel 33. Source Code Jawaban Soal 1 Modul 5.....	109
Tabel 34. Source Code Jawaban Soal 1 Modul 5.....	109
Tabel 35. Source Code Jawaban Soal 1 Modul 5.....	110
Tabel 36. Source Code Jawaban Soal 1 Modul 5.....	110
Tabel 37. Source Code Jawaban Soal 1 Modul 5.....	111
Tabel 38. Source Code Jawaban Soal 1 Modul 5.....	112
Tabel 39. Source Code Jawaban Soal 1 Modul 5.....	112
Tabel 40. Source Code Jawaban Soal 1 Modul 5.....	113
Tabel 41. Source Code Jawaban Soal 1 Modul 5.....	114
Tabel 42. Source Code Jawaban Soal 1 Modul 5.....	116
Tabel 43. Source Code Jawaban Soal 1 Modul 5.....	117
Tabel 44. Source Code Jawaban Soal 1 Modul 5.....	119
Tabel 45. Source Code Jawaban Soal 1 Modul 5.....	120
Tabel 46. Source Code Jawaban Soal 1 Modul 5.....	121
Tabel 47. Source Code Jawaban Soal 1 Modul 5.....	122
Tabel 48. Source Code Jawaban Soal 1 Modul 5.....	122

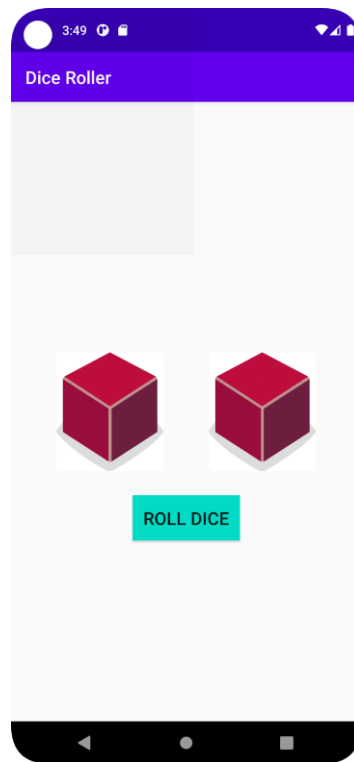
Tabel 49. Source Code Jawaban Soal 1 Modul 5.....	123
Tabel 50. Source Code Jawaban Soal 1 Modul 5.....	123
Tabel 51. Source Code Jawaban Soal 1 Modul 5.....	124
Tabel 52. Source Code Jawaban Soal 1 Modul 5.....	125
Tabel 53. Source Code Jawaban Soal 1 Modul 5.....	125
Tabel 54. Source Code Jawaban Soal 1 Modul 5.....	127
Tabel 55. Source Code Jawaban Soal 1 Modul 5.....	129
Tabel 56. Source Code Jawaban Soal 1 Modul 5.....	129
Tabel 57. Source Code Jawaban Soal 1 Modul 5.....	130
Tabel 58. Source Code Jawaban Soal 1 Modul 5.....	130
Tabel 59. Source Code Jawaban Soal 1 Modul 5.....	130
Tabel 60. Source Code Jawaban Soal 1 Modul 5.....	133

MODUL 1: ANDROID BASIC WITH KOTLIN

SOAL 1

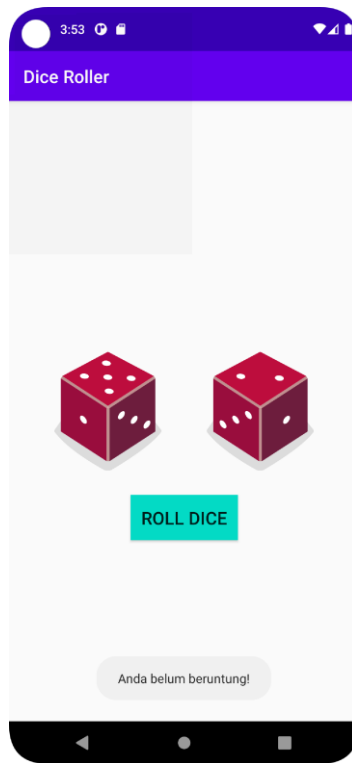
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



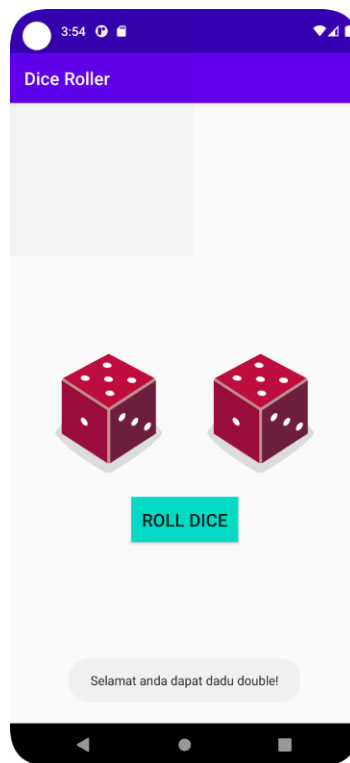
Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah di Roll Soal 1 Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1

A. Source Code

1. MainActivity.kt

```

1 package com.example.modul1
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.inputmethod.InputBinding
6 import android.widget.Button
7 import android.widget.ImageView
8 import android.widget.Toast
9 import androidx.activity.enableEdgeToEdge
10 import androidx.appcompat.app.AppCompatActivity
11 import androidx.core.view.ViewCompat
12 import androidx.core.view.WindowInsetsCompat
13 import com.example.modul1.databinding.ActivityMainBinding
14
15 class MainActivity : AppCompatActivity() {
16     private lateinit var rollbtn: Button
17     private lateinit var dadu1: ImageView
18     private lateinit var dadu2: ImageView
19
20     private lateinit var binding: ActivityMainBinding

```

```

21         override fun onCreate(savedInstanceState: Bundle?) {
22             super.onCreate(savedInstanceState)
23             enableEdgeToEdge()
24             binding =
25 ActivityMainBinding.inflate(layoutInflater)
26             setContentView(binding.root)
27
28             //         dadu1 = findViewById(R.id.dice_image1)
29             //         dadu2 = findViewById(R.id.dice_image2)
30             //
31             //         rollbtn = findViewById(R.id.rollbtn)
32
33             binding.rollbtn.setOnClickListener {
34                 rollDadu()
35             }
36         }
37
38         private fun rollDadu() {
39             val randomInt1 = (1..6).random()
40             val drawableResource1 = when (randomInt1) {
41                 1 -> R.drawable.dice_1
42                 2 -> R.drawable.dice_2
43                 3 -> R.drawable.dice_3
44                 4 -> R.drawable.dice_4
45                 5 -> R.drawable.dice_5
46                 6 -> R.drawable.dice_6
47                 else -> R.drawable.dice_0
48             }
49             val randomInt2 = (1..6).random()
50             val drawableResource2 = when (randomInt2) {
51                 1 -> R.drawable.dice_1
52                 2 -> R.drawable.dice_2
53                 3 -> R.drawable.dice_3
54                 4 -> R.drawable.dice_4
55                 5 -> R.drawable.dice_5
56                 6 -> R.drawable.dice_6
57                 else -> R.drawable.dice_0
58             }
59             binding.diceImage1.setImageResource(drawableResource1)
60             binding.diceImage2.setImageResource(drawableResource2)
61
62             if (randomInt1 == randomInt2) {
63                 Toast.makeText(this, "Selamat anda dapat dadu
double!", Toast.LENGTH_SHORT).show()
64             } else {
65                 Toast.makeText(this, "Anda belum beruntung!",
Toast.LENGTH_SHORT).show()
66             }
67         }
68     }

```

Tabel 1. Source Code Jawaban Soal 1 Modul 1

2. activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/main"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".MainActivity">
10	<TextView
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:layout_marginTop="56dp"
14	android:background="#AA93AE"
15	android:padding="10dp"
16	android:text="Dice Roller"
17	android:textColor="@color/white"
18	android:textSize="25sp"
19	app:layout_constraintStart_toStartOf="parent"
20	app:layout_constraintTop_toTopOf="parent" />
21	<ImageView
22	android:id="@+id/dice_image1"
23	android:layout_width="200dp"
24	android:layout_height="200dp"
25	android:layout_marginStart="25dp"
26	android:src="@drawable/dice_0"
27	app:layout_constraintBottom_toBottomOf="parent"
28	app:layout_constraintStart_toStartOf="parent"
29	app:layout_constraintTop_toTopOf="parent" />
30	<ImageView
31	android:id="@+id/dice_image2"
32	android:layout_width="200dp"
33	android:layout_height="200dp"
34	android:src="@drawable/dice_0"
35	app:layout_constraintBottom_toBottomOf="parent"
36	app:layout_constraintEnd_toEndOf="parent"
37	app:layout_constraintTop_toTopOf="parent" />
38	<Button
39	android:id="@+id/rollbtn"
40	android:text="Roll"
41	
42	
43	

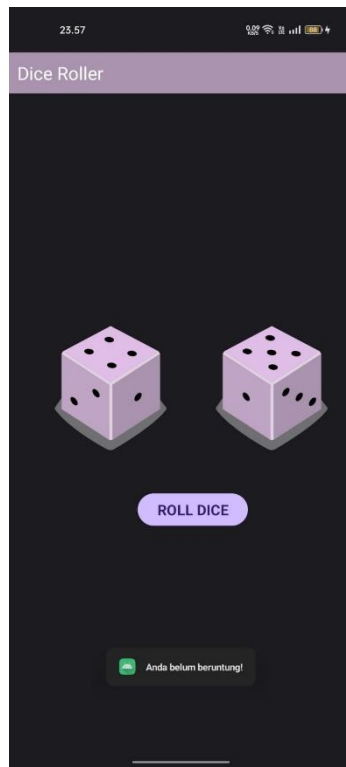
44	android:layout_width="wrap_content"
45	android:layout_height="wrap_content"
46	app:layout_constraintBottom_toBottomOf="parent"
47	app:layout_constraintEnd_toEndOf="parent"
48	app:layout_constraintHorizontal_bias="0.55"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintTop_toTopOf="parent"
51	android:layout_marginTop="300dp"
52	android:textSize="20sp"/>
53	
54	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 2. Source Code Jawaban Soal 1 Modul 1

B. Output Program



Gambar 4. Screenshot Hasil Tampilan Awal Aplikasi Soal 1 Modul 1



Gambar 5. Screenshot Hasil Tampilan Dadu Setelah di Roll Soal 1 Modul 1



C. Pembahasan

1. MainActivity.kt:

Pada line 1, `package com.example.modul1` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 13, terdapat pengimporan kelas dan beberapa fungsi yang diperlukan, diantaranya yaitu `Bundle` untuk menyimpan state saat `onCreate`, `LayoutInflater`, `InputBinding` (yang sebenarnya tidak digunakan di sini), `Button`, `ImageView`, `Toast` untuk elemen UI dan notifikasi sederhana, `enableEdgeToEdge()` digunakan untuk tampilan edge-to-edge (fullscreen modern), `AppCompatActivity` ialah superclass utama untuk aktivitas Android berbasis Material. `ViewCompat`, `WindowInsetsCompat` itu biasanya digunakan untuk mengatur padding/status bar pada tampilan, dan `ActivityMainBinding` ialah auto-generated binding class dari layout XML (`activity_main.xml`). Lalu, pada line 15 mendeklarasikan `MainActivity`, yaitu `Activity` utama yang dijalankan saat aplikasi dibuka. Line 16 sampai 18 terdapat variabel untuk tombol roll dan dua gambar dadu, tetapi tidak digunakan langsung karena digantikan oleh `ViewBinding`. Pada line 20 melakukan binding object untuk mengakses elemen UI tanpa `findViewById()`. Pada line 21 dan 22, fungsi lifecycle Android yang dipanggil saat `Activity` pertama kali dibuat. Di line 23, `enableEdgeToEdge()` memungkinkan tampilan aplikasi agar bisa menyentuh tepi layar sepenuhnya. Pada line 24, menginisialisasi binding menggunakan `LayoutInflater` dari layout `activity_main.xml`. Di line 25, `setContentView(binding.root)` digunakan untuk menampilkan layout ke layar dengan mengatur isi layout dari binding root. Pada line 27 sampai 30 tersebut termasuk komentar karena menggunakan tanda (`//`)

dan karena sudah menggunakan ViewBinding, jadi tidak perlu `findViewById()` lagi. Setelah itu, pada line 32 sampai 35 ada penggunaan kode untuk penekanan tombol, saat tombol `rollbtn` ditekan, fungsi `rollDadu()` akan dipanggil. Di line 37 terdapat fungsi untuk melempar dua buah dadu secara acak dan menampilkan hasilnya. Di line 38, `val randomInt1 = (1..6).random()` itu akan menghasilkan angka acak dari 1 sampai 6 untuk dadu pertama. Pada line 39 sampai 47 disitu memilih gambar (resource drawable) sesuai dengan angka acak dadu pertama. Di line 48, `val randomInt2 = (1..6).random()` ini menghasilkan angka acak dari 1 sampai 6 untuk dadu kedua. Pada line 49 sampai 57 memilih gambar lagi yang cocok untuk angka dadu kedua. Pada line 58 dan 59 disitu menampilkan gambar sesuai angka dadu menggunakan ViewBinding. Line 61 sampai 66 disitu melakukan pengkondisian, jika kedua dadu menghasilkan angka yang sama, maka muncul Toast “Selamat anda dapat dadu double!”, jika tidak, munculkan “Anda belum beruntung!”.

2. **activity_main.xml**

Pada line 1, terdapat deklarasi XML versi 1.0, standar untuk file XML di Android. Pada line 2, `ConstraintLayout` adalah root layout yang fleksibel dan memungkinkan untuk mengatur posisi elemen dengan konstraint (batas antar elemen) dan `android` untuk atribut dasar Android. Kemudian, pada line 3 dan 4 `app` digunakan untuk atribut khusus seperti `layout_constraint...` dan `tools` hanya digunakan di Android Studio untuk preview (tidak muncul saat runtime). Pada line 5 sampai 7 memberikan ID main ke layout, serta membuat lebar dan tinggi mengisi seluruh layar (`match_parent`). Line 8 menandakan layout ini digunakan oleh `MainActivity`, hanya untuk preview Android Studio. Setelah itu, pada line 10 sampai 12 membuat `TextView` selebar layar, dan tinggi menyesuaikan isi teks. Line 13 memberikan margin atas 56dp dari tepi atas parent (untuk

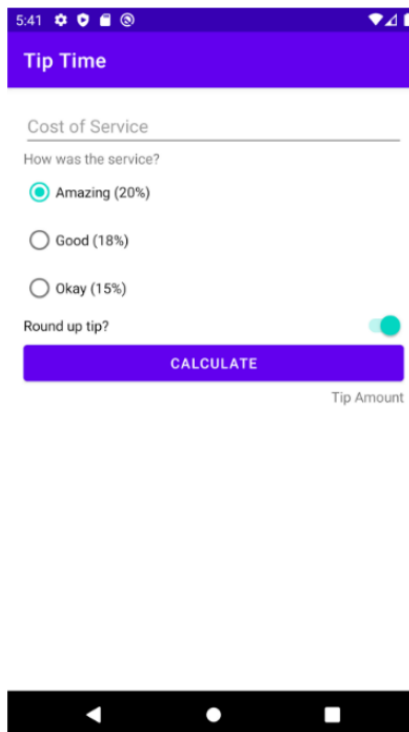
memberi ruang dari status bar). Line 14 memberikan warna pada latar belakang dengan warna ungu keabu-abuan. Line 15 terdapat padding di dalam TextView agar teks tidak terlalu mepet ke pinggir. Lalu, pada line 16 sampai 18 menampilkan teks “Dice Roller” berwarna putih dengan ukuran 25sp. Line 19 dan 20 mengunci sisi kiri dan atas TextView ke sisi kiri dan atas parent (root layout). Pada line 22 sampai 25 untuk gambar dadu pertama dengan ukuran 200x200dp. Line 26 memberikan margin kiri agar tidak terlalu mepet ke sisi kiri layar. Line 27 melakukan pemanggilan pada gambar awal dadu adalah `dice_0` (biasanya dadu kosong atau angka nol). Pada line 28 sampai 30 gambar dadu 1 diletakkan secara vertikal tengah dan menempel ke sisi kiri parent. Selanjutnya, pada line 32 sampai 36 untuk gambar dadu kedua, juga memakai gambar awal `dice_0`. Line 37 sampai 39 disitu yang berartikan gambar dadu 2 diletakkan di tengah vertikal dan menempel ke sisi kanan layout. Button pada line 41 sampai 45 disitu tombol `Roll` dengan ukuran selebar teks dan tinggi mengikuti isi. Pada line 46 sampai 50 mengatur posisi tombol yang akan terhubung ke semua sisi layout, dengan `horizontal_bias="0.55"`, sehingga tombol agak condong ke kanan tengah. Pada line 51 dan 52 memberikan jarak dari atas layout ke tombol sebesar 300dp (agar berada di bawah dua dadu) dan teks tombol yang berukuran 20sp. Terakhir penutup layout pada line 54, menutup `ConstraintLayout` sebagai root element dari UI.

MODUL 2: ANDROID LAYOUT

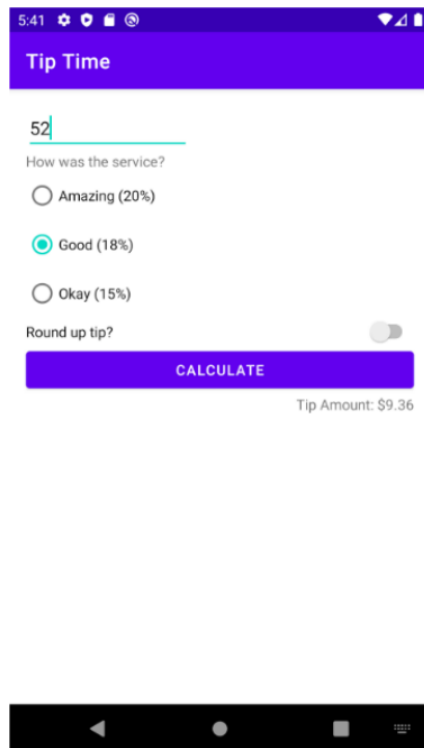
SOAL 1

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 7. Tampilan Awal Aplikasi Soal 1 Modul 2



Gambar 8. Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2

A. Source Code

1. MainActivity.kt

```

1 package com.example.modul2
2
3 import android.os.Bundle
4 import android.widget.*
5 import androidx.activity.viewModels
6 import androidx.appcompat.app.AppCompatActivity
7 import java.text.NumberFormat
8 import kotlin.math.ceil
9
10 class MainActivity : AppCompatActivity() {
11
12     private val viewModel: TipViewModel by viewModels()
13
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16         setContentView(R.layout.activity_main)
17
18         val costOfServiceEditText =
19             findViewById<EditText>(R.id.edit_cost_of_service)
20         val tipOptions =
21             findViewById<RadioGroup>(R.id.tip_options)
22         val roundUpSwitch =

```

21	findViewById<Switch>(R.id.switch_round_up)
22	val calculateButton = findViewById<Button>(R.id.button_calculate)
23	val tipResultTextView = findViewById<TextView>(R.id.text_tip_result)
24	tipResultTextView.text = viewModel.tipResult
25	calculateButton.setOnClickListener {
26	val costInput = costOfServiceEditText.text.toString()
27	val cost = costInput.toDoubleOrNull()
28	
29	
30	if (costInput.isEmpty()) {
31	tipResultTextView.text = ""
32	Toast.makeText(this, "Input tidak boleh kosong. Harap masukkan angka!", Toast.LENGTH_SHORT).show()
33	return@setOnClickListener
34	}
35	
36	if (cost == null cost <= 0.0) {
37	tipResultTextView.text = ""
38	Toast.makeText(this, "Harap masukkan angka yang valid dan lebih dari 0!", Toast.LENGTH_SHORT).show()
39	return@setOnClickListener
40	}
41	
42	val tipPercentage = when (tipOptions.checkedRadioButtonId) {
43	R.id.radio_amazing -> 0.20
44	R.id.radio_good -> 0.18
45	else -> 0.15
46	}
47	var tip = cost * tipPercentage
48	
49	if (roundUpSwitch.isChecked) {
50	tip = ceil(tip)
51	}
52	
53	val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
54	
55	viewModel.tipResult = "Tip Amount: \$formattedTip"
56	tipResultTextView.text = viewModel.tipResult
57	}
58	}
59	}

Tabel 3. Source Code Jawaban Soal 1 Modul 2

2. TipViewModel.kt

```
1 package com.example.modul2
2
3 import androidx.lifecycle.LiveData
4 import androidx.lifecycle.MutableLiveData
5 import androidx.lifecycle.ViewModel
6
7 class TipViewModel : ViewModel() {
8     var tipResult: String = ""
9
10     private val _tipAmount = MutableLiveData<Double>()
11     val tipAmount: LiveData<Double> get() = _tipAmount
12
13     fun calculateTip(totalAmount: Double, tipPercentage:
Double) {
14         val tip = totalAmount * (tipPercentage / 100)
15         _tipAmount.value = tip
16     }
17
18     fun resetTip() {
19         _tipAmount.value = 0.0
20     }
21 }
```

Tabel 4. Source Code Jawaban Soal 1 Modul 2

3. activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <TextView
10         android:id="@+id/text_header"
11         android:layout_width="0dp"
12         android:layout_height="65dp"
13         android:background="#6200EE"
14         android:gravity="center_vertical"
15         android:paddingStart="16dp"
16         android:paddingEnd="16dp"
17         android:text="Tip Time"
18         android:textColor="@android:color/white"
19         android:textSize="23sp"
20         android:textStyle="bold"
21         app:layout_constraintTop_toTopOf="parent"
```



```

22         app:layout_constraintStart_toStartOf="parent"
23         app:layout_constraintEnd_toEndOf="parent" />
24
25     <ScrollView
26         android:id="@+id/scroll_area"
27         android:layout_width="0dp"
28         android:layout_height="0dp"
29         android:padding="16dp"
30         android:fillViewport="true"
31         app:layout_constraintTop_toBottomOf="@id/text_header"
32         app:layout_constraintBottom_toBottomOf="parent"
33         app:layout_constraintStart_toStartOf="parent"
34         app:layout_constraintEnd_toEndOf="parent">
35
36         <LinearLayout
37             android:layout_width="match_parent"
38             android:layout_height="wrap_content"
39             android:orientation="vertical">
40
41             <EditText
42                 android:id="@+id/edit_cost_of_service"
43                 android:layout_width="match_parent"
44                 android:layout_height="wrap_content"
45                 android:hint="Cost of Service"
46                 android:inputType="numberDecimal" />
47
48             <TextView
49                 android:layout_width="match_parent"
50                 android:layout_height="wrap_content"
51                 android:hint="How was the service?"
52                 android:textSize="15sp" />
53
54             <RadioGroup
55                 android:id="@+id/tip_options"
56                 android:layout_width="match_parent"
57                 android:layout_height="wrap_content"
58                 android:orientation="vertical"
59                 android:paddingTop="8dp">
60
61                 <RadioButton
62                     android:id="@+id/radio_amazing"
63                     android:layout_width="wrap_content"
64                     android:layout_height="wrap_content"
65                     android:checked="true"
66                     android:text="Amazing (20%)" />
67
68                 <RadioButton
69                     android:id="@+id/radio_good"
70                     android:layout_width="wrap_content"

```

```

71         android:layout_height="wrap_content"
72         android:text="Good (18%) " />
73
74         <RadioButton
75             android:id="@+id/radio_okay"
76             android:layout_width="wrap_content"
77             android:layout_height="wrap_content"
78             android:text="Okay (15%) " />
79     </RadioGroup>
80
81     <LinearLayout
82         android:layout_width="match_parent"
83         android:layout_height="wrap_content"
84         android:orientation="horizontal"
85         android:layout_marginTop="16dp"
86         android:gravity="center_vertical">
87
88         <TextView
89             android:layout_width="0dp"
90             android:layout_height="wrap_content"
91             android:layout_weight="1"
92             android:text="Round up tip?"
93             android:textSize="15sp"
94             android:textStyle="bold" />
95
96         <Switch
97             android:id="@+id/switch_round_up"
98             android:layout_width="wrap_content"
99             android:layout_height="wrap_content"
100     />
101
102     </LinearLayout>
103
104     <android.widget.Button
105         android:id="@+id/button_calculate"
106         android:layout_width="match_parent"
107         android:layout_height="wrap_content"
108         android:layout_marginTop="20dp"
109         android:background="@drawable/button_shape"
110         android:text="CALCULATE"
111         android:textSize="17sp"
112         android:textColor="@android:color/white"
113     />
114
115     <TextView
116         android:id="@+id/text_tip_result"
117         android:layout_width="match_parent"
118         android:layout_height="wrap_content"
119         android:layout_marginTop="16dp"
120         android:text="Tip Amount:"

```

118	android:textSize="15sp"
119	android:textStyle="bold"
120	android:gravity="end"/>
121	</LinearLayout>
122	</ScrollView>
123	</androidx.constraintlayout.widget.ConstraintLayout>

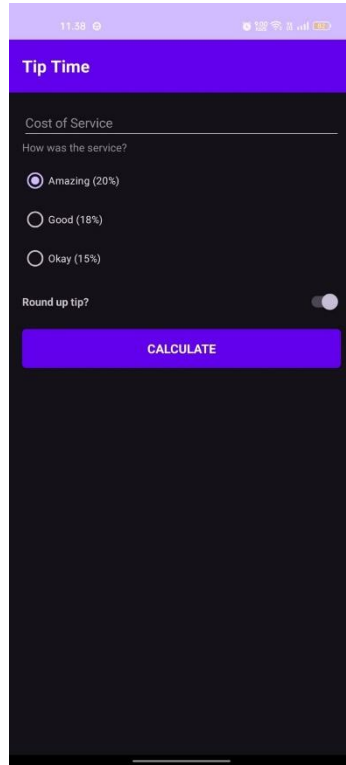
Tabel 5. Source Code Jawaban Soal 1 Modul 2

4. button_shape.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<selector
	xmlns:android="http://schemas.android.com/apk/res/android">
3	<item android:state_pressed="true">
4	<shape android:shape="rectangle">
5	<solid android:color="#6200EE" />
6	<corners android:radius="5dp" />
7	</shape>
8	</item>
9	
10	<item>
11	<shape android:shape="rectangle">
12	<solid android:color="#6200EE" />
13	<corners android:radius="5dp" />
14	</shape>
15	</item>
16	</selector>

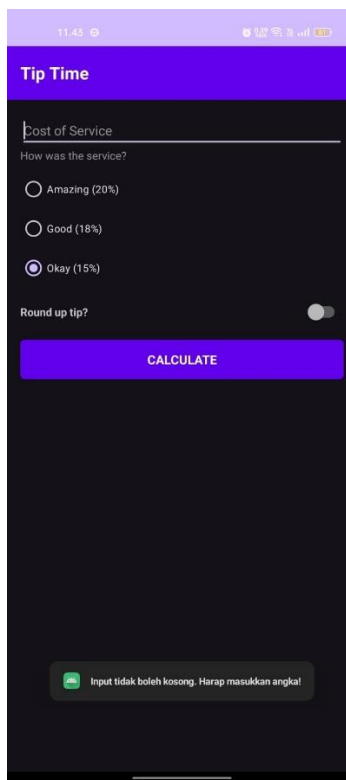
Tabel 6. Source Code Jawaban Soal 1 Modul 2

B. Output Program



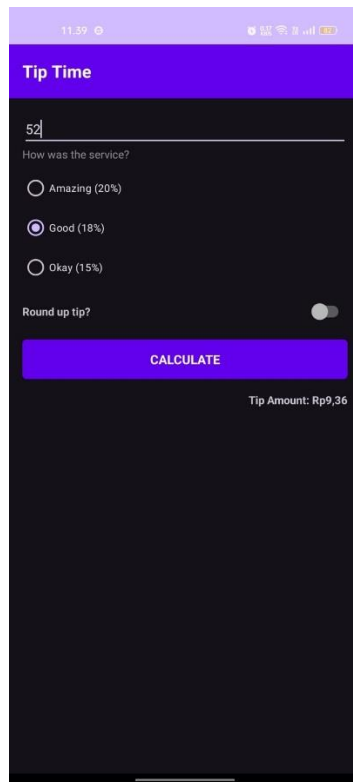
The screenshot shows the 'Tip Time' app interface. At the top, there's a status bar with the time 11:58 and various icons. Below that, the app title 'Tip Time' is displayed in a blue header. The main content area has a dark background. It starts with a text input field labeled 'Cost of Service'. Below this is a question 'How was the service?' followed by three radio button options: 'Amazing (20%)' (which is selected), 'Good (18%)', and 'Okay (15%)'. There is also a toggle switch for 'Round up tip?' which is currently turned off. At the bottom of the form is a large blue button labeled 'CALCULATE'.

Gambar 9. Screenshot Hasil Tampilan Awal Aplikasi Soal 1 Modul 2

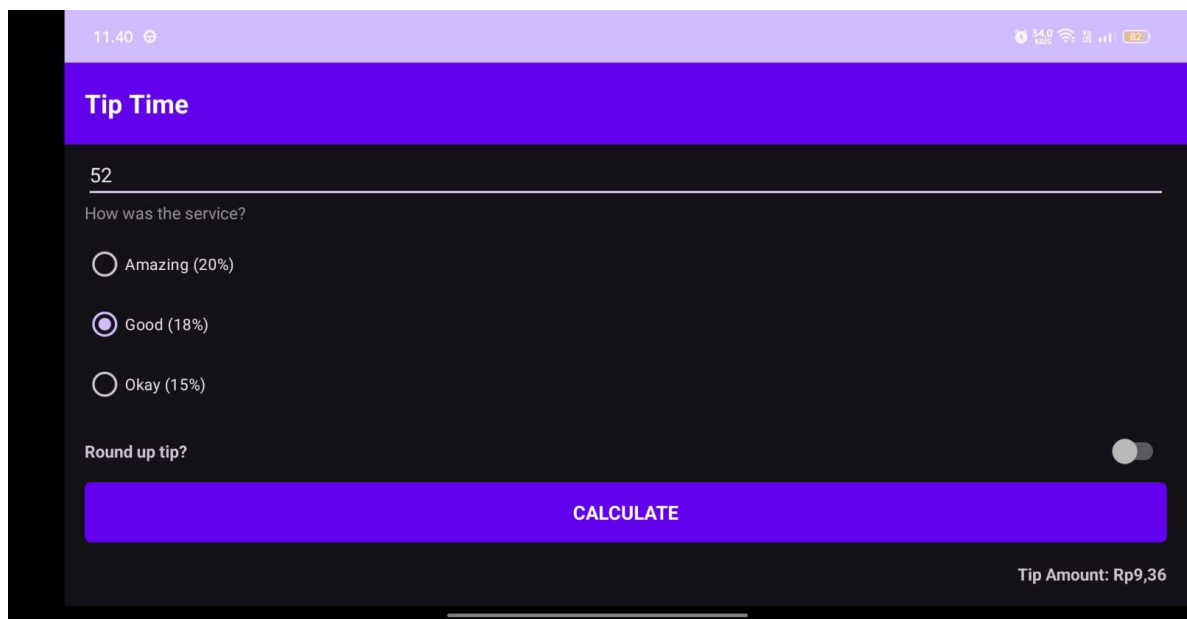


This screenshot shows the same 'Tip Time' app interface, but with an error message at the bottom. The 'Cost of Service' input field is now empty. The radio button options remain the same, with 'Amazing (20%)' still selected. The 'Round up tip?' toggle is still off. The 'CALCULATE' button is still present. At the bottom of the screen, a red error message box appears with the text 'Input tidak boleh kosong. Harap masukkan angka!' (Input cannot be empty. Please enter a number!).

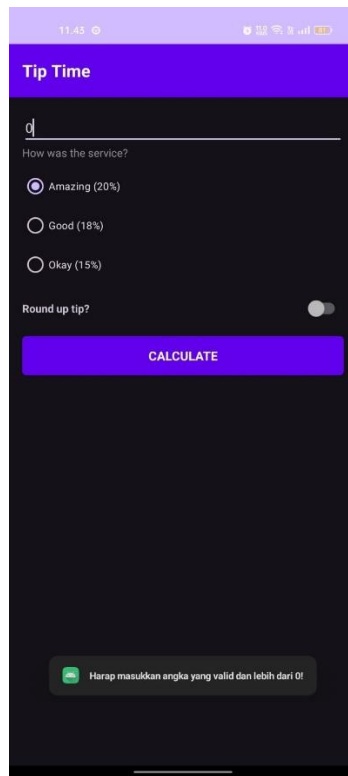
Gambar 10. Screenshot Hasil Tampilan Error Ketika Tidak Ada Penginputan (Kosong) Soal 1 Modul 2



Gambar 11. Screenshot Hasil Tampilan Aplikasi Setelah Dijalankan Soal 1 Modul 2



Gambar 12. Screenshot Hasil Tampilan Aplikasi Setelah Dijalan dengan Merotasi Device Soal 1 Modul 2



Gambar 13. Screenshot Hasil Tampilan Error Ketika Menginputkan 0 Soal 1 Modul 2

C. Pembahasan

1. MainActivity.kt

Pada line 1, `package com.example.modul2` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 8, terdapat pengimporan kelas dan beberapa fungsi yang diperlukan, diantaranya yaitu `Bundle` digunakan untuk menyimpan data saat Activity dibuat, `android.widget.*` untuk semua widget Android seperti `EditText`, `Button`, `TextView`, `dsb`, `viewModels` digunakan untuk mengakses `ViewModel` dalam activity, `AppCompatActivity` ialah Activity dasar yang kompatibel ke belakang, `NumberFormat` digunakan untuk memformat angka ke bentuk mata uang, dan `ceil` ialah fungsi untuk membulatkan angka ke atas. Lalu, pada line 10 mendeklarasikan `MainActivity`, `MainActivity` adalah kelas utama yang mewarisi dari `AppCompatActivity`, yaitu komponen dasar dari sebuah tampilan (activity) di Android. Line 12 menggunakan `ViewModel` dari `androidx.lifecycle` untuk menyimpan dan mengelola data yang bertahan saat rotasi layar atau perubahan konfigurasi. Selain itu, `by viewModels()` adalah cara delegasi untuk mendapatkan instance `TipViewModel` yang sesuai dengan lifecycle `MainActivity`. Pada line 14 terdapat override fungsi `onCreate`, fungsi ini dipanggil saat Activity dibuat pertama kali. Line 15, memanggil implementasi `onCreate` dari `AppCompatActivity` untuk melakukan inisialisasi standar. Line 16, menghubungkan Activity dengan layout XML (`activity_main.xml`) agar komponen UI bisa digunakan.

Pada line 18, mencari komponen `EditText` berdasarkan ID `edit_cost_of_service` di layout dan menyimpannya ke variabel. Line 19, mencari komponen `RadioGroup` berdasarkan ID `tip_options`. Line 20, mencari komponen `Switch` berdasarkan ID `switch_round_up`. Line

21, mencari komponen `Button` berdasarkan ID `button_calculate`. Line 22, mencari komponen `TextView` berdasarkan ID `text_tip_result`. Line 24, menampilkan nilai `tipResult` yang sudah disimpan di `ViewModel` ke dalam `TextView`. Line 26, menambahkan event listener ke `calculateButton`. Artinya, saat tombol diklik, kode di dalam blok ini akan dijalankan. Line 27, mengambil teks dari `EditText` dan mengubahnya menjadi `String`. Line 28, mencoba mengubah input `costInput` menjadi angka `Double`. Jika gagal (misal huruf atau kosong), hasilnya `null`. Line 30 sampai 34, jika input kosong maka membersihkan teks hasil tip, menampilkan pesan toast “Input tidak boleh kosong. Harap masukkan angka!”, dan menghentikan proses klik tombol (`return@setOnClickListener`). Line 36 sampai 40, jika input bukan angka valid atau angka ≤ 0 , maka akan membersihkan teks hasil tip, menampilkan pesan kesalahan (toast) “Harap masukkan angka yang valid dan lebih dari 0!”, dan menghentikan proses. Line 42 sampai 46, menentukan persentase tip berdasarkan radio button yang dipilih, yaitu Amazing (20%), Good (18%), dan lainnya (15%). Line 47, menghitung jumlah tip awal = biaya * persentase tip.

Pada line 49 sampai 51 disitu, jika pengguna mencentang `roundUpSwitch`, bulatkan tip ke atas menggunakan `ceil()`. Line 53, mengubah angka tip menjadi format mata uang lokal (Rp, \$, atau yang sesuai). Line 55, menyimpan hasil tip yang sudah diformat ke dalam `ViewModel` (`tipResult`). Line 56 sampai 59, menampilkan hasil tip di `TextView`, serta mengakhiri dari `calculateButton.setOnClickListener`, `onCreate`, dan class `MainActivity`.

2. TipViewModel.kt

Pada line 1, menyatakan bahwa file ini berada dalam package `com.example.modul2`. Package digunakan untuk mengelompokkan file agar lebih terorganisir. Line 3 sampai 5 ada beberapa pengimporan class,

diantaranya yaitu LiveData ialah objek data yang bisa di-observe (diperhatikan), berguna untuk UI, MutableLiveData ialah versi bisa diubah dari LiveData, dan ViewModel digunakan untuk komponen yang menyimpan dan mengelola data UI secara aman dari perubahan konfigurasi (seperti rotasi layar). Line 7 mendefinisikan class TipViewModel, yang mewarisi (extends) ViewModel. ViewModel ini digunakan untuk menyimpan data yang dibutuhkan MainActivity, agar tidak hilang saat rotasi atau perubahan konfigurasi. Line 8 terdapat variabel publik bertipe String, yang akan menyimpan teks hasil perhitungan tip (“Tip Amount: Rp2,00”), ini bisa langsung diakses dari activity seperti viewModel.tipResult. Line 10, mendefinisikan variabel private `_tipAmount` bertipe `MutableLiveData<Double>`. `MutableLiveData` berarti nilainya bisa diubah (mutable). Angka desimal (`Double`) menyimpan nilai tip mentah sebelum diformat. Tanda `_` (underscore) adalah konvensi penamaan untuk variabel private backing dari LiveData. Line 11, membuat versi publik dari `_tipAmount`, tapi dalam bentuk LiveData yang read-only dari luar. Tujuannya agar hanya ViewModel yang bisa mengubah nilainya, sedangkan komponen UI hanya bisa membaca.

Pada line 13 terdapat fungsi publik yang menerima dua parameter, yaitu `totalAmount` digunakan untuk jumlah biaya dan `tipPercentage` untuk persentase tip (misal 15 untuk 15%). Line 14, melakukan penghitungan nilai tip berdasarkan rumus, $\text{tip} = \text{totalAmount} \times (\text{persentase} \div 100)$. Misal: $100.000 \times 0.15 = 15.000$. Line 15, menyimpan hasil tip ke dalam `_tipAmount` agar LiveData bisa memicu observer di UI. Line 18 terdapat fungsi untuk mereset nilai tip ke 0, yang bisa digunakan ketika pengguna mau menghitung ulang. Line 19 sampai 21, mengatur ulang LiveData menjadi 0.0, yang berarti tidak ada tip saat ini, serta melakukan penutupan fungsi `resetTip` dan class `TipViewModel`.

3. activity_main.xml

Pada line 1, terdapat deklarasi XML versi 1.0, menandakan file ini adalah file XML dengan encoding UTF-8. Pada line 2, Root layout dari tampilan ini adalah `ConstraintLayout`. Artinya semua elemen di dalamnya akan diatur posisinya berdasarkan constraint (seperti "terhubung ke atas/bawah/kiri/kanan"). Kemudian, line 3 dan 4 mendeklarasikan namespace supaya kita bisa pakai atribut `android:`, `tools:`, dan `app:` dalam XML. Line 5 dan 6, `ConstraintLayout` disitu akan berfungsi untuk mengisi seluruh layar. Line 7, menandakan bahwa layout tersebut dipakai oleh `MainActivity`. `tools:` hanya untuk preview di Android Studio, tidak berpengaruh ke app yang jalan. Line 9 sampai 23 merupakan bagian header `TextView` (judul). Bagian ini berisikan teks judul "Tip Time" di bagian atas, dengan background ungu #6200EE, teks putih, ukuran besar (23sp), huruf tebal (bold), dan diatur supaya posisinya nempel di atas layar (pakai `layout_constraintTop_toTopOf="parent"`). Pada line 25 sampai 34 adalah `ScrollView`, bagian bawah dari header. `ScrollView` memungkinkan isi form yang bisa di-scroll kalau kontennya panjang. `ScrollView` ini dihubungkan ke bawah header dan ke bawah parent. Di dalam `ScrollView` ada `LinearLayout` vertical yaitu di line 36 sampai 39. Komponen-komponen di dalamnya akan ditumpuk vertikal dari atas ke bawah.

Line 41 sampai 46 merupakan bagian `EditText` (input biaya servis). Input untuk biaya layanan (service) dari user, yang hanya menerima angka desimal (karena `inputType="numberDecimal"`). Line 48 sampai 52 masuk ke bagian `TextView` (label untuk pertanyaan). Disini memberi label teks "How was the service?", ini cuma teks biasa, tidak bisa diklik. Line 54 sampai 79 ialah bagian `RadioGroup` (pilihan kualitas servis). Kumpulan pilihan jawaban dengan 3 `RadioButton`, Amazing (20%), Good (18%), Okay (15%). Ini hanya bisa memilih satu opsi dalam satu waktu. Line 81

sampai 100 merupakan bagian `LinearLayout` (round up pilihan). Bagian ini membuat baris horizontal yang berisi, `TextView` “Round up tip?” dan `Switch` untuk memilih apakah tip dibulatkan ke atas. Line 102 sampai 110 bagian `Button` (hitung tip). Bagian ini ketika tombol diklik akan menghitung tip-nya. Ada background khusus (`@drawable/button_shape`) untuk mengatur shape dari tombol tersebut yang ingin membentuk seperti kotak. Line 112 sampai 123 ialah bagian `TextView` (hasil perhitungan). Bagian ini akan menampilkan hasil perhitungan tip dan diatur supaya rata kanan (`gravity="end"`) di dalam `LinearLayout`.

4. button_shape.xml

Line 1 ialah baris standar di semua file XML Android. Line 2, selector adalah daftar kondisi tampilan (misalnya tombol ditekan, aktif, tidak aktif, dll). Ini digunakan untuk mengubah gaya tombol sesuai statusnya (pressed, focused, default, dll). Line 3 sampai 8 ialah kondisi saat tombol ditekan (`state_pressed="true"`), saat tombol ditekan, maka tampilannya akan berbentuk kotak (`rectangle`), warna solid ungu `#6200EE`, dan sudut-sudut melengkung dengan radius `5dp`. Line 10 sampai 16 ialah kondisi default (tidak ditekan), ini tampilan default tombol ketika tidak ditekan tetap ungu dan sudutnya melengkung. Karena warna pressed dan default sama, maka tampilannya mungkin terlihat tidak berubah saat ditekan.

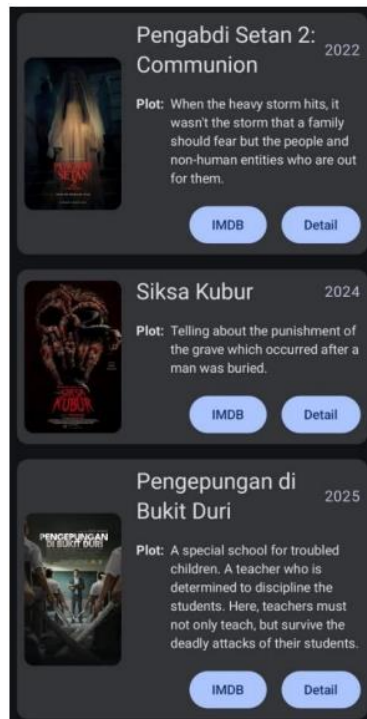
MODUL 3: BUILD A SCROLLABLE LIST

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose).
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas.
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah.
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain.
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item.
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius.
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang.
7. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment).
8. Aplikasi berbasis XML harus menggunakan ViewBinding.

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Dusahakan agar desain UI item list menyerupai UI berikut:



Gambar 14. Contoh UI List Soal 1 Modul 3

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 15. Contoh UI Detail Soal 1 Modul 3

A. Source Code

1. HomeFragment.kt

```
1 package com.example.modul3
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.navigation.fragment.findNavController
9 import androidx.recyclerview.widget.LinearLayoutManager
10 import com.example.modul3.databinding.FragmentHomeBinding
11
12 class HomeFragment : Fragment() {
13     private var _binding: FragmentHomeBinding? = null
14     private val binding get() = _binding!!
15
16     override fun onCreateView(
17         inflater: LayoutInflater, container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View {
20         _binding = FragmentHomeBinding.inflate(inflater,
21 container, false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View,
26 savedInstanceState: Bundle?) {
27         val listTas = generateTasList()
28         val adapter = AdapterTas(listTas,
29 findNavController())
30
31         binding.recyclerView.apply {
32             layoutManager =
33             LinearLayoutManager(requireContext())
34             this.adapter = adapter
35         }
36     }
37
38     override fun onDestroyView() {
39         super.onDestroyView()
40         _binding = null
41     }
42
43     private fun generateTasList(): List<ItemTas> {
44         val context = requireContext()
45         val names =
46         context.resources.getStringArray(R.array.data_name)
```

42	val harga =
	context.resources.getStringArray(R.array.data_harga)
43	val deskripsi =
	context.resources.getStringArray(R.array.data_deskripsi)
44	val links =
	context.resources.getStringArray(R.array.data_link)
45	
46	val imageIds = listOf(
47	R.drawable.marsya_bag,
48	R.drawable.lyn_bag,
49	R.drawable.reigny_bag,
50	R.drawable.madeline_bag,
51	R.drawable.tote_bag,
52	R.drawable.mini_cendy_bag,
53	R.drawable.amora_bag,
54	R.drawable.gradien_bag,
55	R.drawable.lysmare_bag,
56	R.drawable.delphine_bag
57	)
58	
59	val list = mutableListOf<ItemTas>()
60	for (i in names.indices) {
61	list.add(
62	ItemTas(
63	nama = names[i],
64	harga = harga[i],
65	deskripsi = deskripsi[i],
66	gambar = imageIds[i],
67	link = links[i]
68	)
69	)
70	}
71	return list
72	}
73	}

Tabel 7. Source Code Jawaban Soal 1 Modul 3

2. DetailFragment.kt

1	package com.example.modul3
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import
	com.example.modul3.databinding.FragmentDetailBinding

```

10
11 class DetailFragment : Fragment() {
12
13     private var _binding: FragmentDetailBinding? = null
14     private val binding get() = _binding!!
15
16     override fun onCreateView(
17         inflater: LayoutInflater, container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View {
20         _binding = FragmentDetailBinding.inflate(inflater,
21         container, false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View,
26     savedInstanceState: Bundle?) {
27         super.onViewCreated(view, savedInstanceState)
28
29         Log.d("DetailFragment", "Arguments: $arguments")
30         val tas = arguments?.getParcelable<ItemTas>("tas")
31         Log.d("DetailFragment", "Tas received: $tas")
32         tas?.let {
33             binding.tvName.text = it.nama
34             binding.tvHarga.text =
35             getString(R.string.label_harga) + " " + it.harga
36             binding.tvDeskripsi.text = it.deskripsi
37             binding.imgItemPhoto.setImageResource(it.gambar)
38         }
39
40         private fun getImageResourceIdByName(name: String):
41         Int {
42             val cleanName = name.replace("@drawable/", "")
43             return resources.getIdentifier(cleanName,
44             "drawable", requireContext().packageName)
45         }
46
47         override fun onDestroyView() {
48             super.onDestroyView()
49             _binding = null
50         }
51     }
52 }

```

Tabel 8. Source Code Jawaban Soal 1 Modul 3

3. ItemTas.kt

```

1 package com.example.modul3
2

```



```

3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class ItemTas(
8     val nama: String,
9     val gambar: Int,
10    val harga: String,
11    val deskripsi: String,
12    val link: String
13 ) : Parcelable

```

Tabel 9. Source Code Jawaban Soal 1 Modul 3

4. AdapterTas.kt

```

1 package com.example.modul3
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.util.Log
7 import android.view.LayoutInflater
8 import android.view.ViewGroup
9 import androidx.navigation.NavController
10 import androidx.recyclerview.widget.RecyclerView
11 import com.example.modul3.databinding.ItemTasBinding
12
13 class AdapterTas(
14     private val listTas: List<ItemTas>,
15     private val navController: NavController
16 ) : RecyclerView.Adapter<AdapterTas.ViewHolder>() {
17
18     inner class ViewHolder(val binding: ItemTasBinding)
19     :
20         RecyclerView.ViewHolder(binding.root) {
21
22         fun bind(tas: ItemTas) {
23             binding.tvItemName.text = tas.nama
24             binding.tvItemPrice.text = tas.harga
25
26             binding.imgItemPhoto.setImageResource(tas.gambar)
27             binding.buttonDetail.setOnClickListener {
28                 Log.d("AdapterTas", "Tombol detail diklik")
29                 val bundle = Bundle().apply {
30                     putParcelable("tas", tas)
31                 } navController.navigate(R.id.action_
32                     homeFragment_to_detailFragment, bundle)
33             }
34             binding.buttonShopee.setOnClickListener {

```

33	val intent = Intent(Intent.ACTION_VIEW,
	Uri.parse(tas.link))
34	binding.root.context.startActivity(intent)
35	}
36	}
37	}
38	
39	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): ListViewHolder {
40	val binding =
	ItemTasBinding.inflate(LayoutInflater.from(parent.context),
	parent, false)
41	return ListViewHolder(binding)
42	}
43	
44	override fun onBindViewHolder(holder: ListViewHolder,
	position: Int) {
45	holder.bind(listTas[position])
46	}
47	
48	override fun getItemCount(): Int = listTas.size
49	}

Tabel 10. Source Code Jawaban Soal 1 Modul 3

5. MainActivity.kt

1	package com.example.modul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.modul3.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	override fun onCreate(savedInstanceState: Bundle?) {
10	super.onCreate(savedInstanceState)
11	binding =
12	ActivityMainBinding.inflate(layoutInflater)
13	setContentView(binding.root)
14	}
	}

Tabel 11. Source Code Jawaban Soal 1 Modul 3

6. fragment_home.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/homeFragment"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  tools:context=".HomeFragment">
3
4
5
6
7
8
9
10 <TextView
11     android:layout_width="match_parent"
12     android:layout_height="wrap_content"
13     android:layout_marginTop="0dp"
14     android:background="#D1B3FF"
15     android:padding="15dp"
16     android:text="ShopBag - Your Bag Your Style"
17     android:textStyle="bold"
18     android:textColor="@color/black"
19     android:textSize="25sp" />
20
21 <androidx.recyclerview.widget.RecyclerView
22     android:id="@+id/recyclerView"
23     android:layout_width="match_parent"
24     android:layout_height="match_parent"
25     android:padding="8dp" />
26
27 </LinearLayout>
```

Tabel 12. Source Code Jawaban Soal 1 Modul 3

7. item_tas.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.cardview.widget.CardView
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/card_view"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:layout_gravity="center"
  android:layout_marginStart="8dp"
  android:layout_marginTop="4dp"
  android:layout_marginEnd="8dp"
  android:layout_marginBottom="4dp"
  app:cardCornerRadius="12dp"
  app:cardElevation="10dp">
3
4
5
6
7
8
9
10
11
12
13
14
```

```

15
16     <androidx.constraintlayout.widget.ConstraintLayout
17         android:layout_width="match_parent"
18         android:layout_height="wrap_content"
19         android:background="#F6F6F6"
20         android:padding="20dp">
21
22         <ImageView
23             android:id="@+id/img_item_photo"
24             android:layout_width="135dp"
25             android:layout_height="140dp"
26             android:scaleType="centerCrop"
27             app:layout_constraintStart_toStartOf="parent"
28             app:layout_constraintTop_toTopOf="parent"
29             app:layout_constraintBottom_toBottomOf="parent"
30             android:contentDescription="@string/app_name"
31             tools:src="@drawable/marsya_bag" />
32
33         <TextView
34             android:id="@+id/tv_item_name"
35             android:layout_width="0dp"
36             android:layout_height="wrap_content"
37             android:text="Tas Kulit Elegan"
38             android:textColor="@color/black"
39             android:textStyle="bold"
40             android:textSize="20sp"
41             app:layout_constraintTop_toTopOf="parent"
42             app:layout_constraintStart_toEndOf="@id/img_item_photo"
43             app:layout_constraintEnd_toEndOf="parent"
44             app:layout_constraintHorizontal_bias="0.0"
45             android:layout_marginStart="8dp"
46             tools:ignore="MissingConstraints" />
47
48         <TextView
49             android:id="@+id/tv_item_price"
50             android:layout_width="0dp"
51             android:layout_height="wrap_content"
52             android:text="Rp 350.000"
53             android:textSize="16sp"
54             android:textColor="@color/black"
55             app:layout_constraintTop_toBottomOf="@id/tv_item_name"
56             app:layout_constraintStart_toStartOf="@id/tv_item_name"
57             app:layout_constraintEnd_toEndOf="parent"
58             android:layout_marginTop="4dp" />
59
60         <Button
61             android:id="@+id/button_shopee"
62             android:layout_width="wrap_content"
63             android:layout_height="wrap_content"

```

64	android:layout_marginStart="8dp"
65	android:layout_marginTop="40dp"
66	android:text="@string/btn_shopee"
67	app:layout_constraintStart_toStartOf="@id/tv_item_name"
68	app:layout_constraintTop_toBottomOf="@id/tv_item_price"/>
69	
70	<Button
71	android:id="@+id/button_detail"
72	android:layout_width="wrap_content"
73	android:layout_height="wrap_content"
74	android:layout_marginStart="20dp"
75	android:text="@string/btn_detail"
76	app:layout_constraintStart_toEndOf="@id/button_shopee"
77	app:layout_constraintTop_toTopOf="@id/button_shopee"/>
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</androidx.cardview.widget.CardView>

Tabel 13. Source Code Jawaban Soal 1 Modul 3

8. fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="16dp"
8	android:background="#F6F6F6"
9	tools:context=".DetailFragment">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:background="#F6F6F6">
15	
16	<ImageView
17	android:id="@+id/img_item_photo"
18	android:layout_width="350dp"
19	android:layout_height="350dp"
20	android:scaleType="centerCrop"
21	app:layout_constraintTop_toTopOf="parent"
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintEnd_toEndOf="parent"
24	android:contentDescription="@string/app_name"
25	tools:src="@drawable/marsya_bag" />
26	
27	<TextView
28	android:id="@+id/tv_name"

29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:textStyle="bold"
32	android:textColor="@color/black"
33	android:textSize="25sp"
34	android:layout_marginTop="16dp"
35	app:layout_constraintTop_toBottomOf="@id/img_item_photo"
36	app:layout_constraintStart_toStartOf="parent"
37	app:layout_constraintEnd_toEndOf="parent"
38	tools:text="BOSTANTEN [Marsya Bag]" />
39	
40	<TextView
41	android:id="@+id/tv_harga"
42	android:layout_width="wrap_content"
43	android:layout_height="wrap_content"
44	android:textColor="@android:color/holo_red_dark"
45	android:textStyle="bold"
46	android:textSize="18sp"
47	android:layout_marginTop="8dp"
48	app:layout_constraintTop_toBottomOf="@id/tv_name"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintEnd_toEndOf="parent"
51	tools:text="Harga: Rp179.999" />
52	
53	<TextView
54	android:id="@+id/tv_deskripsi"
55	android:layout_width="match_parent"
56	android:layout_height="wrap_content"
57	android:layout_marginTop="30dp"
58	android:textSize="16sp"
59	android:textColor="@color/black"
60	android:background="#F6F6F6"
61	android:textAlignment="center"
62	android:lineSpacingExtra="5dp"
63	android:padding="8dp"
64	app:layout_constraintTop_toBottomOf="@id/tv_harga"
65	app:layout_constraintStart_toStartOf="parent"
66	app:layout_constraintEnd_toEndOf="parent"
67	tools:text="Deskripsi lengkap tentang tas berbahan kulit PU premium, cocok untuk kuliah dan kerja." />
68	
69	</androidx.constraintlayout.widget.ConstraintLayout>
70	</ScrollView>

Tabel 14. Source Code Jawaban Soal 1 Modul 3

9. nav_graph.xml

1	<?xml version="1.0" encoding="utf-8"?>
---	--

```

2 <navigation
  xmlns:android="http://schemas.android.com/apk/res/android"
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   xmlns:tools="http://schemas.android.com/tools"
5   app:startDestination="@id/homeFragment">
6
7   <fragment
8     android:id="@+id/homeFragment"
9     android:name="com.example.modul3.HomeFragment"
10    android:label="fragment_home"
11    tools:layout="@layout/fragment_home">
12    <action
13      android:id="@+id/action_homeFragment_to_detailFragment"
14      app:destination="@id/detailFragment" />
15    </fragment>
16
17    <fragment
18      android:id="@+id/detailFragment"
19      android:name="com.example.modul3.DetailFragment"
20      android:label="fragment_detail"
21      tools:layout="@layout/fragment_detail" />
22  </navigation>

```

Tabel 15. Source Code Jawaban Soal 1 Modul 3

10. strings.xml

```

1 <resources>
2   <string name="app_name">MODUL3</string>
3   <string name="btn_detail">Detail</string>
4   <string name="btn_shopee">Shopee</string>
5
6   <string-array name="data_name">
7     <item>BOSTANTEN [Marsya Bag]</item>
8     <item>BOSTANTEN [LYN Bag]</item>
9     <item>BOSTANTEN [Reigny Bag]</item>
10    <item>BOSTANTEN [Madeline Bag]</item>
11    <item>BOSTANTEN [Totebag]</item>
12    <item>BOSTANTEN [Mini Cendy Bag]</item>
13    <item>BOSTANTEN [Amora Bag]</item>
14    <item>MOSSDOOM [Gradien Hand Bag]</item>
15    <item>MOSSDOOM [Lysmare Bag]</item>
16    <item>MOSSDOOM [Delphine Bag]</item>
17  </string-array>
18
19  <string-array name="data_photo">
20    <item>@drawable/marsya_bag</item>
21    <item>@drawable/lyn_bag</item>
22    <item>@drawable/reigny_bag</item>

```

23	<item>@drawable/madeline_bag</item>
24	<item>@drawable/tote_bag</item>
25	<item>@drawable/mini_cendy_bag</item>
26	<item>@drawable/amora_bag</item>
27	<item>@drawable/gradien_bag</item>
28	<item>@drawable/lysmare_bag</item>
29	<item>@drawable/delphine_bag</item>
30	</string-array>
31	
32	<string-array name="data_link">
33	<item>https://id.shp.ee/ixmk3vP</item>
34	<item>https://id.shp.ee/56fWCro</item>
35	<item>https://id.shp.ee/957MHuZ</item>
36	<item>https://id.shp.ee/7ARV68N</item>
37	<item>https://id.shp.ee/quuhB8b</item>
38	<item>https://id.shp.ee/RPVhVds</item>
39	<item>https://id.shp.ee/vKZm8c9</item>
40	<item>https://id.shp.ee/6tzCzpz</item>
41	<item>https://id.shp.ee/t3osSMq</item>
42	<item>https://id.shp.ee/GaVtJQ1</item>
43	</string-array>
44	
45	<string-array name="data_harga">
46	<item>Rp179.999</item>
47	<item>Rp159.999</item>
48	<item>Rp189.999</item>
49	<item>Rp179.999</item>
50	<item>Rp169.999</item>
51	<item>Rp159.999</item>
52	<item>Rp189.999</item>
53	<item>Rp249.000</item>
54	<item>Rp189.000</item>
55	<item>Rp199.000</item>
56	</string-array>
57	
58	<string-array name="data_deskripsi">
59	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami berusaha menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Bahan terbuat dari kulit PU dengan pilihan warna Coklat, Hitam, Off Putih, Biru dan Coklat, Aprikot dan Hitam, Kuning Biru Pink, Off Putih dan Coklat, serta Off Putih dan Hitam. Berukuran 19.5 cm (Lebar) × 11.5 cm (Tinggi) × 7 cm (Tebal), dengan berat sekitar 0.35 kg. Tas ini memiliki ruang penyimpanan besar, satu kantong, serta tali belakang yang dapat disesuaikan antara 98-122 cm. Cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis,

60	belanja, pertemuan, makan, kencan, dan acara sosial seperti pesta atau konser. Target penggunaanya adalah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item> <item>(Bostanten) 100% Asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dengan pilihan warna Coklat, Off Putih, Hitam, dan Aprikot Coklat. Memiliki dimensi 29 cm (Lebar) × 14 cm (Tinggi) × 9 cm (Tebal), dan berat sekitar 0.34 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas masukan dalam, dan satu tas zipper internal. Pegangan tas memiliki jarak sedang yaitu 31 cm. Sangat cocok digunakan untuk
61	sekolah, kantor, pesta, dan perjalanan.</item> <item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini menggunakan bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, dan Hitam. Ukurannya adalah 20 cm (Lebar) × 16 cm (Tinggi) × 9 cm (Tebal), dengan berat 0.27 kg. Dilengkapi dengan ruang penyimpanan besar, tas masukan, dan sabuk bahu dengan tinggi tengah sekitar 92-112 cm yang dapat disesuaikan dan dilepas pasang. Pegangan tas berukuran sedang, yaitu 9 cm.
62	Cocok untuk sekolah, kantor, pesta, maupun perjalanan.</item> <item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, Hitam, Abu-abu, dan Coklat. Berukuran 28.5 cm (Lebar) × 21.5 cm (Tinggi) × 7 cm (Tebal), dengan berat 0.29 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas zipper belakang, dan satu saku masukan depan. Sabuk bahunya memiliki tinggi tengah sekitar 0-111 cm dan tidak dapat disesuaikan maupun dilepas. Tas ini cocok digunakan untuk sekolah, kantor, pesta, maupun perjalanan.</item>
63	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah

	dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Kecerbagunaan, kepraktisan, elean, dan gaya kedewasaan adalah inti keindahan dari tas ini. Bahan warna solid dan lekukan yang natural memberikan tampilan yang serbaguna serta memiliki plastisitas yang tepat untuk menonjolkan gaya dewasa wanita. Barang pegangan terpopuler di musim ini adalah tote bag-tas jinjing berkapasitas besar yang berkualitas baik dan terbuat dari bahan tahan air. Kapasitasnya sangat luar biasa; Anda bisa memasukkan laptop, buku teks, tablet, dan barang lainnya dengan mudah. Kepraktisannya sempurna dan sangat ramah bagi pelajar, pekerja kantor, maupun ibu-ibu yang membutuhkan ruang ekstra. Tas ini terbuat dari bahan kulit PU dan tersedia dalam kombinasi warna Off Putih dan Coklat, Off Putih dan Pink, serta Off Putih dan Hijau. Berukuran 40 cm (Lebar) × 27 cm (Tinggi) × 12 cm (Tebal) dengan berat 0.50 kg. Dilengkapi ruang penyimpanan besar, kantong, dan kantong resleting, serta tali belakang sepanjang 64 cm. Tote bag ini bukan sekadar barang sederhana yang bisa dipakai kapan saja, tetapi juga mencerminkan sikap terhadap kehidupan. Cocok digunakan di sekolah, kantor, perjalanan, bisnis, belanja, pertemuan, makan, kencan, maupun acara sosial seperti pesta dan konser.
64	Target penggunaanya adalah wanita, gadis, pekerja kantor, pelajar, dan remaja.</item> <item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam berbagai warna, seperti Coklat, Off Putih, Hitam, Aprikot dan Coklat, Off Putih dan Coklat, serta Off Putih dan Hitam. Dengan dimensi 12 cm (Lebar) × 18.5 cm (Tinggi) × 6.5 cm (Tebal) dan berat 0.16 kg, tas ini memiliki ruang penyimpanan besar dan satu kantong tambahan. Dilengkapi dengan tali belakang sepanjang 132 cm, tas ini cocok digunakan di sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan,
65	kencan, serta berbagai acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantor, pelajar, dan remaja.</item> <item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam pilihan warna Off Putih, Hitam, Coklat, dan

66	Aprikot. Dengan dimensi 22.5 cm (Lebar) × 26.5 cm (Tinggi) × 3.5 cm (Tebal) dan berat 0.28 kg, tas ini dilengkapi dengan ruang penyimpanan besar, satu kantong, dan satu kantong resleting. Tali belakangnya memiliki panjang antara 103-124 cm. Tas ini cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, serta acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item>
67	<item>Selamat datang di produk MOSSDOOM, dijamin keasliannya 100%! Tas ini terbuat dari bahan kulit PU. Dengan ukuran panjang 19.5 cm, lebar 8 cm, tinggi 16.5 cm, dan dengan berat 0.36 kg. Tersedia hanya warna Pink. Tas ini dilengkapi dengan satu saku penyimpanan besar, satu saku sisipan, satu saku beritsleting, dan tinggi pegangan tas tangan sebesar 7cm. Untuk tali bahu memiliki panjang antara 47.5-59cm (dapat disesuaikan). Semua gambar produk ini diambil dalam kehidupan nyata, jadi dijamin keasliannya.</item>
68	<item>Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Berukuran 20 cm (Panjang) × 8 cm (Lebar) × 14 cm (Tinggi) dan berat 0.36 kg. Tas ini tersedia dalam pilihan warna Blue, Light Pink, Pink, Brown, Beige White, Black. Tas ini dilengkapi dengan ruang penyimpanan besar, satu saku sisipan, satu saku beresleting, dan dengan tali bahu setinggi kira-kira 106-130cm (yang dapat disesuaikan, bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya.</item>
69	<item>Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Dengan ukuran 17 cm (Panjang) × 9 cm (Lebar) × 10 cm (Tinggi) dan berat 0.29 kg. Tersedia
70	dengan beberapa pilhan warna Pink, Blue, Purple, Apricot, Beige White, Black. Tas ini memiliki struktur internal
71	dengan saku penyimpanan besar, tinggi tali bahunya sekitar
72	110-121 cm (yang dapat disesuaikan dan bisa dilepas
73	pasang). Semua foto produk kami adalah foto asli, diambil
74	dari store official. Jadi dijamin keasliannya.</item>
75	</string-array>
76	<string name="label_nama">Nama</string> <string name="label_harga">Harga:</string> <string name="placeholder_nama">Nama Tas</string> <string name="placeholder_harga">Harga Tas</string> </resources>

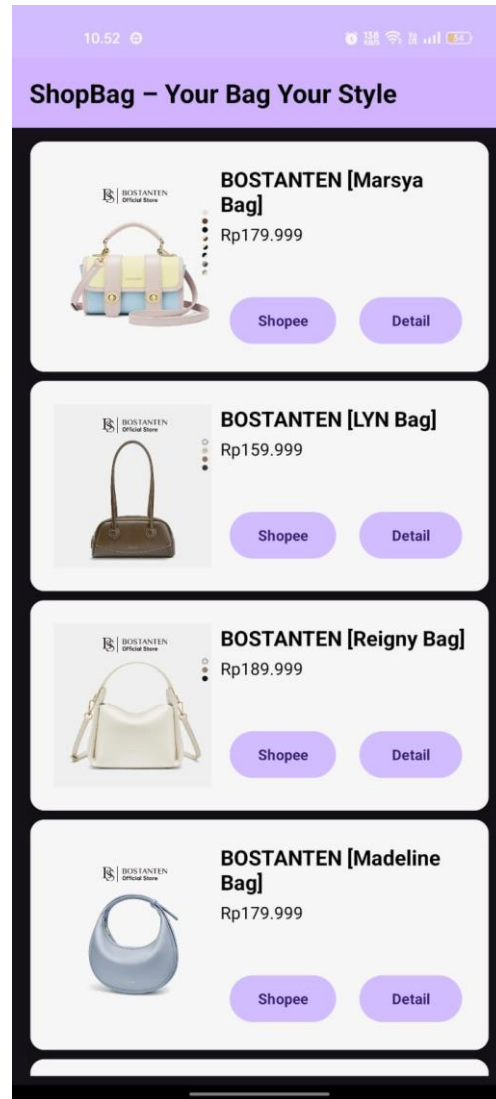
Tabel 16. Source Code Jawaban Soal 1 Modul 3

11. activity_main.xml

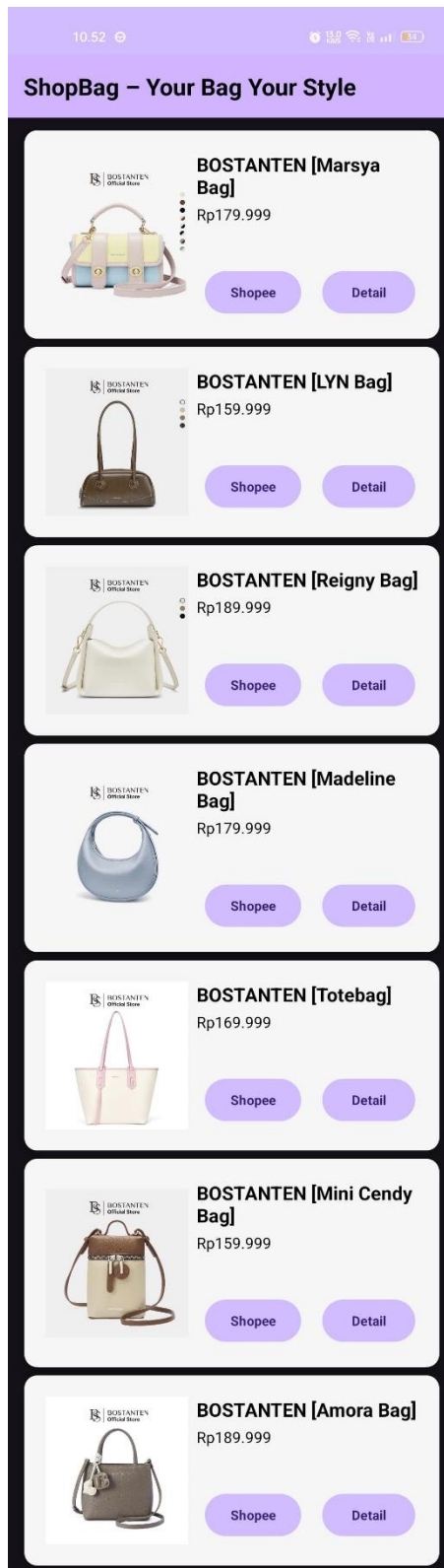
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:tools="http://schemas.android.com/tools"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity"
9	tools:ignore="ExtraText">
10	
11	<androidx.fragment.app.FragmentContainerView
12	android:id="@+id/nav_host_fragme
13	
14	android:name="androidx.navigation.fragment.NavHostFragment"
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	app:navGraph="@navigation/nav_graph"
18	app:defaultNavHost="true" />
	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 17. Source Code Jawaban Soal 1 Modul 3

B. Output Program



Gambar 16. Screenshot Hasil UI List Soal 1 Modul 3



Gambar 17. Screenshot Hasil UI List 10 Item Soal 1 Modul 3



Gambar 18. Screenshot Hasil Berpindah ke Aplikasi Lain Saat Menekan Tombol Shopee Soal 1 Modul 3



Gambar 19. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 3

C. Pembahasan

1. HomeFragment.kt

Pada line 1, `package com.example.modul3` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 10, class-class penting yang digunakan seperti `Fragment`, `LayoutInflater`, `RecyclerView`, dan `binding` untuk layout `FragmentHome`. Lalu, line 12 membuat fragment bernama `HomeFragment` yang merupakan turunan dari `Fragment`. Line 13 dan 14 menggunakan `View Binding` agar lebih aman dan mudah mengakses komponen UI. `_binding` digunakan untuk menyimpan objek binding yang diisi saat layout di-inflate. `binding` adalah versi non-null yang digunakan di dalam fragment. Line 16 sampai 22 membuat dan mengembalikan tampilan (UI) fragment. `FragmentHomeBinding.inflate(...)` digunakan untuk meng-inflate layout XML untuk fragment ini dan menyimpannya ke `_binding`. Setelah tampilan dibuat, data tas di-generate dan dimasukkan ke adapter yaitu pada line 24 sampai 32. Adapter diatur untuk `RecyclerView`, menggunakan `LinearLayoutManager` agar tampil vertikal. Line 34 sampai 37, menghapus referensi binding ketika tampilan dihancurkan untuk mencegah memory leak.

Pada line 39 sampai 44, mengambil resource string array dari `res/values/strings.xml` (atau file arrays lain) untuk nama tas, harga, deskripsi, dan link produk. Line 46 sampai 57, menyediakan daftar ID gambar tas dalam bentuk list. Line 59 sampai 73, menggabungkan data teks dan gambar menjadi objek `ItemTas`, lalu dikembalikan sebagai list yang akan ditampilkan di `RecyclerView`.

2. DetailFragment.kt

Pada line 1, `package com.example.modul3` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 9, class-class penting yang digunakan seperti `lifecycle`, `fragment`, `log debugging`, dan `View Binding`. Lalu, line 12 mendefinisikan fragment detail bernama `DetailFragment` sebagai turunan dari `Fragment`. Line 13 dan 14, `view binding` digunakan untuk mengakses elemen UI pada layout `fragment_detail.xml`. `_binding` adalah nullable, dan `binding` adalah versi non-null yang digunakan di dalam fragment. Line 16 sampai 22 membuat dan mengembalikan tampilan (UI) fragment. `FragmentDetailBinding.inflate(...)` digunakan untuk meng-inflate layout XML untuk fragment ini dan menyimpannya ke `_binding`. `binding.root` mengembalikan root view yang akan ditampilkan fragment. Line 24 dan 25, melakukan pemanggilan setelah view selesai dibuat. Umumnya dipakai untuk setup logic seperti isi data. Line 27, mencetak data `arguments` ke `Logcat` untuk debugging. Line 28 dan 29, mengambil data `ItemTas` yang dikirim melalui `Bundle` (via `Navigation Component`) dari fragment sebelumnya dan `ItemTas` harus implement `Parcelable`.

Pada line 30 sampai 36, jika data `tas` tidak null, isi elemen UI (`TextView`, `ImageView`) dengan data dari objek `ItemTas`. Line 38 sampai 41 terdapat fungsi opsional untuk mencari ID drawable berdasarkan nama string. Tidak digunakan langsung di kode ini, tapi berguna jika gambar ingin di-set dari nama string. Line 43 sampai 47, membersihkan binding saat tampilan dihancurkan agar tidak terjadi memory leak.

3. ItemTas.kt

Pada line 1, `package com.example.modul3` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah

namespace unik untuk memisahkan file dalam aplikasi. Line 3 dan 4 mengimpor `Parcelable`, yang di mana itu interface untuk mengirim data antar komponen Android (seperti antar fragment atau activity) dan `@Parcelize` adalah annotation dari Kotlin untuk membuat proses parceling otomatis tanpa harus menulis `writeToParcel()` dan `CREATOR`. Lalu, line 6 sampai 13 membuat class bisa dikirim lewat `Bundle` dengan mudah dengan cara `@Parcelize`. `data class` ialah class khusus di Kotlin untuk menyimpan data dengan otomatis menyediakan fungsi seperti `equals()`, `hashCode()`, dan `toString()`. `ItemTas` menyimpan informasi untuk 1 item tas, nama untuk nama produknya, gambar untuk ID resource drawable gambar produk (`Int`), harga untuk harga produk (sebagai string, bisa mengandung Rp, titik, dsb), deskripsi untuk penjelasan produk di halaman detail, dan `link` adalah URL untuk ke Shopee atau marketplace lainnya. Fungsi utama dari class ini digunakan untuk menyimpan data tas dalam daftar dan mengirim data ke `DetailFragment` melalui `Bundle` secara aman dan efisien.

4. AdapterTas.kt

Pada line 1, `package com.example.modul3` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 11, class-class penting yang digunakan. Lalu, line 13 sampai 16 mendefinisikan class `AdapterTas`. `AdapterTas` menerima dua parameter, `listTas` ialah daftar objek `ItemTas` untuk ditampilkan dan `navController` untuk melakukan navigasi antar fragment (dalam hal ini ke `DetailFragment`). Adapter ini menghubungkan data dengan tampilan di `RecyclerView`. Line 18 dan 19, inner class `ViewHolder` merepresentasikan tiap item di dalam daftar `RecyclerView` dengan menggunakan View Binding (`ItemTasBinding`) untuk mengakses elemen

UI. Line 21 sampai 24, mengisi teks dan gambar dari objek `ItemTas` ke komponen layout (`TextView`, `ImageView`).

Line 25 sampai 31 terdapat proses ketika tombol Detail diklik, maka data tas dimasukkan ke dalam `Bundle`, lalu navigasi ke `DetailFragment` dilakukan menggunakan `NavController`, dengan membawa `bundle` tersebut. Line 32 sampai 37 melakukan proses pada tombol Shopee, ketika tombol Shopee diklik, maka `intent` dibuka untuk mengunjungi link eksternal (ke halaman Shopee). Line 39 sampai 42, membuat `ViewHolder` baru dari layout XML `item_tas.xml`. line 44 sampai 46, mengisi data ke `ViewHolder` berdasarkan posisi dalam list. Line 48 dan 49, mengembalikan jumlah total item dalam list.

5. MainActivity.kt

Pada line 1, `package com.example.modul3` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 5, mengimpor kelas yang dibutuhkan, diantaranya yaitu `Bundle` untuk menyimpan state saat Activity dibuat, `AppCompatActivity` ialah superclass dari semua activity modern, dan `ActivityMainBinding` ialah class auto-generated dari layout `activity_main.xml` (fitur View Binding). Line 7 mendeklarasikan `MainActivity`. `MainActivity` adalah *entry point* aplikasi dan activity utama yang pertama kali ditampilkan. Line 8, mendeklarasikan variabel binding menggunakan View Binding untuk mempermudah akses ke elemen layout tanpa `findViewById()`. Line 9 dan 10 terdapat fungsi lifecycle `onCreate()`, method ini dipanggil saat activity pertama kali dibuat. Line 11, menginisialisasi View Binding untuk `activity_main.xml`. Line 12 sampai 14, menetapkan tampilan layout utama ke root view dari View Binding.

6. fragment_home.xml

Pada line 1 sampai 8 terdapat `LinearLayout` yang di manakomponen dasar dengan orientasi vertikal untuk menyusun anak-anaknya dari atas ke bawah, `match_parent` digunakan untuk memenuhi seluruh lebar dan tinggi layar, dan `tools:context` hanya untuk keperluan preview di Android Studio (menunjukkan layout ini dipakai oleh `HomeFragment`). Line 10 sampai 19, menampilkan judul aplikasi di bagian atas, dengan background ungu (`#D1B3FF`), teks hitam, ukuran besar `25sp`, tebal, dan padding `15dp` agar teks tidak terlalu mepet. Kemudian, line 21 sampai 27 ialah tempat untuk menampilkan daftar produk tas menggunakan `RecyclerView`, juga menggunakan seluruh sisa ruang layar di bawah `TextView`. id tersebut yang diakses di `HomeFragment.kt` untuk menampilkan daftar tas melalui adapter.

7. item_tas.xml

Pada line 1 sampai 14 terdapat `CardView` (root) untuk membungkus seluruh isi item agar terlihat seperti kartu dengan bayangan. `cardCornerRadius="12dp"` untuk mengatur sudut membulat dan `cardElevation="10dp"` untuk memberikan efek bayangan agar item terlihat menonjol. Line 16 sampai 20 terdapat `ConstraintLayout` (Container dalam Card), layout fleksibel untuk mengatur posisi komponen UI berdasarkan kait (constraint) terhadap komponen lain atau parent-nya. Kemudian, line 22 sampai 31 menampilkan foto produk, dengan ukuran tetap `135dp x 140dp`, dan `scaleType="centerCrop"` agar gambar memenuhi tampilan dengan cropping bagian luar. Line 33 sampai 46, menampilkan nama tas. `textStyle="bold"`, `textSize="20sp"` untuk mengatur teks nya agar nama terlihat jelas, dan ditempatkan di sebelah kanan gambar. Lalu, line 48 sampai 58 disitu berfungsi untuk menampilkan harga tas dan ditempatkan di

bawah nama produk. Line 60 sampai 68 membuat tombol untuk berpindah ke marketplace Shopee. Dengan posisi di bawah harga dan di sebelah kiri. Constraint ke `@id/tv_item_price` (top) dan Constraint ke `@id/tv_item_name` (start). Line 70 sampai 79 membuat tombol untuk navigasi ke halaman detail produk. Dengan posisi di sebelah kanan tombol Shopee, menyatu sejajar atasnya.

8. `fragment_detail.xml`

Line 1 sampai 9 terdapat `<ScrollView>` untuk membungkus seluruh isi layout agar bisa di-scroll jika kontennya lebih panjang dari layar. ini sangat berguna untuk tampilan detail yang bisa panjang, seperti pada deskripsi produk. Line 11 sampai 14, `ConstraintLayout` (di dalam `ScrollView`) digunakan untuk menyusun posisi elemen-elemen UI dengan fleksibel. Pada line 16 sampai 25 menampilkan gambar produk ukuran besar (350dp x 350dp). `centerCrop` agar gambar tetap proporsional memenuhi area dan diposisikan di bagian atas tengah layout. Kemudian, line 27 sampai 38 menampilkan nama produk dengan ukuran besar (25sp) dan bold. Diposisikan di bawah gambar dan disejajarkan ke tengah. Line 40 sampai 51, menampilkan harga dengan warna merah gelap (`holo_red_dark`) dan bold. Diposisikan di bawah nama produk dan tetap di tengah. Lalu, line 53 sampai 70 menampilkan deskripsi lengkap produk. `lineSpacingExtra="5dp"` mengatur agar teks lebih nyaman dibaca. `textAlignment="center"` digunakan untuk isi teks diratakan ke tengah. `padding` dan `marginTop` cukup besar untuk memberi jarak visual. Serta lebar penuh (`match_parent`) agar deskripsi tidak terpotong.

9. `nav_graph.xml`

Pada line 1 sampai 5, menentukan bahwa fragment pertama kali yang ditampilkan saat app berjalan adalah `HomeFragment`. Line 7 sampai 11, mewakili untuk tampilan utama (daftar tas). Fragment ini berada di package

`com.example.modul3.` Layout yang digunakan ialah `fragment_home.xml`. Line 12 sampai 15, mendefinisikan rute navigasi dari `HomeFragment` ke `DetailFragment`, hal ini akan dipicu saat user klik tombol “Detail” pada item tas. Line 17 sampai 22 adalah fragment untuk menampilkan detail dari item tas yang dipilih, dengan menggunakan layout `fragment_detail.xml`.

10. strings.xml

Pada line 1 sampai 4 terdapat `<resources>`, yang di mana semua string dan array harus berada di dalam tag `<resources>` tersebut. Fungsi-fungsinya terdiri dari `app_name` itu nama aplikasi. `btn_detail` dan `btn_shopee` adalah label pada tombol navigasi atau detail dan tombol ke Shopee. `label_nama` dan `label_harga` untuk label yang muncul pada halaman detail produk. `placeholder_nama` dan `placeholder_harga` ialah placeholder atau teks default saat input kosong atau belum ada data. Line 6 sampai 17 terdapat `<string-array name="data_name">`, yang di mana pada array string tersebut berisi nama-nama tas dari dua merek, yaitu BOSTANTEN dan MOSSDOOM. Kemudian, line 19 sampai 30 array string untuk `data_photo`, yang berisi referensi ke gambar tas di folder `res/drawable`, dengan menggunakan notasi `@drawable/nama_gambar`. Line 32 sampai 43 terdapat array string untuk `data_link`, link akan menuju produk yang bersangkutan di marketplace Shopee. Lalu, line 45 sampai 56 terdapat array string `data_harga`, yang di mana akan menampilkan harga produk dalam Rupiah. Line 58 sampai 76 terdapat array string `data_deskripsi`, setiap `<item>` disitu menjelaskan fitur, ukuran, warna, dan target pengguna dari tas. Informasi tersebut sangat lengkap dan berorientasi pada e-commerce (penjelasan produk seperti di Shopee).

11. activity_main.xml

Pada line 1 sampai 9, `ConstraintLayout` adalah layout utama yang digunakan di sini untuk mengatur posisi elemen UI di dalamnya. `ConstraintLayout` memungkinkan penataan elemen UI dengan fleksibilitas tinggi melalui constraint (penghubung). `xmlns:android` adalah deklarasi namespace untuk atribut Android standar. `xmlns:tools` digunakan oleh Android Studio untuk memberikan data atau tips visual saat pengembangan tanpa mempengaruhi aplikasi yang sebenarnya. `tools:context=".MainActivity"` memberikan konteks di mana layout ini digunakan (dalam hal ini, `MainActivity`). `tools:ignore="ExtraText"` memberikan petunjuk kepada Android Studio untuk mengabaikan peringatan tertentu yang tidak penting (misalnya teks ekstra).

Kemudian, line 11 sampai 18 terdapat `FragmentContainerView`, yang di mana komponen ini digunakan untuk menampilkan dan mengelola fragment di dalam activity. `FragmentContainerView` menggantikan penggunaan `FrameLayout` untuk penempatan fragment dan lebih terintegrasi dengan `Navigation Component` Android. `android:id="@+id/nav_host_fragme"` memberikan ID untuk `FragmentContainerView`, yang nantinya bisa digunakan untuk referensi dalam kode. `android:name="androidx.navigation.fragment.NavHostFragment"` untuk menyatakan bahwa fragment yang akan ditampilkan di dalam `FragmentContainerView` adalah `NavHostFragment`, yang merupakan bagian dari `Navigation Component`. Fragment ini bertugas mengelola navigasi antar fragment. `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk membuat `FragmentContainerView` mengambil ruang penuh dari layar,

dengan lebar dan tinggi `match_parent`, sehingga mengisi seluruh layout. `app:navGraph="@navigation/nav_graph"` untuk menyediakan `NavGraph` yang berisi alur navigasi antar fragment. `@navigation/nav_graph` adalah referensi ke file `nav_graph.xml`, yang mendefinisikan destinasi dan aksi antar fragment dalam aplikasi. `app:defaultNavHost="true"` digunakan untuk menandakan bahwa `NavHostFragment` ini adalah host utama untuk navigasi di activity ini. Artinya, fragment ini akan menangani back stack navigasi, dan tombol back perangkat akan berfungsi untuk kembali ke fragment sebelumnya (jika ada).

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView masih digunakan dalam pengembangan Android meskipun Jetpack Compose dengan LazyColumn menawarkan pendekatan yang lebih modern dan ringkas. Berikut beberapa alasan mengapa RecyclerView masih digunakan:

1. Kompatibilitas dengan Proyek Legacy

Banyak aplikasi Android yang sudah ada dibangun menggunakan sistem tampilan berbasis XML. Mengganti seluruh arsitektur UI ke Jetpack Compose memerlukan waktu dan sumber daya yang signifikan. Oleh karena itu, RecyclerView tetap digunakan untuk mempertahankan stabilitas dan menghindari risiko yang terkait dengan migrasi besar-besaran.

2. Fleksibilitas dan Kustomisasi Tinggi

RecyclerView menawarkan fleksibilitas yang tinggi dalam menampilkan berbagai jenis item dan tata letak yang kompleks. Dengan dukungan untuk berbagai LayoutManager seperti GridLayoutManager dan StaggeredGridLayoutManager, RecyclerView memungkinkan pengembang untuk menciptakan antarmuka pengguna yang kompleks dan dinamis.

3. Performa dan Kontrol yang Lebih Besar

RecyclerView memberikan kontrol penuh kepada pengembang dalam hal pengelolaan tampilan dan data. Dengan menggunakan ViewHolder dan Adapter, pengembang dapat mengoptimalkan performa aplikasi, terutama saat menangani dataset yang besar. Meskipun LazyColumn juga efisien, RecyclerView memungkinkan optimasi manual yang lebih mendalam.

4. Dukungan untuk Fitur-Fitur Lanjutan

RecyclerView mendukung berbagai fitur lanjutan seperti animasi item, drag-and-drop, dan swipe-to-dismiss. Fitur-fitur ini memungkinkan pengembang untuk menciptakan pengalaman pengguna yang interaktif dan responsif, yang

mungkin memerlukan upaya tambahan jika diimplementasikan menggunakan LazyColumn.

Referensi:

<https://medium.com/@dobri.kostadinov/jetpack-compose-lazycolumn-vs-view-system-recyclerview-18454b4738c8>

<https://developer.android.com/develop/ui/compose/migrate/migration-scenarios/recycler-view?hl=id>

<https://blog.stackademic.com/lazycolumn-vs-recyclerview-which-one-to-use-in-2025-4fed8226998b>

https://www.linkedin.com/pulse/recyclerview-xml-vs-lazycolumn-jetpack-compose-which-one-levindo-glaef?utm_source

MODUL 4: VIEWMODEL AND DEBUGGING

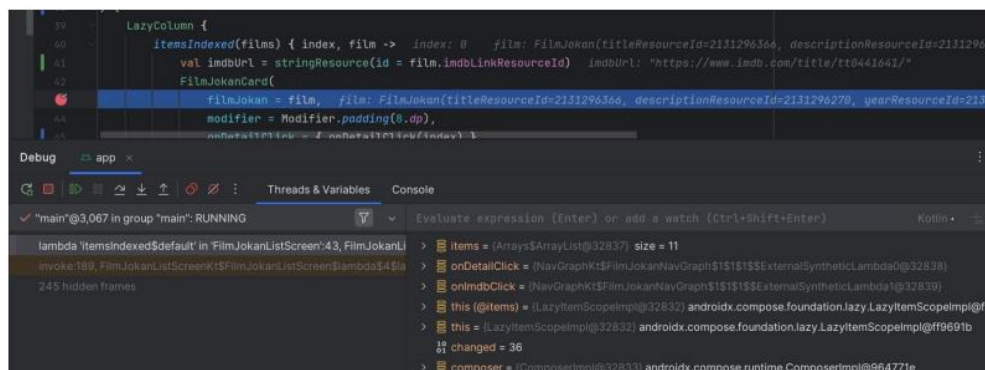
SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada

Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- Gunakan ViewModelFactory dalam pembuatan ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- Gunakan logging untuk event berikut:
 - Log saat data item masuk ke dalam list
 - Log saat tombol Detail dan tombol Explicit Intent ditekan
 - Log data dari list yang dipilih ketika berpindah ke halaman Detail
- Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



A. Source Code

1. HomeFragment.kt

```
1 package com.example.modul4
2
3 import android.os.Bundle
4 import android.os.Parcelable
5 import androidx.fragment.app.Fragment
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import androidx.lifecycle.ViewModelProvider
10 import androidx.lifecycle.lifecycleScope
11 import androidx.navigation.fragment.findNavController
12 import androidx.recyclerview.widget.LinearLayoutManager
13 import
14 com.example.modul4.databinding.FragmentHomeBinding
15
16 class HomeFragment : Fragment() {
17     private var recyclerViewState: Parcelable? = null
18     private lateinit var viewModel: TasViewModel
19     private var _binding: FragmentHomeBinding? = null
20     private val binding get() = _binding!!
21
22     override fun onCreateView(
23         inflater: LayoutInflater, container: ViewGroup?,
24         savedInstanceState: Bundle?
25     ): View {
26         _binding = FragmentHomeBinding.inflate(inflater,
27             container, false)
28         return binding.root
29     }
30
31     override fun onViewCreated(view: View,
32         savedInstanceState: Bundle?) {
33         val factory = TasViewModelFactory()
34         viewModel = ViewModelProvider(requireActivity(),
35             factory)[TasViewModel::class.java]
36         viewModel.loadData(generateTasList())
37         lifecycleScope.launchWhenStarted {
38             viewModel.tasList.collect { tasList ->
39                 val adapter = AdapterTas(tasList) { item
40 ->
41                     viewModel.selectTas(item)
42                 }
43             }
44         }
45     }
46 }
```

```

39         binding.recyclerView.layoutManager =
40     LinearLayoutManager(requireContext())
41         binding.recyclerView.adapter = adapter
42     }
43 }
44     lifecycleScope.launchWhenStarted {
45         viewModel.navigateToDetail.collect {
46             if
47             (findNavController().currentDestination?.id ==
48             R.id.homeFragment)
49             findNavController().navigate(R.id.action_homeFragment_to
50             _detailFragment)
51         }
52     }
53 }
54 }
55
56     override fun onDestroyView() {
57         super.onDestroyView()
58         _binding = null
59     }
60
61     private fun generateTasList(): List<ItemTas> {
62         val context = requireContext()
63         val names =
64     context.resources.getStringArray(R.array.data_name)
65         val harga =
66     context.resources.getStringArray(R.array.data_harga)
67         val deskripsi =
68     context.resources.getStringArray(R.array.data_deskripsi)
69         val links =
70     context.resources.getStringArray(R.array.data_link)
71
72         val imageIds = listOf(
73             R.drawable.marsya_bag,
74             R.drawable.lyn_bag,
75             R.drawable.reigny_bag,
76             R.drawable.madeline_bag,
77             R.drawable.tote_bag,
78             R.drawable.mini_cendy_bag,
79             R.drawable.amora_bag,
80             R.drawable.gradien_bag,
81             R.drawable.lysmare_bag,
82             R.drawable.delphine_bag
83         )
84
85         val list = mutableListOf<ItemTas>()
86         for (i in names.indices) {
87             list.add(

```

79	ItemTas (
80	nama = names[i],
81	harga = harga[i],
82	deskripsi = deskripsi[i],
83	gambar = imageIds[i],
84	link = links[i]
85	)
86	)
87	}
88	return list
89	}
90	}

Tabel 18. Source Code Jawaban Soal 1 Modul 4

2. DetailFragment.kt

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.lifecycle.ViewModelProvider
10	import androidx.lifecycle.LifecycleScope
11	import
	com.example.modul4.databinding.FragmentDetailBinding
12	import kotlinx.coroutines.launch
13	
14	class DetailFragment : Fragment() {
15	private lateinit var viewModel: TasViewModel
16	private var _binding: FragmentDetailBinding? = null
17	private val binding get() = _binding!!
18	
19	override fun onCreateView(
20	inflater: LayoutInflater, container:
	ViewGroup?,
21	savedInstanceState: Bundle?
22	): View {
23	_binding =
	FragmentDetailBinding.inflate(inflater, container,
	false)
24	return binding.root
25	}
26	
27	override fun onViewCreated(view: View,
	savedInstanceState: Bundle?) {
28	super.onViewCreated(view, savedInstanceState)

```

29         viewModel =
ViewModelProvider(requireActivity())[TasViewModel::clas
s.java]
30         lifecycleScope.launch {
31             viewModel.selectedTas.collect { tas ->
32                 Log.d("DetailFragment", "Tas dari
ViewModel: $tas")
33                 if (tas != null) {
34                     binding.tvName.text = tas.nama
35                     binding.tvHarga.text =
getString(R.string.label_harga) + " " + tas.harga
36                     binding.tvDeskripsi.text =
tas.deskripsi
37                 }
binding.imgItemPhoto.setImageResource(tas.gambar)
38             } else {
39                 Log.d("DetailFragment", "Tas masih
null dari ViewModel")
40             }
41         }
42     }
43 }
44
45     override fun onDestroyView() {
46         super.onDestroyView()
47         viewModel.clearSelectedTas()
48         _binding = null
49     }
50 }

```

Tabel 19. Source Code Jawaban Soal 1 Modul 4

3. ItemTas.kt

```

1 package com.example.modul4
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class ItemTas(
8     val nama: String,
9     val gambar: Int,
10    val harga: String,
11    val deskripsi: String,
12    val link: String
13 ) : Parcelable

```

Tabel 20. Source Code Jawaban Soal 1 Modul 4

4. AdapterTas.kt

```
1 package com.example.modul4
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.util.Log
7 import android.view.LayoutInflater
8 import android.view.ViewGroup
9 import androidx.navigation.NavController
10 import androidx.recyclerview.widget.RecyclerView
11 import com.example.modul4.databinding.ItemTasBinding
12
13 class AdapterTas(
14     private val listTas: List<ItemTas>,
15     private val onItemClick: (ItemTas) -> Unit
16 ) : RecyclerView.Adapter<AdapterTas.ViewHolder>() {
17
18     inner class ViewHolder(val binding: ItemTasBinding)
19     :
20         RecyclerView.ViewHolder(binding.root) {
21
22         fun bind(tas: ItemTas) {
23             binding.tvItemName.text = tas.nama
24             binding.tvItemPrice.text = tas.harga
25
26             binding.imgItemPhoto.setImageResource(tas.gambar)
27             binding.buttonDetail.setOnClickListener {
28                 Log.d("AdapterTas", "Tombol detail diklik")
29                 onItemClick(tas)
30             }
31             binding.buttonShopee.setOnClickListener {
32                 Log.d("AdapterTas", "Tombol Shopee diklik:
33                 ${tas.link}")
34                 val intent = Intent(Intent.ACTION_VIEW,
35                 Uri.parse(tas.link))
36                 binding.root.context.startActivity(intent)
37             }
38
39             override fun onCreateViewHolder(parent: ViewGroup,
40             viewType: Int): ViewHolder {
41                 val binding =
42                 ItemTasBinding.inflate(LayoutInflater.from(parent.context),
43                 parent, false)
44                 return ViewHolder(binding)
45             }
```

43	override fun onBindViewHolder(holder: ListViewHolder,
44	position: Int) {
45	holder.bind(listTas[position])
46	}
47	override fun getItemCount(): Int = listTas.size
	}

Tabel 21. Source Code Jawaban Soal 1 Modul 4

5. MainActivity.kt

1	package com.example.modul4
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.navigation.findNavController
6	import androidx.navigation.fragment.NavHostFragment
7	import com.example.modul4.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	private lateinit var binding: ActivityMainBinding
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	binding = ActivityMainBinding.inflate(layoutInflater)
14	setContentView(binding.root)
15	}
16	override fun onBackPressed() {
17	val navHostFragment =
	supportFragmentManager.findFragmentById(R.id.nav_host_fragment)
	as NavHostFragment
18	val navController = navHostFragment.navController
19	if (!navController.popBackStack()) {
20	super.onBackPressed()
21	}
22	}
23	}

Tabel 22. Source Code Jawaban Soal 1 Modul 4

6. TasViewModel.kt

1	package com.example.modul4
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelScope
6	import kotlinx.coroutines.flow.*
7	import kotlinx.coroutines.launch

```

8
9 class TasViewModel : ViewModel() {
10
11     private val _tasList =
12     MutableStateFlow<List<ItemTas>>(emptyList())
13     val tasList: StateFlow<List<ItemTas>> = _tasList
14
15     private val _selectedTas =
16     MutableStateFlow<ItemTas?>(null)
17     val selectedTas: StateFlow<ItemTas?> = _selectedTas
18
19     private val _navigateToDetail =
20     MutableSharedFlow<Unit>()
21     val navigateToDetail: SharedFlow<Unit> =
22     _navigateToDetail
23
24     fun selectTas(item: ItemTas?) {
25         _selectedTas.value = item
26         Log.d("TasViewModel", "Item dipilih:
27         ${item?.nama}")
28         if (item != null) {
29             viewModelScope.launch {
30                 _navigateToDetail.emit(Unit)
31             }
32         }
33     }
34
35     fun loadData(data: List<ItemTas>) {
36         _tasList.value = data
37         Log.d("TasViewModel", "Data item dimuat:
38         ${data.size} item")
39     }
40
41     fun clearSelectedTas() {
42         _selectedTas.value = null
43     }
44 }

```

Tabel 23. Source Code Jawaban Soal 1 Modul 4

7. TasViewModelFactory.kt

```

1 package com.example.modul4
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5
6 class TasViewModelFactory : ViewModelProvider.Factory {
7     override fun <T : ViewModel> create(modelClass:
8     Class<T>): T {

```

8	if (modelClass.isAssignableFrom(TasViewModel::class.java))
9	{
10	return TasViewModel() as T
11	}
12	throw IllegalArgumentException("Unknown ViewModel
13	class: \${modelClass.name}")
14	}
15	}

Tabel 24. Source Code Jawaban Soal 1 Modul 4

8. fragment_home.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/homeFragment"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:orientation="vertical"
9	tools:context=".HomeFragment">
10	<TextView
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:layout_marginTop="0dp"
14	android:background="#D1B3FF"
15	android:padding="15dp"
16	android:text="ShopBag - Your Bag Your Style"
17	android:textStyle="bold"
18	android:textColor="@color/black"
19	android:textSize="25sp" />
20	
21	<androidx.recyclerview.widget.RecyclerView
22	android:id="@+id/recyclerView"
23	android:layout_width="match_parent"
24	android:layout_height="match_parent"
25	android:padding="8dp" />
26	
27	</LinearLayout>

Tabel 25. Source Code Jawaban Soal 1 Modul 4

9. item_tas.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"

4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/card_view"
6	android:layout_width="match_parent"
7	android:layout_height="wrap_content"
8	android:layout_gravity="center"
9	android:layout_marginStart="8dp"
10	android:layout_marginTop="4dp"
11	android:layout_marginEnd="8dp"
12	android:layout_marginBottom="4dp"
13	app:cardCornerRadius="12dp"
14	app:cardElevation="10dp">
15	
16	<androidx.constraintlayout.widget.ConstraintLayout
17	android:layout_width="match_parent"
18	android:layout_height="wrap_content"
19	android:background="#F6F6F6"
20	android:padding="20dp">
21	
22	<ImageView
23	android:id="@+id/img_item_photo"
24	android:layout_width="135dp"
25	android:layout_height="140dp"
26	android:scaleType="centerCrop"
27	app:layout_constraintStart_toStartOf="parent"
28	app:layout_constraintTop_toTopOf="parent"
29	app:layout_constraintBottom_toBottomOf="parent"
30	android:contentDescription="@string/app_name"
31	tools:src="@drawable/marsya_bag" />
32	
33	<TextView
34	android:id="@+id/tv_item_name"
35	android:layout_width="0dp"
36	android:layout_height="wrap_content"
37	android:text="Tas Kulit Elegan"
38	android:textColor="@color/black"
39	android:textStyle="bold"
40	android:textSize="20sp"
41	app:layout_constraintTop_toTopOf="parent"
42	app:layout_constraintStart_toEndOf="@id/img_item_photo"
43	app:layout_constraintEnd_toEndOf="parent"
44	app:layout_constraintHorizontal_bias="0.0"
45	android:layout_marginStart="8dp"
46	tools:ignore="MissingConstraints" />
47	
48	<TextView
49	android:id="@+id/tv_item_price"
50	android:layout_width="0dp"
51	android:layout_height="wrap_content"
52	android:text="Rp 350.000"

53	android:textSize="16sp"
54	android:textColor="@color/black"
55	app:layout_constraintTop_toBottomOf="@id/tv_item_name"
56	app:layout_constraintStart_toStartOf="@id/tv_item_name"
57	app:layout_constraintEnd_toEndOf="parent"
58	android:layout_marginTop="4dp" />
59	
60	<Button
61	android:id="@+id/button_shopee"
62	android:layout_width="wrap_content"
63	android:layout_height="wrap_content"
64	android:layout_marginStart="8dp"
65	android:layout_marginTop="40dp"
66	android:text="@string/btn_shopee"
67	app:layout_constraintStart_toStartOf="@id/tv_item_name"
68	app:layout_constraintTop_toBottomOf="@id/tv_item_price"/>
69	
70	<Button
71	android:id="@+id/button_detail"
72	android:layout_width="wrap_content"
73	android:layout_height="wrap_content"
74	android:layout_marginStart="20dp"
75	android:text="@string/btn_detail"
76	app:layout_constraintStart_toEndOf="@id/button_shopee"
77	app:layout_constraintTop_toTopOf="@id/button_shopee"/>
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</androidx.cardview.widget.CardView>

Tabel 26. Source Code Jawaban Soal 1 Modul 4

10. fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="16dp"
8	android:background="#F6F6F6"
9	tools:context=".DetailFragment">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:background="#F6F6F6">
15	
16	<ImageView
17	android:id="@+id/img_item_photo"

```

18         android:layout_width="350dp"
19         android:layout_height="350dp"
20         android:scaleType="centerCrop"
21         app:layout_constraintTop_toTopOf="parent"
22         app:layout_constraintStart_toStartOf="parent"
23         app:layout_constraintEnd_toEndOf="parent"
24         android:contentDescription="@string/app_name"
25         tools:src="@drawable/marsya_bag" />
26
27     <TextView
28         android:id="@+id/tv_name"
29         android:layout_width="wrap_content"
30         android:layout_height="wrap_content"
31         android:textStyle="bold"
32         android:textColor="@color/black"
33         android:textSize="25sp"
34         android:layout_marginTop="16dp"
35         app:layout_constraintTop_toBottomOf="@id/img_item_photo"
36         app:layout_constraintStart_toStartOf="parent"
37         app:layout_constraintEnd_toEndOf="parent"
38         tools:text="BOSTANTEN [Marsya Bag]" />
39
40     <TextView
41         android:id="@+id/tv_harga"
42         android:layout_width="wrap_content"
43         android:layout_height="wrap_content"
44         android:textColor="@android:color/holo_red_dark"
45         android:textStyle="bold"
46         android:textSize="18sp"
47         android:layout_marginTop="8dp"
48         app:layout_constraintTop_toBottomOf="@id/tv_name"
49         app:layout_constraintStart_toStartOf="parent"
50         app:layout_constraintEnd_toEndOf="parent"
51         tools:text="Harga: Rp179.999" />
52
53     <TextView
54         android:id="@+id/tv_deskripsi"
55         android:layout_width="match_parent"
56         android:layout_height="wrap_content"
57         android:layout_marginTop="30dp"
58         android:textSize="16sp"
59         android:textColor="@color/black"
60         android:background="#F6F6F6"
61         android:textAlignment="center"
62         android:lineSpacingExtra="5dp"
63         android:padding="8dp"
64         app:layout_constraintTop_toBottomOf="@id/tv_harga"
65         app:layout_constraintStart_toStartOf="parent"
66         app:layout_constraintEnd_toEndOf="parent"

```

67	tools:text="Deskripsi lengkap tentang tas berbahan kulit PU premium, cocok untuk kuliah dan kerja." </>
68	
69	</androidx.constraintlayout.widget.ConstraintLayout>
70	</ScrollView>

Tabel 27. Source Code Jawaban Soal 1 Modul 4

11. nav_graph.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	app:startDestination="@id/homeFragment">
6	
7	<fragment
8	android:id="@+id/homeFragment"
9	android:name="com.example.modul3.HomeFragment"
10	android:label="fragment_home"
11	tools:layout="@layout/fragment_home">
12	<action
13	android:id="@+id/action_homeFragment_to_detailFragment"
14	app:destination="@id/detailFragment" />
15	</fragment>
16	
17	<fragment
18	android:id="@+id/detailFragment"
19	android:name="com.example.modul3.DetailFragment"
20	android:label="fragment_detail"
21	tools:layout="@layout/fragment_detail" />
22	</navigation>

Tabel 28. Source Code Jawaban Soal 1 Modul 4

12. strings.xml

1	<resources>
2	<string name="app_name">MODUL3</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_shopee">Shopee</string>
5	
6	<string-array name="data_name">
7	<item>BOSTANTEN [Marsya Bag]</item>
8	<item>BOSTANTEN [LYN Bag]</item>
9	<item>BOSTANTEN [Reigny Bag]</item>
10	<item>BOSTANTEN [Madeline Bag]</item>
11	<item>BOSTANTEN [Totebag]</item>

12	<item>BOSTANTEN [Mini Cendy Bag]</item>
13	<item>BOSTANTEN [Amora Bag]</item>
14	<item>MOSSDOOM [Gradien Hand Bag]</item>
15	<item>MOSSDOOM [Lysmare Bag]</item>
16	<item>MOSSDOOM [Delphine Bag]</item>
17	</string-array>
18	
19	<string-array name="data_photo">
20	<item>@drawable/marsya_bag</item>
21	<item>@drawable/lyn_bag</item>
22	<item>@drawable/reigny_bag</item>
23	<item>@drawable/madeline_bag</item>
24	<item>@drawable/tote_bag</item>
25	<item>@drawable/mini_cendy_bag</item>
26	<item>@drawable/amora_bag</item>
27	<item>@drawable/gradien_bag</item>
28	<item>@drawable/lysmare_bag</item>
29	<item>@drawable/delphine_bag</item>
30	</string-array>
31	
32	<string-array name="data_link">
33	<item>https://id.shp.ee/ixmk3vP</item>
34	<item>https://id.shp.ee/56fWCro</item>
35	<item>https://id.shp.ee/957MHuZ</item>
36	<item>https://id.shp.ee/7ARV68N</item>
37	<item>https://id.shp.ee/quuhB8b</item>
38	<item>https://id.shp.ee/RPVhVds</item>
39	<item>https://id.shp.ee/vKZm8c9</item>
40	<item>https://id.shp.ee/6tzCzpz</item>
41	<item>https://id.shp.ee/t3osSMq</item>
42	<item>https://id.shp.ee/GaVtJQ1</item>
43	</string-array>
44	
45	<string-array name="data_harga">
46	<item>Rp179.999</item>
47	<item>Rp159.999</item>
48	<item>Rp189.999</item>
49	<item>Rp179.999</item>
50	<item>Rp169.999</item>
51	<item>Rp159.999</item>
52	<item>Rp189.999</item>
53	<item>Rp249.000</item>
54	<item>Rp189.000</item>
55	<item>Rp199.000</item>
56	</string-array>
57	
58	<string-array name="data_deskripsi">
59	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami berusaha menyediakan produk

	yang memenuhi kebutuhan kualitas dan efisiensi pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Bahan terbuat dari kulit PU dengan pilihan warna Coklat, Hitam, Off Putih, Biru dan Coklat, Aprikot dan Hitam, Kuning Biru Pink, Off Putih dan Coklat, serta Off Putih dan Hitam. Berukuran 19.5 cm (Lebar) × 11.5 cm (Tinggi) × 7 cm (Tebal), dengan berat sekitar 0.35 kg. Tas ini memiliki ruang penyimpanan besar, satu kantong, serta tali belakang yang dapat disesuaikan antara 98-122 cm. Cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, dan acara sosial seperti pesta atau konser. Target penggunaannya adalah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item>
60	<item>(Bostanten) 100% Asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dengan pilihan warna Coklat, Off Putih, Hitam, dan Aprikot Coklat. Memiliki dimensi 29 cm (Lebar) × 14 cm (Tinggi) × 9 cm (Tebal), dan berat sekitar 0.34 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas masukan dalam, dan satu tas zipper internal. Pegangan tas memiliki jarak sedang yaitu 31 cm. Sangat cocok digunakan untuk sekolah, kantor, pesta, dan perjalanan.</item>
61	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini menggunakan bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, dan Hitam. Ukurannya adalah 20 cm (Lebar) × 16 cm (Tinggi) × 9 cm (Tebal), dengan berat 0.27 kg. Dilengkapi dengan ruang penyimpanan besar, tas masukan, dan sabuk bahu dengan tinggi tengah sekitar 92-112 cm yang dapat disesuaikan dan dilepas pasang. Pegangan tas berukuran sedang, yaitu 9 cm. Cocok untuk sekolah, kantor, pesta, maupun perjalanan.</item>
62	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, Hitam, Abu-

63	abu, dan Coklat. Berukuran 28.5 cm (Lebar) × 21.5 cm (Tinggi) × 7 cm (Tebal), dengan berat 0.29 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas zipper belakang, dan satu saku masukan depan. Sabuk bahunya memiliki tinggi tengah sekitar 0-111 cm dan tidak dapat disesuaikan maupun dilepas. Tas ini cocok digunakan untuk sekolah, kantor, pesta, maupun perjalanan.</item>
64	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Keserbagunaan, kepraktisan, elegan, dan gaya kedewasaan adalah inti keindahan dari tas ini. Bahan warna solid dan lekukan yang natural memberikan tampilan yang serbaguna serta memiliki plastisitas yang tepat untuk menonjolkan gaya dewasa wanita. Barang pegangan terpopuler di musim ini adalah tote bag-tas jinjing berkapasitas besar yang berkualitas baik dan terbuat dari bahan tahan air. Kapasitasnya sangat luar biasa; Anda bisa memasukkan laptop, buku teks, tablet, dan barang lainnya dengan mudah. Kepraktisannya sempurna dan sangat ramah bagi pelajar, pekerja kantoran, maupun ibu-ibu yang membutuhkan ruang ekstra. Tas ini terbuat dari bahan kulit PU dan tersedia dalam kombinasi warna Off Putih dan Coklat, Off Putih dan Pink, serta Off Putih dan Hijau. Berukuran 40 cm (Lebar) × 27 cm (Tinggi) × 12 cm (Tebal) dengan berat 0.50 kg. Dilengkapi ruang penyimpanan besar, kantong, dan kantong resleting, serta tali belakang sepanjang 64 cm. Tote bag ini bukan sekadar barang sederhana yang bisa dipakai kapan saja, tetapi juga mencerminkan sikap terhadap kehidupan. Cocok digunakan di sekolah, kantor, perjalanan, bisnis, belanja, pertemuan, makan, kencan, maupun acara sosial seperti pesta dan konser. Target penggunaanya adalah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item>
64	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam berbagai warna, seperti Coklat, Off Putih, Hitam, Aprikot dan Coklat, Off Putih dan Coklat, serta Off Putih dan Hitam. Dengan dimensi 12 cm (Lebar) × 18.5 cm (Tinggi) × 6.5 cm (Tebal) dan berat 0.16 kg, tas ini memiliki ruang penyimpanan besar dan satu kantong tambahan. Dilengkapi dengan tali belakang sepanjang 132

	cm, tas ini cocok digunakan di sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, serta berbagai acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item>
65	<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam pilihan warna Off Putih, Hitam, Coklat, dan Aprikot. Dengan dimensi 22.5 cm (Lebar) × 26.5 cm (Tinggi) × 3.5 cm (Tebal) dan berat 0.28 kg, tas ini dilengkapi dengan ruang penyimpanan besar, satu kantong, dan satu kantong resleting. Tali belakangnya memiliki panjang antara 103-124 cm. Tas ini cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, serta acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantoran, pelajar, dan remaja.</item>
66	<item>Selamat datang di produk MOSSDOOM, dijamin keasliannya 100%! Tas ini terbuat dari bahan kulit PU. Dengan ukuran panjang 19.5 cm, lebar 8 cm, tinggi 16.5 cm, dan dengan berat 0.36 kg. Tersedia hanya warna Pink. Tas ini dilengkapi dengan satu saku penyimpanan besar, satu saku sisipan, satu saku beritsleting, dan tinggi pegangan tas tangan sebesar 7cm. Untuk tali bahu memiliki panjang antara 47.5-59cm (dapat disesuaikan). Semua gambar produk ini diambil dalam kehidupan nyata, jadi dijamin keaslanya.</item>
67	<item>Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Berukuran 20 cm (Panjang) × 8 cm (Lebar) × 14 cm (Tinggi) dan berat 0.36 kg. Tas ini tersedia dalam pilihan warna Blue, Light Pink, Pink, Brown, Beige White, Black. Tas ini dilengkapi dengan ruang penyimpanan besar, satu saku sisipan, satu saku beresleting, dan dengan tali bahu setinggi kira-kira 106-130cm (yang dapat disesuaikan, bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya.</item>
68	<item>Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Dengan ukuran 17 cm (Panjang) × 9 cm (Lebar) × 10 cm (Tinggi) dan berat 0.29 kg. Tersedia dengan beberapa pilhan warna Pink, Blue, Purple, Apricot, Beige White, Black. Tas ini memiliki struktur internal dengan saku penyimpanan besar, tinggi tali bahunya

	sekitar 110-121 cm (yang dapat disesuaikan dan bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya.</item>
69	</string-array>
70	
71	<string name="label_nama">Nama</string>
72	<string name="label_harga">Harga:</string>
73	
74	<string name="placeholder_nama">Nama Tas</string>
75	<string name="placeholder_harga">Harga Tas</string>
76	</resources>

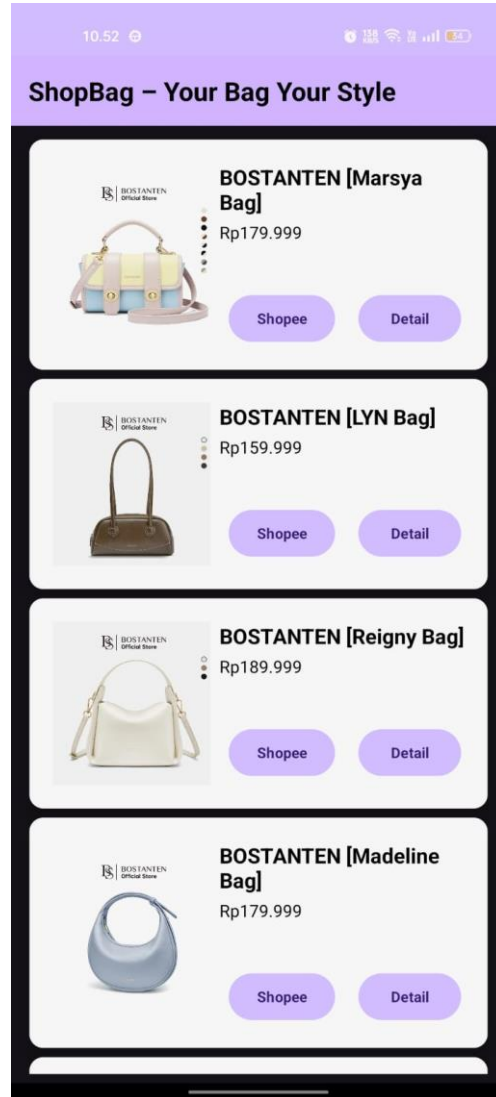
Tabel 29. Source Code Jawaban Soal 1 Modul 4

13. activity_main.xml

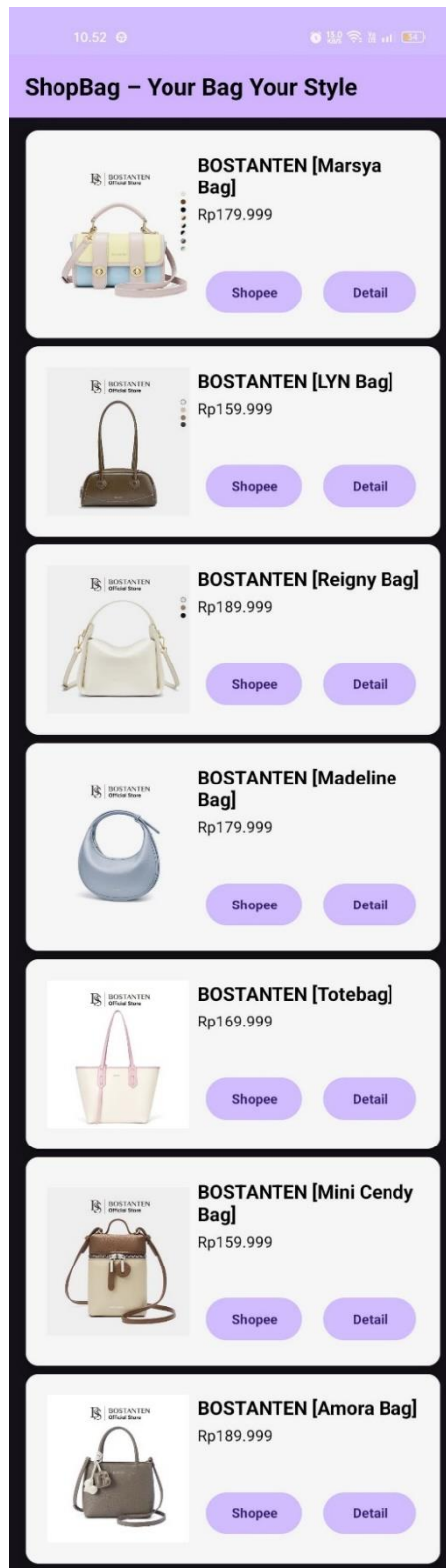
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:tools="http://schemas.android.com/tools"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity"
9	tools:ignore="ExtraText">
10	
11	<androidx.fragment.app.FragmentContainerView
12	android:id="@+id/nav_host_fragme
13	android:name="androidx.navigation.fragment.NavHostFragment"
14	android:layout_width="match_parent"
15	android:layout_height="match_parent"
16	app:navGraph="@navigation/nav_graph"
17	app:defaultNavHost="true" />
18	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 30. Source Code Jawaban Soal 1 Modul 4

B. Output Program



Gambar 21. Screenshot Hasil UI List Soal 1 Modul 4



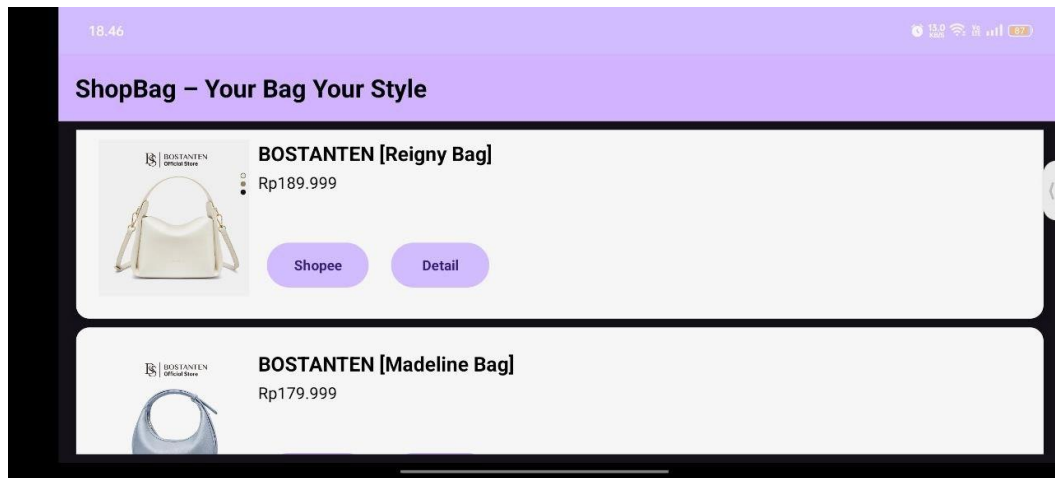
Gambar 22. Screenshot Hasil UI List 10 Item Soal 1 Modul 4



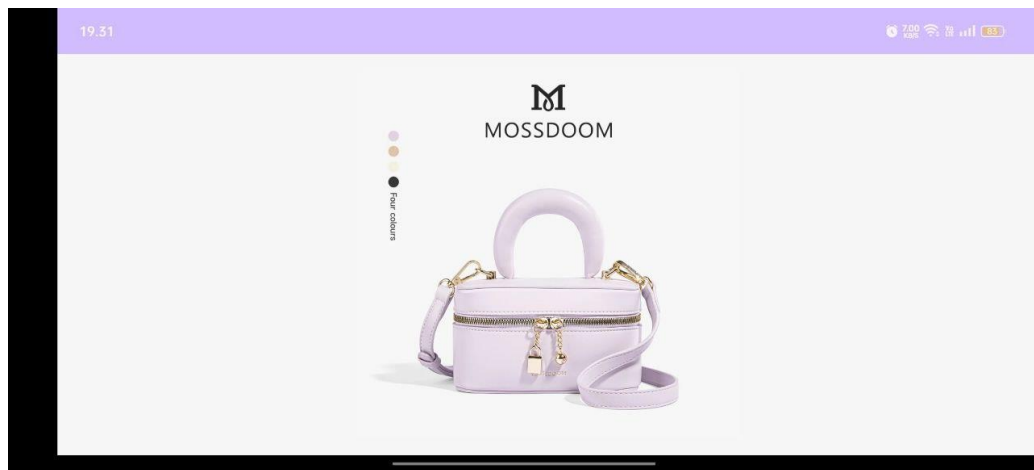
Gambar 23. Screenshot Hasil Berpindah ke Aplikasi Lain Saat Menekan Tombol Shopee Soal 1 Modul 4



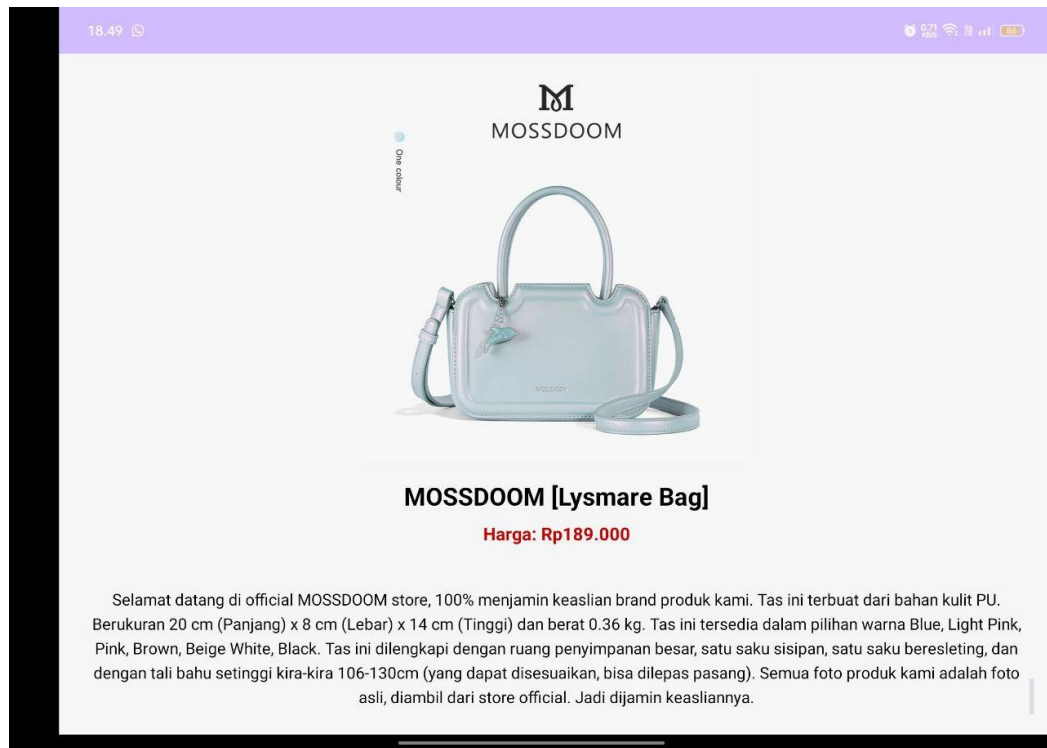
Gambar 24. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 4



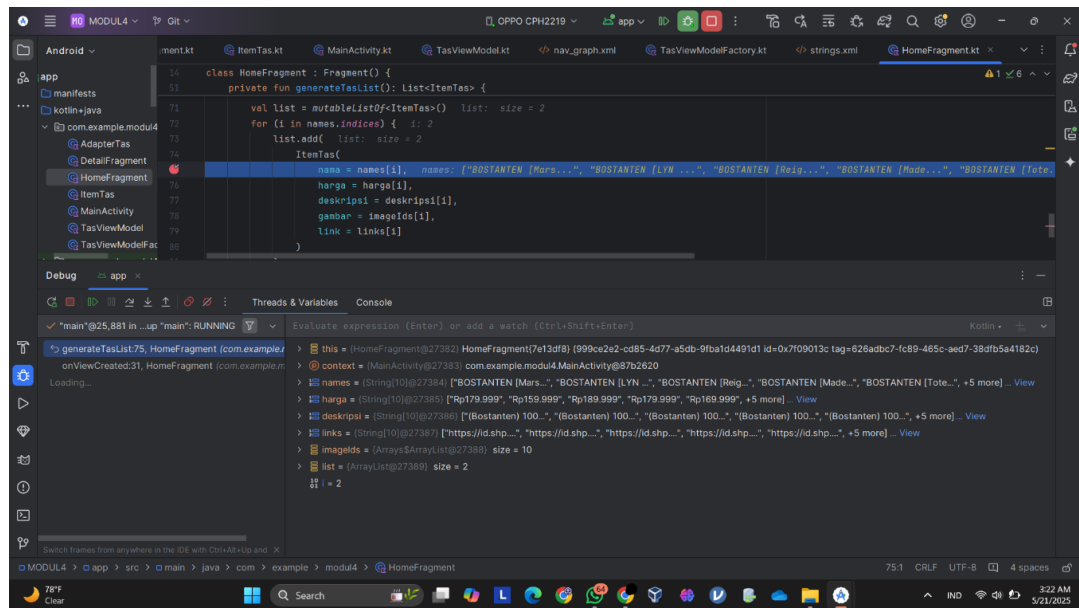
Gambar 25. Screenshot Hasil UI List Item Rotate Soal 1 Modul 4



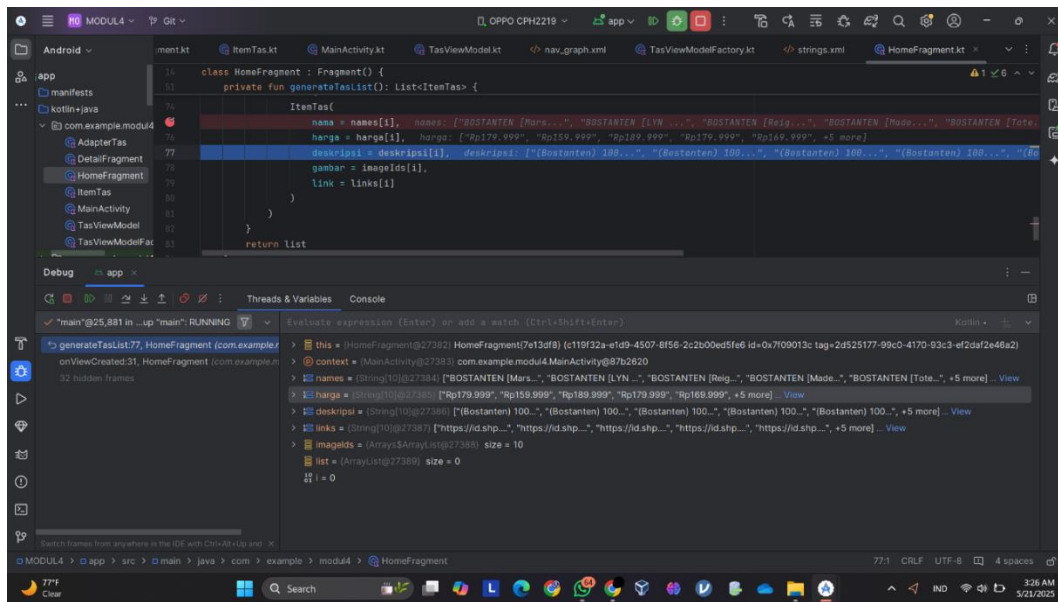
Gambar 26. Screenshot Hasil UI Detail Item Rotate Soal 1 Modul 4



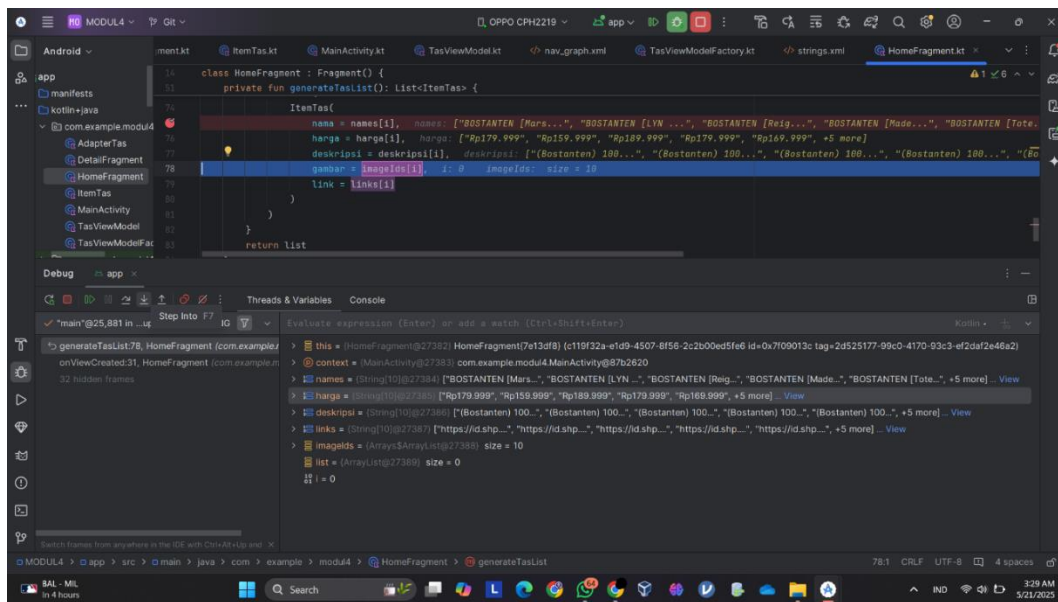
Gambar 27. Screenshot Hasil UI Detail Item Rotate dan Scrollable Soal 1 Modul 4



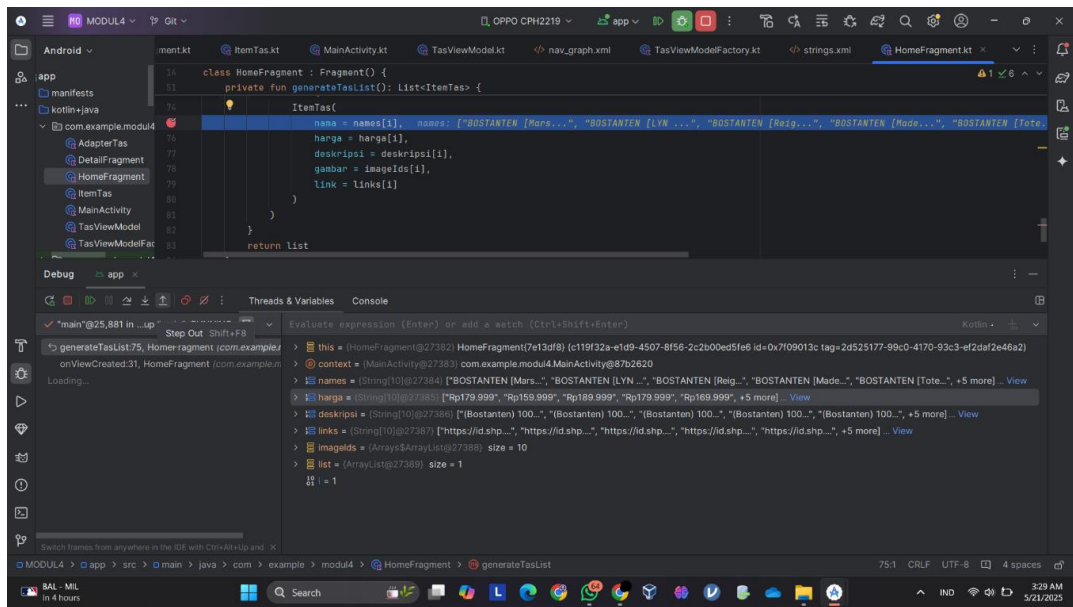
Gambar 28. Screenshot Hasil Debugging Aplikasi Soal 1 Modul 4



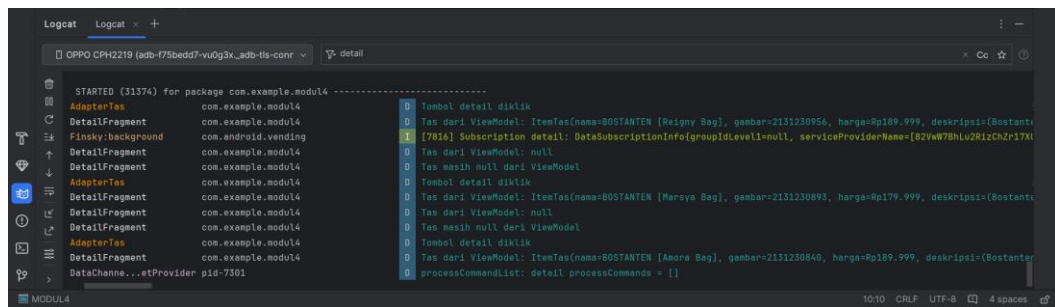
Gambar 29. Screenshot Hasil Debugging Aplikasi Step Over Soal 1 Modul 4



Gambar 30. Screenshot Hasil Debugging Aplikasi Step Into Soal 1 Modul 4



Gambar 31. Screenshot Hasil Debugging Aplikasi Step Out Soal 1 Modul 4



Gambar 32. Screenshot Hasil Logging Soal 1 Modul 4

C. Pembahasan

1. HomeFragment.kt

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 13, class-class penting yang digunakan seperti `Fragment`, `LayoutInflater`, `RecyclerView`, `binding` untuk layout `FragmentHome`, dan lain sebagainya. Lalu, line 15 sampai 19 membuat fragment bernama `HomeFragment` yang merupakan turunan dari `Fragment`. `recyclerViewState` disiapkan untuk menyimpan posisi scroll `RecyclerView`. `viewModel` ialah untuk referensi ke `TasViewModel` yang menyimpan data tas dan navigasi. `_binding` digunakan untuk mengakses elemen layout dari `FragmentHomeBinding`. `binding` getter yang memastikan `_binding` tidak null (pakai `!!`). Line 21 sampai 27, membuat tampilan `Fragment` dari layout XML (`fragment_home.xml`) menggunakan `View Binding`. Line 29 sampai 49 terdapat fungsi `onViewCreated` untuk memulai penggunaan UI. Juga terdapat inisialisasi `ViewModel`, `ViewModelProvider` digunakan untuk mendapatkan instance `TasViewModel` agar datanya tetap bertahan saat fragment berpindah-pindah dan `requireActivity()` artinya `ViewModel` milik `activity` (bisa diakses banyak fragment). Pada saat memuat data akan melakukan pemanggilan fungsi `loadData()` untuk mengisi list tas ke dalam `StateFlow` di `ViewModel`, `generateTasList()` adalah fungsi yang membuat data dummy dari resource XML. Di line tersebut juga terdapat fungsi untuk menampilkan `RecyclerView`, menunggu data dari `viewModel.tasList` (yang berupa `StateFlow`). Saat data muncul, maka akan membuat `AdapterTas` baru dan memasukkan daftar item, menyambungkan `RecyclerView` ke adapter, dan mengatur layout-nya sebagai daftar vertikal (`LinearLayoutManager`). `Callback item -> viewModel.selectTas(item)` digunakan untuk memilih

item yang akan ditampilkan di detail (disimpan di ViewModel). Pada navigasi ke DetailFragment, melakukan pemantauan event `navigateToDetail` dari ViewModel. Jika event terjadi (misalnya item diklik), dan saat ini masih di HomeFragment, maka akan berpindah ke DetailFragment lewat Navigation Component.

Selanjutnya, pada line 51 sampai 54 terdapat method `onDestroyView`, yang di mana akan membersihkan View Binding agar tidak terjadi memory leak ketika fragment dihancurkan. Pada line 56 sampai 61 terdapat fungsi `generateTasList()`, fungsi untuk membuat list dummy tas dari `res/values/strings.xml` dan `res/drawable/`. Fungsi ini melakukan penggabungan nama tas, harga, deskripsi, gambar, dan link produk. Line 63 sampai 74, menyediakan daftar ID gambar tas dalam bentuk list. Line 76 sampai 90, menggabungkan data teks dan gambar menjadi objek `ItemTas`, lalu dikembalikan sebagai list yang akan ditampilkan di `RecyclerView`.

2. DetailFragment.kt

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 12, class-class penting yang digunakan seperti `lifecycle`, `fragment`, `log`, `debugging`, `View Binding`, dan lain sebagainya. Lalu, line 14 sampai 17 mendefinisikan fragment detail bernama `DetailFragment` sebagai turunan dari `Fragment`, yang di dalamnya terdapat `viewModel` instance dari `TasViewModel`, digunakan untuk mengambil data tas yang dipilih. `_binding` untuk View Binding dari layout `fragment_detail.xml`, dan `binding` akses yang aman (non-null) ke `_binding`. Line 19 sampai 25 membuat dan mengembalikan tampilan (UI) fragment. Layout di-inflate menggunakan View Binding, sehingga bisa langsung akses elemen UI dari `fragment_detail.xml` lewat `binding`.

Line 27 sampai 43 terdapat penginisialan ViewModel dengan mengambil TasViewModel milik Activity agar bisa berbagi data dengan HomeFragment. Mengamati data `selectedTas` (kemungkinan `StateFlow<ItemTas?>`). Saat tas dipilih, akan otomatis masuk ke sini. Jika tas tidak kosong, maka akan menampilkan nama tas ke `tvName`, harga ke `tvHarga`, deskripsi ke `tvDeskripsi`, dan gambar ke `imgItemPhoto`. Selain itu, terdapat logging ntuk debugging di Logcat, mengecek apakah data tas benar-benar diterima. Line 45 sampai 50, `viewModel.clearSelectedTas()` digunakan untuk membersihkan data `selectedTas`, agar ketika kembali ke Home, data sebelumnya tidak nyangkut. `_binding = null` digunakan untuk mencegah memory leak.

3. ItemTas.kt

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Line 3 dan 4 mengimpor `Parcelable`, yang di mana itu interface untuk mengirim data antar komponen Android (seperti antar fragment atau activity) dan `@Parcelize` adalah annotation dari Kotlin untuk membuat proses parceling otomatis tanpa harus menulis `writeToParcel()` dan `CREATOR`. Lalu, line 6 sampai 13 membuat class bisa dikirim lewat Bundle dengan mudah dengan cara `@Parcelize`. `data class` ialah class khusus di Kotlin untuk menyimpan data dengan otomatis menyediakan fungsi seperti `equals()`, `hashCode()`, dan `toString()`. `ItemTas` menyimpan informasi untuk 1 item tas, nama untuk nama produknya, gambar untuk ID resource drawable gambar produk (`Int`), harga untuk harga produk (sebagai string, bisa mengandung Rp, titik, dsb), deskripsi untuk penjelasan produk di halaman detail, dan `link` adalah URL untuk ke Shopee atau marketplace lainnya. Fungsi utama dari class ini digunakan untuk menyimpan data tas dalam daftar

dan mengirim data ke `DetailFragment` melalui `Bundle` secara aman dan efisien.

4. `AdapterTas.kt`

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 11, class-class penting yang digunakan. Lalu, line 13 sampai 16 mendefinisikan class `AdapterTas`. `AdapterTas` menerima dua parameter, `listTas` ialah daftar objek `ItemTas` untuk ditampilkan dan `onItemClick` lambda function untuk menangani klik tombol detail. Fungsi ini akan dipanggil di `HomeFragment` untuk navigasi ke `DetailFragment`. Adapter ini menghubungkan data dengan tampilan di `RecyclerView`. Line 18 dan 19, inner class `ViewHolder` merepresentasikan tiap item di dalam daftar `RecyclerView` dengan menggunakan View Binding (`ItemTasBinding`) untuk mengakses elemen UI. Line 21 sampai 24, mengisi teks dan gambar dari objek `ItemTas` ke komponen layout (`TextView`, `ImageView`) dan menampilkan nama, harga, dan gambar tas ke tampilan masing-masing.

Line 25 sampai 28 terdapat proses ketika tombol Detail diklik, saat tombol “Detail” diklik, fungsi `onItemClick(tas)` dipanggil. Fungsi ini didefinisikan di `HomeFragment`, yang kemudian akan memicu navigasi ke `DetailFragment`. Line 29 sampai 35 melakukan proses pada tombol Shopee, ketika tombol Shopee diklik, maka intent dibuka untuk mengunjungi link eksternal (ke halaman Shopee). Line 37 sampai 40, membuat `ViewHolder` baru dari layout XML `item_tas.xml`. Line 42 sampai 44, mengisi data ke `ViewHolder` berdasarkan posisi dalam list. Line 46 dan 47, mengembalikan jumlah total item dalam list, dengan menentukan jumlah item yang akan ditampilkan di `RecyclerView`.

5. MainActivity.kt

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 7, mengimpor kelas yang dibutuhkan, diantaranya yaitu `Bundle` untuk menyimpan state saat Activity dibuat, `AppCompatActivity` ialah superclass dari semua activity modern, dan `ActivityMainBinding` ialah class auto-generated dari layout `activity_main.xml` (fitur View Binding). Line 9 sampai 15, `ActivityMainBinding` ialah View Binding untuk `layout activity_main.xml`. `setContentView(binding.root)` untuk menampilkan layout utama yang diatur lewat View Binding. Dalam layout `activity_main.xml`, ada `FragmentContainerView` atau `NavHostFragment` dengan `id = nav_host_fragment`. Line 16 sampai 23 melakukan penanganan tombol kembali (`onBackPressed`). Ketika tombol back ditekan, maka akan mengecek apakah masih ada fragment di backstack navigasi. Jika masih ada, akan kembali ke fragment sebelumnya menggunakan `popBackStack()`. Jika tidak ada lagi, maka akan memanggil `super.onBackPressed()` untuk keluar dari aplikasi atau kembali ke activity sebelumnya.

6. TasViewModel.kt

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 sampai 7 terdapat pengimporan kelas-kelas yang dibutuhkan. Line 9 sampai 11, `_tasList` ialah flow internal (mutable) berisi list tas dan `tasList` ialah flow eksternal (immutable) yang diamati `HomeFragment`. Line 14 dan 15

melakukan penyimpanan 1 tas yang sedang dipilih user dan digunakan oleh `DetailFragment` untuk menampilkan detail. Line 17 dan 18 ialah event navigasi ke halaman detail, digunakan untuk memberitahu UI agar pindah ke `DetailFragment`. `SharedFlow` cocok untuk event satu-kali seperti navigasi (berbeda dengan `StateFlow` yang menyimpan nilai terakhir).

Pada line 20 sampai 28, menyimpan item yang dipilih ke `_selectedTas`, lalu mengirimkan sinyal event navigasi melalui `_navigateToDetail`. Line 30 sampai 33 melakukan pemuatan data tas, dengan memasukkan daftar tas ke dalam `StateFlow` dan dipanggil sekali saat `HomeFragment` muncul. Line 35 sampai 38 melakukan pembersihan item tas yang terpilih, dipanggil saat `DetailFragment` dihancurkan agar `ViewModel` tidak menyimpan referensi ke data lama.

7. `TasViewModelFactory.kt`

Pada line 1, `package com.example.modul4` digunakan untuk menentukan nama package dari aplikasi Android. Package ini adalah namespace unik untuk memisahkan file dalam aplikasi. Kemudian, pada line 3 dan 4 terdapat pengimporan kelas-kelas yang dibutuhkan. Line 6 sampai 13, `ViewModelProvider.Factory` adalah interface untuk membuat instance `ViewModel`. `create()` akan dipanggil oleh `ViewModelProvider` saat memanggil `isAssignableFrom()` digunakan untuk mengecek apakah `modelClass` adalah turunan dari `TasViewModel`. `return TasViewModel() as T` artinya akan membuat instance `TasViewModel`, lalu meng-cast-nya ke `T`.

8. `fragment_home.xml`

Pada line 1 sampai 8 terdapat `LinearLayout` yang di manakomponen dasar dengan orientasi vertikal untuk menyusun anak-anaknya dari atas ke bawah, `match_parent` digunakan untuk memenuhi seluruh lebar dan tinggi

layar, dan `tools:context` hanya untuk keperluan preview di Android Studio (menunjukkan layout ini dipakai oleh `HomeFragment`). Line 10 sampai 19, menampilkan judul aplikasi di bagian atas, dengan background ungu (`#D1B3FF`), teks hitam, ukuran besar 25sp, tebal, dan padding 15dp agar teks tidak terlalu mepet. Kemudian, line 21 sampai 27 ialah tempat untuk menampilkan daftar produk tas menggunakan `RecyclerView`, juga menggunakan seluruh sisa ruang layar di bawah `TextView`. id tersebut yang diakses di `HomeFragment.kt` untuk menampilkan daftar tas melalui adapter.

9. `item_tas.xml`

Pada line 1 sampai 14 terdapat `CardView` (root) untuk membungkus seluruh isi item agar terlihat seperti kartu dengan bayangan. `cardCornerRadius="12dp"` untuk mengatur sudut membulat dan `cardElevation="10dp"` untuk memberikan efek bayangan agar item terlihat menonjol. Line 16 sampai 20 terdapat `ConstraintLayout` (Container dalam Card), layout fleksibel untuk mengatur posisi komponen UI berdasarkan kait (constraint) terhadap komponen lain atau parent-nya. Kemudian, line 22 sampai 31 menampilkan foto produk, dengan ukuran tetap 135dp x 140dp, dan `scaleType="centerCrop"` agar gambar memenuhi tampilan dengan cropping bagian luar. Line 33 sampai 46, menampilkan nama tas. `textStyle="bold", textSize="20sp"` untuk mengatur teks nya agar nama terlihat jelas, dan ditempatkan di sebelah kanan gambar. Lalu, line 48 sampai 58 disitu berfungsi untuk menampilkan harga tas dan ditempatkan di bawah nama produk. Line 60 sampai 68 membuat tombol untuk berpindah ke marketplace Shopee. Dengan posisi di bawah harga dan di sebelah kiri. Constraint ke `@id/tv_item_price` (top) dan Constraint ke `@id/tv_item_name` (start). Line 70 sampai 79 membuat tombol untuk

navigasi ke halaman detail produk. Dengan posisi di sebelah kanan tombol Shopee, menyatu sejajar atasnya.

10. fragment_detail.xml

Line 1 sampai 9 terdapat `<ScrollView>` untuk membungkus seluruh isi layout agar bisa di-scroll jika kontennya lebih panjang dari layar. ini sangat berguna untuk tampilan detail yang bisa panjang, seperti pada deskripsi produk. Line 11 sampai 14, `ConstraintLayout` (di dalam `ScrollView`) digunakan untuk menyusun posisi elemen-elemen UI dengan fleksibel. Pada line 16 sampai 25 menampilkan gambar produk ukuran besar (350dp x 350dp). `centerCrop` agar gambar tetap proporsional memenuhi area dan diposisikan di bagian atas tengah layout. Kemudian, line 27 sampai 38 menampilkan nama produk dengan ukuran besar (25sp) dan bold. Diposisikan di bawah gambar dan disejajarkan ke tengah. Line 40 sampai 51, menampilkan harga dengan warna merah gelap (`holo_red_dark`) dan bold. Diposisikan di bawah nama produk dan tetap di tengah. Lalu, line 53 sampai 70 menampilkan deskripsi lengkap produk. `lineSpacingExtra="5dp"` mengatur agar teks lebih nyaman dibaca. `textAlignment="center"` digunakan untuk isi teks diratakan ke tengah. `padding` dan `marginTop` cukup besar untuk memberi jarak visual. Serta lebar penuh (`match_parent`) agar deskripsi tidak terpotong.

11. nav_graph.xml

Pada line 1 sampai 5, menentukan bahwa fragment pertama kali yang ditampilkan saat app berjalan adalah `HomeFragment`. Line 7 sampai 11, mewakili untuk tampilan utama (daftar tas). Fragment ini berada di package `com.example.modul3`. Layout yang digunakan ialah `fragment_home.xml`. Line 12 sampai 15, mendefinisikan rute navigasi dari `HomeFragment` ke `DetailFragment`, hal ini akan dipicu saat user klik tombol “Detail” pada item tas. Line 17 sampai 22 adalah fragment untuk

menampilkan detail dari item tas yang dipilih, dengan menggunakan layout `fragment_detail.xml`.

12. strings.xml

Pada line 1 sampai 4 terdapat `<resources>`, yang di mana semua string dan array harus berada di dalam tag `<resources>` tersebut. Fungsi-fungsinya terdiri dari `app_name` itu nama aplikasi. `btn_detail` dan `btn_shopee` adalah label pada tombol navigasi atau detail dan tombol ke Shopee. `label_nama` dan `label_harga` untuk label yang muncul pada halaman detail produk. `placeholder_nama` dan `placeholder_harga` ialah placeholder atau teks default saat input kosong atau belum ada data. Line 6 sampai 17 terdapat `<string-array name="data_name">`, yang di mana pada array string tersebut berisi nama-nama tas dari dua merek, yaitu BOSTANTEN dan MOSSDOOM. Kemudian, line 19 sampai 30 array string untuk `data_photo`, yang berisi referensi ke gambar tas di folder `res/drawable`, dengan menggunakan notasi `@drawable/nama_gambar`. Line 32 sampai 43 terdapat array string untuk `data_link`, link akan menuju produk yang bersangkutan di marketplace Shopee. Lalu, line 45 sampai 56 terdapat array string `data_harga`, yang di mana akan menampilkan harga produk dalam Rupiah. Line 58 sampai 76 terdapat array string `data_deskripsi`, setiap `<item>` disitu menjelaskan fitur, ukuran, warna, dan target pengguna dari tas. Informasi tersebut sangat lengkap dan berorientasi pada e-commerce (penjelasan produk seperti di Shopee).

13. activity_main.xml

Pada line 1 sampai 9, `ConstraintLayout` adalah layout utama yang digunakan di sini untuk mengatur posisi elemen UI di dalamnya. `ConstraintLayout` memungkinkan penataan elemen UI dengan

fleksibilitas tinggi melalui constraint (penghubung). `xmlns:android` adalah deklarasi namespace untuk atribut Android standar. `xmlns:tools` digunakan oleh Android Studio untuk memberikan data atau tips visual saat pengembangan tanpa mempengaruhi aplikasi yang sebenarnya. `tools:context=".MainActivity"` memberikan konteks di mana layout ini digunakan (dalam hal ini, `MainActivity`). `tools:ignore="ExtraText"` memberikan petunjuk kepada Android Studio untuk mengabaikan peringatan tertentu yang tidak penting (misalnya teks ekstra).

Kemudian, line 11 sampai 18 terdapat `FragmentContainerView`, yang di mana komponen ini digunakan untuk menampilkan dan mengelola fragment di dalam activity. `FragmentContainerView` menggantikan penggunaan `FrameLayout` untuk penempatan fragment dan lebih terintegrasi dengan `Navigation Component` Android. `android:id="@+id/nav_host_fragme"` memberikan ID untuk `FragmentContainerView`, yang nantinya bisa digunakan untuk referensi dalam kode. `android:name="androidx.navigation.fragment.NavHostFragment"` untuk menyatakan bahwa fragment yang akan ditampilkan di dalam `FragmentContainerView` adalah `NavHostFragment`, yang merupakan bagian dari `Navigation Component`. Fragment ini bertugas mengelola navigasi antar fragment. `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk membuat `FragmentContainerView` mengambil ruang penuh dari layar, dengan lebar dan tinggi `match_parent`, sehingga mengisi seluruh layout. `app:navGraph="@navigation/nav_graph"` untuk menyediakan `NavGraph` yang berisi alur navigasi antar fragment. `@navigation/nav_graph` adalah referensi ke file `nav_graph.xml`,

yang mendefinisikan destinasi dan aksi antar fragment dalam aplikasi. `app:defaultNavHost="true"` digunakan untuk menandakan bahwa `NavHostFragment` ini adalah host utama untuk navigasi di activity ini. Artinya, fragment ini akan menangani back stack navigasi, dan tombol back perangkat akan berfungsi untuk kembali ke fragment sebelumnya (jika ada).

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya!

A. Pembahasan

Application adalah kelas dasar dari `android.app.Application` yang merepresentasikan global application state. Kelas ini hanya dibuat satu kali selama siklus hidup aplikasi, sebelum aktivitas, service, atau komponen lainnya dibuat. Android secara otomatis akan membuat instance dari kelas ini sebelum komponen pertama aplikasi dimulai (seperti Activity, Service, BroadcastReceiver, atau ContentProvider).

Berikut beberapa fungsi dan peran Application class dalam arsitektur:

5. Inisialisasi Global

Tempat utama untuk menginisialisasi dependency global seperti Retrofit, Room, Firebase, atau Dagger/Hilt.

6. Dependency Injection (DI)

Dipakai untuk menyediakan dependensi ke berbagai komponen tanpa harus inisialisasi berulang.

7. Penyimpanan State Global

Menyimpan data yang harus hidup selama lifecycle aplikasi (jangan disalahgunakan untuk menyimpan UI state).

8. Konfigurasi Logging atau Monitoring

Ideal untuk menginisialisasi alat log & monitoring seperti Timber, Crashlytics, Sentry, dan sebagainya.

9. Akses Context Aplikasi

Karena Application adalah turunan dari Context, maka bisa menggunakannya di mana saja sebagai global context.

Application ini bekerja dengan cara Android Framework memanggil Application saat proses aplikasi dimulai. Hanya ada satu instance Application dalam satu proses aplikasi. Lalu, `onCreate()` dipanggil sekali, cocok untuk konfigurasi awal.

Sehingga, kelas ini tetap hidup selama proses aplikasi masih berjalan (kecuali aplikasi dihentikan oleh sistem atau user).

Referensi:

<https://developer.android.com/reference/android/app/Application>

<https://developer.android.com/guide/components/activities/activity-lifecycle?hl=id#app-startup>

<https://developer.android.com/topic/architecture?hl=id>

<https://stackoverflow.com/questions/3821784/whats-the-difference-between-n-and-r-n/3821817#3821817>

MODUL 5: CONNECT TO THE INTERNET

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

1. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
2. Gunakan KotlinX Serialization sebagai library JSON.
3. Gunakan library seperti Coil atau Glide untuk image loading.
4. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
5. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
6. Gunakan caching strategy pada Room.
7. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. app/src/main/java/com/example/modul5/data/api/ApiClient.kt

```
1 package com.example.modul5.data.api
2
3 import
4     com.jakewharton.retrofit2.converter.kotlinx.serialization.
5     asConverterFactory
6 import kotlinx.serialization.json.Json
7 import okhttp3.MediaType.Companion.toMediaType
8 import retrofit2.Retrofit
9
10 object ApiClient {
11     private const val BASE_URL =
12     "https://modul5.free.beeceptor.com"
13
14     private val json = Json { ignoreUnknownKeys = true }
15
16     val retrofit: TasApiService = Retrofit.Builder()
17         .baseUrl(BASE_URL)
18         .addConverterFactory(json.asConverterFactory
19         ("application/json".toMediaType()))
20         .build()
21         .create(TasApiService::class.java)
22 }
```

Tabel 31. Source Code Jawaban Soal 1 Modul 5

2. app/src/main/java/com/example/modul5/data/api/TasApiService.kt

```
1 package com.example.modul5.data.api
2
3 import com.example.modul5.domain.model.Tas
4 import retrofit2.http.GET
5
6 interface TasApiService {
7     @GET("data")
8     suspend fun getTas(): List<TasDto>
9 }
```

Tabel 32. Source Code Jawaban Soal 1 Modul 5

3. app/src/main/java/com/example/modul5/data/api/TasDto.kt

```
1 package com.example.modul5.data.api
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 data class TasDto(
```

7	val id: Int,
8	val nama: String,
9	val harga: String,
10	val gambar: String,
11	val shopeeLink: String,
12	val deskripsi: String
13	)

Tabel 33. Source Code Jawaban Soal 1 Modul 5

4. app/src/main/java/com/example/modul5/data/local/TasDao.kt

1	package com.example.modul5.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import com.example.modul5.data.local.TasEntity
8	import kotlinx.coroutines.flow.Flow
9	
10	@Dao
11	interface TasDao {
12	@Query("SELECT * FROM tas")
13	fun getAllTas(): Flow<List<TasEntity>>
14	
15	@Insert(onConflict = OnConflictStrategy.REPLACE)
16	suspend fun insertAll(tas: List<TasEntity>)
17	
18	@Query("UPDATE tas SET isFavorite = :isFav WHERE id =
19	:id")
20	suspend fun updateFavorite(id: Int, isFav: Boolean)
21	}

Tabel 34. Source Code Jawaban Soal 1 Modul 5

5. app/src/main/java/com/example/modul5/data/local/TasDatabase.kt

1	package com.example.modul5.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	
8	@Database(entities = [TasEntity::class], version = 1,
9	exportSchema = false)
10	abstract class TasDatabase : RoomDatabase() {
11	abstract fun tasDao(): TasDao

12	companion object {
13	@Volatile
14	private var INSTANCE: TasDatabase? = null
15	
16	fun getDatabase(context: Context): TasDatabase {
17	return INSTANCE ?: synchronized(this) {
18	val instance = Room.databaseBuilder(
19	context.applicationContext,
20	TasDatabase::class.java,
21	"tas_db"
22	).build()
23	INSTANCE = instance
24	instance
25	}
26	}
27	}
28	}

Tabel 35. Source Code Jawaban Soal 1 Modul 5

6. app/src/main/java/com/example/modul5/data/local/TasEntity.kt

1	package com.example.modul5.data.local
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "tas")
7	data class TasEntity(
8	@PrimaryKey val id: Int,
9	val nama: String,
10	val harga: String,
11	val gambar: String,
12	val shopeeLink: String,
13	val deskripsi: String,
14	val isFavorite: Boolean = false
15	)

Tabel 36. Source Code Jawaban Soal 1 Modul 5

7. app/src/main/java/com/example/modul5/data/mapper/TasMapper.kt

1	package com.example.modul5.data.mapper
2	
3	import com.example.modul5.data.api.TasDto
4	import com.example.modul5.data.local.TasEntity
5	import com.example.modul5.domain.model.Tas
6	
7	fun TasDto.toEntity(): TasEntity {
8	return TasEntity(

9	id = this.id,
10	nama = this.nama,
11	harga = this.harga,
12	gambar = this.gambar,
13	shopeeLink = this.shopeeLink,
14	deskripsi = this.deskripsi,
15	isFavorite = false // default
16	)
17	}
18	
19	fun TasEntity.toDomain(): Tas {
20	return Tas(
21	id = this.id,
22	nama = this.nama,
23	harga = this.harga,
24	gambar = this.gambar,
25	shopeeLink = this.shopeeLink,
26	deskripsi = this.deskripsi,
27	isFavorite = this.isFavorite
28	)
29	}

Tabel 37. Source Code Jawaban Soal 1 Modul 5

8. app/src/main/java/com/example/modul5/data/repository/TasRepository.kt

1	package com.example.modul5.data.repository
2	
3	import android.util.Log
4	import com.example.modul5.domain.model.Tas
5	import com.example.modul5.data.api.TasApiService
6	import com.example.modul5.data.local.TasDao
7	import com.example.modul5.data.mapper.toDomain
8	import com.example.modul5.data.mapper.toEntity
9	import kotlinx.coroutines.flow.Flow
10	import kotlinx.coroutines.flow.firstOrNull
11	import kotlinx.coroutines.flow.map
12	
13	class TasRepository(
14	private val api: TasApiService,
15	private val dao: TasDao
16	) {
17	val tasList: Flow<List<Tas>> = dao.getAllTas().map {
18	list ->
19	list.map { it.toDomain() }
20	}
21	suspend fun refreshFromApi() {

22	val response = api.getTas()
23	val existing = dao.getAllTas().firstOrNull()
24	if (existing.isNullOrEmpty()) {
25	dao.insertAll(response.map { it.toEntity() })
26	}
27	}
28	
29	suspend fun toggleFavorite(id: Int, current: Boolean)
30	{
31	dao.updateFavorite(id, !current)
32	}

Tabel 38. Source Code Jawaban Soal 1 Modul 5

9. app/src/main/java/com/example/modul5/domain/model/Tas.kt

1	package com.example.modul5.domain.model
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	import kotlinx.serialization.Serializable
6	import kotlinx.serialization.Transient
7	
8	@Serializable
9	@Parcelize
10	data class Tas(
11	val id: Int,
12	val nama: String,
13	val harga: String,
14	val gambar: String,
15	val shopeeLink: String,
16	val deskripsi: String,
17	@Transient val isFavorite: Boolean = false
18	) : Parcelable

Tabel 39. Source Code Jawaban Soal 1 Modul 5

10. app/src/main/java/com/example/modul5/ui/adapter/AdapterTas.kt

1	package com.example.modul5.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import coil.load
7	import com.example.modul5.R
8	import com.example.modul5.databinding.ItemTasBinding
9	import com.example.modul5.domain.model.Tas
10	


```

11 class AdapterTas(
12     private val onDetailClick: (Tas) -> Unit,
13     private val onShopeeClick: (Tas) -> Unit,
14     private val onFavoriteClick: (Tas) -> Unit
15 ) : RecyclerView.Adapter<AdapterTas.TasViewHolder>() {
16     private var tasList: List<Tas> = emptyList()
17
18     fun updateList(newList: List<Tas>) {
19         tasList = newList
20         notifyDataSetChanged()
21     }
22     inner class TasViewHolder(val binding: ItemTasBinding)
23 : RecyclerView.ViewHolder(binding.root)
24
25     override fun onCreateViewHolder(parent: ViewGroup,
26 viewType: Int): TasViewHolder {
27         val binding = ItemTasBinding.inflate(LayoutInflater.from(parent.context),
28 parent, false)
29         return TasViewHolder(binding)
30     }
31
32     override fun onBindViewHolder(holder: TasViewHolder,
33 position: Int) {
34         val tas = tasList[position]
35         with(holder.binding) {
36             tvItemName.text = tas.nama
37             tvItemPrice.text = tas.harga
38             imgItemPhoto.load(tas.gambar)
39             btnFavorite.setImageResource(
40                 if (tas.isFavorite)
41                 R.drawable.ic_love_filled else R.drawable.ic_love_border
42             )
43             btnFavorite.setOnClickListener {
44                 onFavoriteClick(tas) }
45             buttonShopee.setOnClickListener {
46                 onShopeeClick(tas) }
47             buttonDetail.setOnClickListener {
48                 onDetailClick(tas) }
49         }
50     }
51
52     override fun getItemCount(): Int = tasList.size
53 }

```

Tabel 40. Source Code Jawaban Soal 1 Modul 5

**11. app/src/main/java/com/example/modul5/ui/adapter/HomePagerAdapter.
kt**

1	package com.example.modul5.ui.adapter
2	
3	import androidx.fragment.app.Fragment
4	import androidx.viewpager2.adapter.FragmentStateAdapter
5	import com.example.modul5.ui.home.tabs.AllProductFragment
6	import com.example.modul5.ui.home.tabs.FavoriteFragment
7	
8	class HomePagerAdapter(fragment: Fragment) : FragmentStateAdapter(fragment) {
9	
10	override fun getItemCount(): Int = 2
11	
12	override fun createFragment(position: Int): Fragment {
13	return when (position) {
14	0 -> AllProductFragment()
15	1 -> FavoriteFragment()
16	else -> throw
	IllegalArgumentExpection("Invalid position: \$position")
17	}
18	}
19	}

Tabel 41. Source Code Jawaban Soal 1 Modul 5

12. app/src/main/java/com/example/modul5/ui/detail/DetailFragment.kt

1	package com.example.modul5.ui.detail
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Toast
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.fragment.app.Fragment
11	import androidx.fragment.app.activityViewModels
12	import androidx.lifecycle.ViewModelProvider
13	import androidx.lifecycle.LifecycleScope
14	import coil.load
15	import com.example.modul5.R
16	import
	com.example.modul5.databinding.FragmentDetailBinding
17	import com.example.modul5.viewmodel.TasViewModel
18	import kotlinx.coroutines.flow.collectLatest
19	import kotlinx.coroutines.launch
20	import androidx.navigation.fragment.findNavController
21	import com.example.modul5.viewmodel.TasViewModelFactory
22	
23	class DetailFragment : Fragment() {

```

24     private val viewModel: TasViewModel by
activityViewModels {
25         TasViewModelFactory(requireContext())
26     }
27     private var _binding: FragmentDetailBinding? = null
28     private val binding get() = _binding!!
29
30     override fun onCreateView(
31         inflater: LayoutInflater, container: ViewGroup?,
32         savedInstanceState: Bundle?
33     ): View {
34         _binding =
FragmentDetailBinding.inflate(inflater, container, false)
35         return binding.root
36     }
37
38     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
39         super.onViewCreated(view, savedInstanceState)
40         val activity = requireActivity() as
AppCompatActivity
41         activity.supportActionBar(binding.toolbar)
42         activity.supportActionBar?.apply {
43             setDisplayHomeAsUpEnabled(true)
44             title = "ShopBag - Your Bag Your Style"
45         }
46
47         binding.toolbar.setNavigationOnClickListener {
48             findNavController().navigateUp()
49         }
50
51         lifecycleScope.launch {
52             viewModel.selectedTas.collectLatest { tas ->
53                 if (tas != null) {
54                     binding.tvName.text = tas.nama
55                     binding.tvHarga.text =
getString(R.string.label_harga) + ": " + tas.harga
56                     binding.tvDeskripsi.text =
tas.deskripsi
57                     binding.imgItemPhoto.load(tas.gambar)
58                 }
59                 placeholder(R.drawable.placeholder)
60                 error(R.drawable.ic_error)
61             }
62         } else {
63             findNavController().navigateUp()
64         }
65     }

```

66	}
67	}
68	
69	override fun onDestroyView() {
70	super.onDestroyView()
71	viewModel.clearSelectedTas()
72	_binding = null
73	}
	}

Tabel 42. Source Code Jawaban Soal 1 Modul 5

13. app/src/main/java/com/example/modul5/ui/home/HomeFragment.kt

1	package com.example.modul5.ui.home
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.*
6	import androidx.fragment.app.Fragment
7	import androidx.fragment.app.activityViewModels
8	import androidx.lifecycle.lifecycleScope
9	import com.example.modul5.R
10	import androidx.navigation.fragment.findNavController
11	import com.example.modul5.databinding.FragmentHomeBinding
12	import com.example.modul5.ui.adapter.HomePagerAdapter
13	import com.example.modul5.viewmodel.TasViewModel
14	import com.example.modul5.viewmodel.TasViewModelFactory
15	import com.google.android.material.tabs.TabLayoutMediator
16	import kotlinx.coroutines.launch
17	
18	class HomeFragment : Fragment() {
19	private val viewModel: TasViewModel by
	activityViewModels {
20	TasViewModelFactory(requireContext())
21	}
22	
23	private var _binding: FragmentHomeBinding? = null
24	private val binding get() = _binding!!
25	
26	private lateinit var adapter: HomePagerAdapter
27	
28	override fun onCreateView(
29	inflater: LayoutInflater, container: ViewGroup?,
30	savedInstanceState: Bundle?
31	): View {
32	_binding = FragmentHomeBinding.inflate
	(inflater, container, false)
33	return binding.root
34	}

35	
36	override fun onCreateView(view: View,
37	savedInstanceState: Bundle?) {
38	super.onCreateView(view, savedInstanceState)
39	adapter = HomePagerAdapter(this)
40	binding.viewPager.adapter = adapter
41	
42	TabLayoutMediator(binding.tabLayout,
43	binding.viewPager) { tab, position ->
44	tab.text = if (position == 0) "All Product" else
45	"Favorite"
46	}.attach()
47	
48	viewLifecycleOwner.lifecycleScope.launch {
49	viewModel.navigateToDetail.collect {
50	val selected = viewModel.selectedTas.value
51	if (selected != null) {
52	findNavController().navigate(R.id.action_
53	homeFragment_to_detailFragment)
54	} else {
55	Log.e("HomeFragment",
56	"navigateToDetail: selectedTas is null")
57	}
58	}
59	}
60	override fun onDestroyView() {
61	super.onDestroyView()
62	_binding = null
	}
	}

Tabel 43. Source Code Jawaban Soal 1 Modul 5

14. app/src/main/java/com/example/modul5/ui/home/tabs/AllProductFragement.kt

1	package com.example.modul5.ui.home.tabs
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.*
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.lifecycleScope

```

10 import androidx.recyclerview.widget.LinearLayoutManager
11 import
   com.example.modul5.databinding.FragmentAllProductBinding
12 import com.example.modul5.ui.adapter.AdapterTas
13 import com.example.modul5.viewmodel.TasViewModel
14 import com.example.modul5.viewmodel.TasViewModelFactory
15 import kotlinx.coroutines.launch
16
17 class AllProductFragment : Fragment() {
18     private var _binding: FragmentAllProductBinding? =
null
19     private val binding get() = _binding!!
20     private val viewModel: TasViewModel by
activityViewModels {
21         TasViewModelFactory(requireContext()) }
22     private lateinit var adapter: AdapterTas
23
24     override fun onCreateView(inflater: LayoutInflater,
container: ViewGroup?, savedInstanceState: Bundle?): View
{
25         _binding
FragmentAllProductBinding.inflate(inflater, container,
false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
30         adapter = AdapterTas(
31             onDetailClick = { viewModel.selectTas(it) },
32             onShopeeClick = {
33                 val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(it.shopeeLink))
34                 startActivity(intent)
35             },
36             onFavoriteClick = {
viewModel.toggleFavorite(it) }
37         )
38         binding.recyclerViewAll.apply {
39             layoutManager
LinearLayoutManager(requireContext())
40             adapter = this@AllProductFragment.adapter
41         }
42         viewLifecycleOwner.lifecycleScope.launch {
43             viewModel.tasList.collect { list ->
44                 adapter.updateList(list)
45             }
46         }
47     }

```

48	
49	override fun onDestroyView() {
50	super.onDestroyView()
51	_binding = null
52	}
53	}

Tabel 44. Source Code Jawaban Soal 1 Modul 5

15. app/src/main/java/com/example/modul5/ui/home/tabs/FavoriteFragment.kt

1	package com.example.modul5.ui.home.tabs
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.*
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.lifecycleScope
10	import androidx.recyclerview.widget.LinearLayoutManager
11	import
	com.example.modul5.databinding.FragmentFavoriteBinding
12	import com.example.modul5.ui.adapter.AdapterTas
13	import com.example.modul5.viewmodel.TasViewModel
14	import com.example.modul5.viewmodel.TasViewModelFactory
15	import kotlinx.coroutines.launch
16	
17	class FavoriteFragment : Fragment() {
18	private var _binding: FragmentFavoriteBinding? = null
19	private val binding get() = _binding!!
20	private val viewModel: TasViewModel by
	activityViewModels {
21	TasViewModelFactory(requireContext()) }
22	private lateinit var adapter: AdapterTas
23	
24	override fun onCreateView(inflater: LayoutInflater,
	container: ViewGroup?, savedInstanceState: Bundle?): View
	{
25	_binding =
	FragmentFavoriteBinding.inflate(inflater, container,
	false)
26	return binding.root
27	}
28	
29	override fun onViewCreated(view: View,
	savedInstanceState: Bundle?) {
30	adapter = AdapterTas(

31	onDetailClick = { viewModel.selectTas(it) },
32	onShopeeClick = {
33	val intent = Intent(Intent.ACTION_VIEW,
34	Uri.parse(it.shopeeLink))
35	startActivity(intent)
36	},
37	onFavoriteClick = {
38	viewModel.toggleFavorite(it) }
39	)
40	binding.recyclerViewFavorite.apply {
41	layoutManager
42	LinearLayoutManager(requireContext())
43	adapter = this@FavoriteFragment.adapter
44	}
45	viewLifecycleOwner.lifecycleScope.launch {
46	viewModel.tasList.collect { list ->
47	val favorites = list.filter {
48	it.isFavorite }
49	adapter.updateList(favorites)
50	}
51	}
52	override fun onDestroyView() {
53	super.onDestroyView()
54	_binding = null
55	}
56	}

Tabel 45. Source Code Jawaban Soal 1 Modul 5

16. app/src/main/java/com/example/modul5/viewmodel/TasViewModel.kt

1	package com.example.modul5.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.modul5.domain.model.Tas
6	import com.example.modul5.data.repository.TasRepository
7	import kotlinx.coroutines.flow.*
8	import kotlinx.coroutines.launch
9	
10	class TasViewModel(
11	private val repository: TasRepository
12	) : ViewModel() {
13	val tasList: StateFlow<List<Tas>> =
14	repository.tasList
15	.onEach { println("ViewModel: Jumlah data dari
16	Room = \${it.size}") }

16	.stateIn(viewModelScope,	
	SharingStarted.Lazily, emptyList())	
17	private val _selectedTas	=
	MutableStateFlow<Tas?>(null)	
18	val selectedTas: StateFlow<Tas?> = _selectedTas	
19	private val _navigateToDetail	=
	MutableSharedFlow<Unit>()	
20	val navigateToDetail: SharedFlow<Unit>	=
	_navigateToDetail	
21		
22	init {	
23	refreshFromApi()	
24	}	
25		
26	fun refreshFromApi() {	
27	viewModelScope.launch {	
28	try {	
29	repository.refreshFromApi()	
30	} catch (e: Exception) {	
31	}	
32	}	
33	}	
34		
35	fun toggleFavorite(tas: Tas) {	
36	viewModelScope.launch {	
37	repository.toggleFavorite(tas.id,	
	tas.isFavorite)	
38	}	
39	}	
40		
41	fun selectTas(item: Tas?) {	
42	_selectedTas.value = item	
43	if (item != null) {	
44	viewModelScope.launch {	
45	_navigateToDetail.emit(Unit)	
46	}	
47	}	
48	}	
49		
50	fun clearSelectedTas() {	
51	_selectedTas.value = null	
52	}	
53	}	

Tabel 46. Source Code Jawaban Soal 1 Modul 5

17. app/src/main/java/com/example/modul5/viewmodel/TasViewModelFactory.kt

```

1 package com.example.modul5.viewmodel
2
3 import android.content.Context
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6 import com.example.modul5.data.api.ApiClient
7 import com.example.modul5.data.local.TasDatabase
8 import com.example.modul5.data.repository.TasRepository
9
10 class TasViewModelFactory(private val context: Context) :
11     ViewModelProvider.Factory {
12     override fun <T : ViewModel> create(modelClass:
13         Class<T>): T {
14         if
15             (modelClass.isAssignableFrom(TasViewModel::class.java)) {
16             val db = TasDatabase.getDatabase(context)
17             val dao = db.tasDao()
18             val api = ApiClient.retrofit
19             val repository = TasRepository(api, dao)
20             return TasViewModel(repository) as T
21         }
22         throw IllegalArgumentException("Unknown ViewModel
23             class: ${modelClass.name}")
24     }
25 }

```

Tabel 47. Source Code Jawaban Soal 1 Modul 5

18. app/src/main/java/com/example/modul5/MainActivity.kt

```

1 package com.example.modul5
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.modul5.databinding.ActivityMainBinding
6
7 class MainActivity : AppCompatActivity() {
8     private lateinit var binding: ActivityMainBinding
9     override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         binding =
12             ActivityMainBinding.inflate(layoutInflater)
13         setContentView(binding.root)
14     }
15 }

```

Tabel 48. Source Code Jawaban Soal 1 Modul 5

19. app/src/main/res/drawable/ic_love_border.xml

1	<vector
	xmlns:android="http://schemas.android.com/apk/res/android"
2	android:width="24dp" android:height="24dp"
3	android:viewportWidth="24" android:viewportHeight="24">
4	<path
5	android:pathData="M12.1,8.64L12,8.77l-0.1,-
	0.13C10.14,6.36 6.6,6.49 4.6,8.5c-1.96,1.96 -1.96,5.14
	0,7.1L12,23l7.4,-7.4c1.96,-1.96 1.96,-5.14 0,-7.1c-2,-2.01
	-5.54,-2.14 -7.4,0.14z"
6	android:strokeColor="#000000"
7	android:strokeWidth="1.5"
8	android:fillColor="@android:color/transparent" />
9	</vector>

Tabel 49. Source Code Jawaban Soal 1 Modul 5

20. app/src/main/res/drawable/ic_love_filled.xml

1	<vector
	xmlns:android="http://schemas.android.com/apk/res/android"
2	android:width="24dp" android:height="24dp"
	android:viewportWidth="24"
3	android:viewportHeight="24">
4	<path
5	android:fillColor="#FF0000"
6	android:pathData="M12 21.35l-1.45-1.32C5.4 15.36 2
	12.28 2 8.5 2 5.42 4.42 3
7	7.5 3c1.74 0 3.41 0.81 4.5 2.09C13.09 3.81
8	14.76 3
9	16.5 3 19.58 3 22 5.42 22 8.5c0 3.78-3.4 6.86-
10	8.55
	11.54L12 21.35z"/>
	</vector>

Tabel 50. Source Code Jawaban Soal 1 Modul 5

21. app/src/main/res/layout/fragment_home.xml

```
1 <LinearLayout
2   xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:orientation="vertical">
6
7     <TextView
8         android:layout_width="match_parent"
9         android:layout_height="wrap_content"
10        android:background="#D1B3FF"
11        android:padding="15dp"
12        android:text="ShopBag - Your Bag Your Style"
13        android:textStyle="bold"
14        android:textColor="@color/black"
15        android:textSize="25sp" />
16
17     <com.google.android.material.tabs.TabLayout
18         android:id="@+id/tabLayout"
19         android:layout_width="match_parent"
20         android:layout_height="wrap_content" />
21
22     <androidx.viewpager2.widget.ViewPager2
23         android:id="@+id/viewPager"
24         android:layout_width="match_parent"
25         android:layout_height="0dp"
26         android:layout_weight="1" />
27 </LinearLayout>
```

Tabel 51. Source Code Jawaban Soal 1 Modul 5

22. app/src/main/res/layout/fragment_all_product.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3   xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical"
7     android:background="#F6F6F6">
8
9     <androidx.recyclerview.widget.RecyclerView
10        android:id="@+id/recyclerViewAll"
11        android:layout_width="match_parent"
12        android:layout_height="match_parent"
13        android:padding="4dp"
14        android:clipToPadding="false" />
15 </LinearLayout>
```

Tabel 52. Source Code Jawaban Soal 1 Modul 5

23. app/src/main/res/layout/fragment_favorite.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:background="#F6F6F6">
8      <androidx.recyclerview.widget.RecyclerView
9          android:id="@+id/recyclerViewFavorite"
10         android:layout_width="match_parent"
11         android:layout_height="match_parent"
12         android:padding="4dp"
13         android:clipToPadding="false" />
14
15 </LinearLayout>
```

Tabel 53. Source Code Jawaban Soal 1 Modul 5

24. app/src/main/res/layout/item_tas.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.cardview.widget.CardView
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:id="@+id/card_view"
7      android:layout_width="match_parent"
8      android:layout_height="wrap_content"
9      android:layout_gravity="center"
10     android:layout_marginStart="8dp"
11     android:layout_marginTop="4dp"
12     android:layout_marginEnd="8dp"
13     android:layout_marginBottom="4dp"
14     app:cardCornerRadius="12dp"
15     app:cardElevation="10dp">
16     <androidx.constraintlayout.widget.ConstraintLayout
17         android:layout_width="match_parent"
18         android:layout_height="wrap_content"
19         android:background="#F6F6F6"
20         android:padding="20dp">
21
22         <ImageView
23             android:id="@+id/img_item_photo"
```

```

24         android:layout_width="135dp"
25         android:layout_height="140dp"
26         android:scaleType="centerCrop"
27         app:layout_constraintStart_toStartOf="parent"
28         app:layout_constraintTop_toTopOf="parent"
29         app:layout_constraintBottom_toBottomOf="parent"
30         android:contentDescription="@string/app_name" />
31
32     <ImageButton
33         android:id="@+id/btn_favorite"
34         android:layout_width="24dp"
35         android:layout_height="24dp"
36         android:layout_marginStart="4dp"
37         android:layout_marginTop="4dp"
38         android:background="@android:color/transparent"
39         android:src="@drawable/ic_love_border"
40         android:contentDescription="Favorite"
41         app:layout_constraintStart_toStartOf="@id/img_item_photo"
42         app:layout_constraintTop_toTopOf="@id/img_item_photo" />
43
44     <TextView
45         android:id="@+id/tv_item_name"
46         android:layout_width="0dp"
47         android:layout_height="wrap_content"
48         android:text="Tas Kulit Elegan"
49         android:textColor="@color/black"
50         android:textStyle="bold"
51         android:textSize="20sp"
52         app:layout_constraintTop_toTopOf="parent"
53         app:layout_constraintStart_toEndOf="@id/img_item_photo"
54         app:layout_constraintEnd_toEndOf="parent"
55         app:layout_constraintHorizontal_bias="0.0"
56         android:layout_marginStart="8dp"
57         tools:ignore="MissingConstraints" />
58
59     <TextView
60         android:id="@+id/tv_item_price"
61         android:layout_width="0dp"
62         android:layout_height="wrap_content"
63         android:text="Rp 350.000"
64         android:textSize="16sp"
65         android:textColor="@color/black"
66         app:layout_constraintTop_toBottomOf="@id/tv_item_name"
67         app:layout_constraintStart_toStartOf="@id/tv_item_name"
68         app:layout_constraintEnd_toEndOf="parent"
69         android:layout_marginTop="4dp" />
70
71     <Button
72         android:id="@+id/button_shopee"

```

73	android:layout_width="wrap_content"
74	android:layout_height="wrap_content"
75	android:layout_marginStart="8dp"
76	android:layout_marginTop="40dp"
77	android:text="@string/btn_shopee"
78	app:layout_constraintStart_toStartOf="@id/tv_item_name"
79	app:layout_constraintTop_toBottomOf="@id/tv_item_price" />
80	
81	<Button
82	android:id="@+id/button_detail"
83	android:layout_width="wrap_content"
84	android:layout_height="wrap_content"
85	android:layout_marginStart="20dp"
86	android:text="@string/btn_detail"
87	app:layout_constraintStart_toEndOf="@id/button_shopee"
88	app:layout_constraintTop_toTopOf="@id/button_shopee" />
89	</androidx.constraintlayout.widget.ConstraintLayout>
90	</androidx.cardview.widget.CardView>

Tabel 54. Source Code Jawaban Soal 1 Modul 5

25. app/src/main/res/layout/fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:background="#F6F6F6">
8	
9	<androidx.appcompat.widget.Toolbar
10	android:id="@+id/toolbar"
11	android:layout_width="match_parent"
12	android:layout_height="?attr/actionBarSize"
13	android:background="#D1B3FF"
14	android:text="ShopBag - Your Bag Your Style"
15	android:textColor="@color/black"
16	android:theme="@style/ToolbarStyle"/>
17	
18	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
19	xmlns:app="http://schemas.android.com/apk/res-auto"
20	android:layout_width="match_parent"
21	android:layout_height="match_parent"
22	android:padding="16dp"
23	android:background="#F6F6F6">
24	
25	<androidx.constraintlayout.widget.ConstraintLayout

```

26         android:layout_width="match_parent"
27         android:layout_height="wrap_content"
28         android:background="#F6F6F6">
29
30         <ImageView
31             android:id="@+id/img_item_photo"
32             android:layout_width="350dp"
33             android:layout_height="400dp"
34             android:scaleType="centerCrop"
35             app:layout_constraintTop_toTopOf="parent"
36             app:layout_constraintStart_toStartOf="parent"
37             app:layout_constraintEnd_toEndOf="parent"
38             android:contentDescription="@string/app_name" />
39
40         <TextView
41             android:id="@+id/tv_name"
42             android:layout_width="wrap_content"
43             android:layout_height="wrap_content"
44             android:textStyle="bold"
45             android:textColor="@color/black"
46             android:textSize="25sp"
47             android:layout_marginTop="16dp"
48             app:layout_constraintTop_toBottomOf="@id/img_item_photo"
49             app:layout_constraintStart_toStartOf="parent"
50             app:layout_constraintEnd_toEndOf="parent"/>
51
52         <TextView
53             android:id="@+id/tv_harga"
54             android:layout_width="wrap_content"
55             android:layout_height="wrap_content"
56             android:textColor="@android:color/holo_red_dark"
57             android:textStyle="bold"
58             android:textSize="18sp"
59             android:layout_marginTop="8dp"
60             app:layout_constraintTop_toBottomOf="@id/tv_name"
61             app:layout_constraintStart_toStartOf="parent"
62             app:layout_constraintEnd_toEndOf="parent" />
63
64         <TextView
65             android:id="@+id/tv_deskripsi"
66             android:layout_width="match_parent"
67             android:layout_height="wrap_content"
68             android:layout_marginTop="30dp"
69             android:textSize="16sp"
70             android:textColor="@color/black"
71             android:background="#F6F6F6"
72             android:textAlignment="center"
73             android:lineSpacingExtra="5dp"
74             android:padding="8dp"

```


75	app:layout_constraintTop_toBottomOf="@id/tv_harga"
76	app:layout_constraintStart_toStartOf="parent"
77	app:layout_constraintEnd_toEndOf="parent" />
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</ScrollView>
80	</LinearLayout>

Tabel 55. Source Code Jawaban Soal 1 Modul 5

26. app/src/main/res/layout/activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:tools="http://schemas.android.com/tools"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity"
9	tools:ignore="ExtraText">
10	
11	<androidx.fragment.app.FragmentContainerView
12	android:id="@+id/nav_host_fragme
13	
14	android:name="androidx.navigation.fragment.NavHostFragment"
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	app:navGraph="@navigation/nav_graph"
18	app:defaultNavHost="true" />
	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 56. Source Code Jawaban Soal 1 Modul 5

27. app/src/main/res/navigation/nav_graph.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	app:startDestination="@id/homeFragment">
6	
7	<fragment
8	android:id="@+id/homeFragment"
9	android:name="com.example.modul5.ui.home.HomeFragment"
10	android:label="fragment_home"
11	tools:layout="@layout/fragment_home">
12	<action
13	android:id="@+id/action_homeFragment_to_detailFragment"

14	app:destination="@id/detailFragment" />
15	</fragment>
16	
17	<fragment
18	android:id="@+id/detailFragment"
19	android:name="com.example.modul5.ui.detail.DetailFragment"
20	android:label="fragment_detail"
21	tools:layout="@layout/fragment_detail" />
22	</navigation>

Tabel 57. Source Code Jawaban Soal 1 Modul 5

28. app/src/main/res/values/strings.xml

1	<resources>
2	<string name="app_name">MODUL5</string>
3	<string name="label_harga">Harga</string>
4	<string name="label_deskripsi">Deskripsi</string>
5	<string name="btn_detail">Detail</string>
6	<string name="btn_shopee">Shopee</string>
7	</resources>

Tabel 58. Source Code Jawaban Soal 1 Modul 5

29. app/src/main/res/values/styles.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<style name="ToolbarStyle"
	parent="ThemeOverlay.AppCompat.Light">
4	<item
	name="android:textColorPrimary">@color/black</item>
5	<item name="android:textStyle">bold</item>
6	<item
	name="titleTextAppearance">@style/ToolbarTitleText</item>
7	</style>
8	
9	<style name="ToolbarTitleText"
	parent="TextAppearance.Widget.AppCompat.Toolbar.Title">
10	<item name="android:textColor">@color/black</item>
11	<item name="android:textStyle">bold</item>
12	<item name="android:textSize">25sp</item>
13	</style>
14	</resources>

Tabel 59. Source Code Jawaban Soal 1 Modul 5

30. Data JSON dari Beeceptor

```
[
  {
    "id": 1,
    "nama": "BOSTANTEN [Amora Bag]",
    "harga": "Rp159.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbkb-m9jogtk5msuqdb.webp",
    "shopeeLink": "https://id.shp.ee/vKZm8c9",
    "deskripsi": "(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam pilihan warna Off Putih, Hitam, Coklat, dan Aprikot. Dengan dimensi 22.5 cm (Lebar) × 26.5 cm (Tinggi) × 3.5 cm (Tebal) dan berat 0.28 kg, tas ini dilengkapi dengan ruang penyimpanan besar, satu kantong, dan satu kantong resleting. Tali belakangnya memiliki panjang antara 103-124 cm. Tas ini cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, serta acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantoran, pelajar, dan remaja."
  },
  {
    "id": 2,
    "nama": "BOSTANTEN [LYN Bag]",
    "harga": "Rp159.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk7-m9jl9r3bsgl780.webp",
    "shopeeLink": "https://id.shp.ee/56fWCro",
    "deskripsi": "(Bostanten) 100% Asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dengan pilihan warna Coklat, Off Putih, Hitam, dan Aprikot Coklat. Memiliki dimensi 29 cm (Lebar) × 14 cm (Tinggi) × 9 cm (Tebal), dan berat sekitar 0.34 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas masukan dalam, dan satu tas zipper internal. Pegangan tas memiliki jarak sedang yaitu 31 cm. Sangat cocok digunakan untuk sekolah, kantor, pesta, dan perjalanan."
  },
  {
    "id": 3,
    "nama": "BOSTANTEN [Madeline Bag]",
```

```

    "harga": "Rp179.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk0-m9jkl0819vql50.webp",
    "shopeeLink": "https://id.shp.ee/7ARV68N",
    "deskripsi": "<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, Hitam, Abu-abu, dan Coklat. Berukuran 28.5 cm (Lebar) × 21.5 cm (Tinggi) × 7 cm (Tebal), dengan berat 0.29 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas zipper belakang, dan satu saku masukan depan. Sabuk bahunya memiliki tinggi tengah sekitar 0-111 cm dan tidak dapat disesuaikan maupun dilepas. Tas ini cocok digunakan untuk sekolah, kantor, pesta, maupun perjalanan."
  },
  {
    "id": 4,
    "nama": "BOSTANTEN [Reigny Bag]",
    "harga": "Rp189.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk0-m9nvrvmvwt736@resize_w900_nl.webp",
    "shopeeLink": "https://id.shp.ee/957MHuZ",
    "deskripsi": "(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini menggunakan bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, dan Hitam. Ukurannya adalah 20 cm (Lebar) × 16 cm (Tinggi) × 9 cm (Tebal), dengan berat 0.27 kg. Dilengkapi dengan ruang penyimpanan besar, tas masukan, dan sabuk bahu dengan tinggi tengah sekitar 92-112 cm yang dapat disesuaikan dan dilepas pasang. Pegangan tas berukuran sedang, yaitu 9 cm. Cocok untuk sekolah, kantor, pesta, maupun perjalanan."
  },
  {
    "id": 5,
    "nama": "MOSSDOOM [Delphine Bag]",
    "harga": "Rp153.900",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rasl-m63hsk5ln0ao17@resize_w900_nl.webp",
    "shopeeLink": "https://id.shp.ee/GaVtJQ1",
    "deskripsi": "Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Dengan ukuran 17 cm (Panjang) x 9 cm (Lebar) x 10 cm (Tinggi) dan berat 0.29 kg. Tersedia dengan beberapa

```

pilhan warna Pink, Blue, Purple, Apricot, Beige White, Black. Tas ini memiliki struktur internal dengan saku penyimpanan besar, tinggi tali bahunya sekitar 110-121 cm (yang dapat disesuaikan dan bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya."

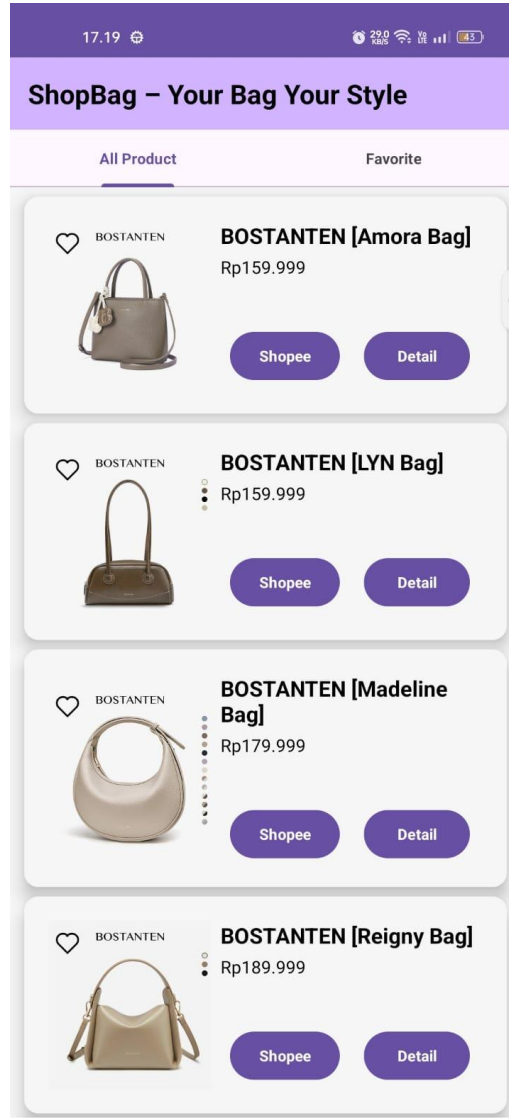
```

    },
    {
      "id": 6,
      "nama": "MOSSDOOM [Gradien Hand Bag]",
      "harga": "Rp249.000",
      "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk3-m8a5q159vrpp01.webp",
      "shopeeLink": "https://id.shp.ee/6tzCzpz",
      "deskripsi": "Selamat datang di produk MOSSDOOM, dijamin keasliannya 100%! Tas ini terbuat dari bahan kulit PU. Dengan ukuran panjang 19.5 cm, lebar 8 cm, tinggi 16.5 cm, dan dengan berat 0.36 kg. Tersedia hanya warna Pink. Tas ini dilengkapi dengan satu saku penyimpanan besar, satu saku sisipan, satu saku beritsleting, dan tinggi pegangan tas tangan sebesar 7cm. Untuk tali bahu memiliki panjang antara 47.5-59cm (dapat disesuaikan). Semua gambar produk ini diambil dalam kehidupan nyata, jadi dijamin keaslinya."
    },
    {
      "id": 7,
      "nama": "MOSSDOOM [Lysmare Bag]",
      "harga": "Rp189.000",
      "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rase-m63hsk5lptfkee.webp",
      "shopeeLink": "https://id.shp.ee/t3osSMq",
      "deskripsi": "Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Berukuran 20 cm (Panjang) x 8 cm (Lebar) x 14 cm (Tinggi) dan berat 0.36 kg. Tas ini tersedia dalam pilihan warna Blue, Light Pink, Pink, Brown, Beige White, Black. Tas ini dilengkapi dengan ruang penyimpanan besar, satu saku sisipan, satu saku beresleting, dan dengan tali bahu setinggi kira-kira 106-130cm (yang dapat disesuaikan, bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya."
    }
  ]

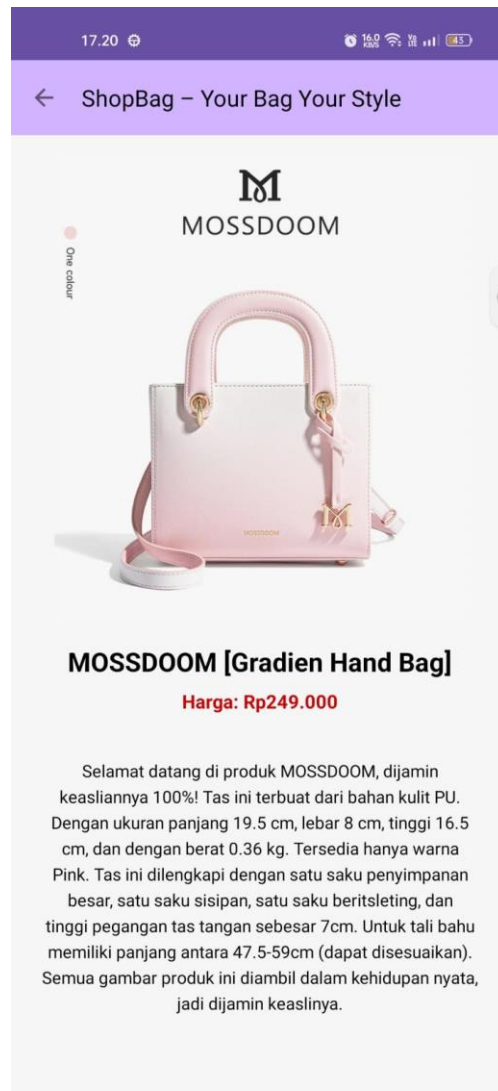
```

Tabel 60. Source Code Jawaban Soal 1 Modul 5

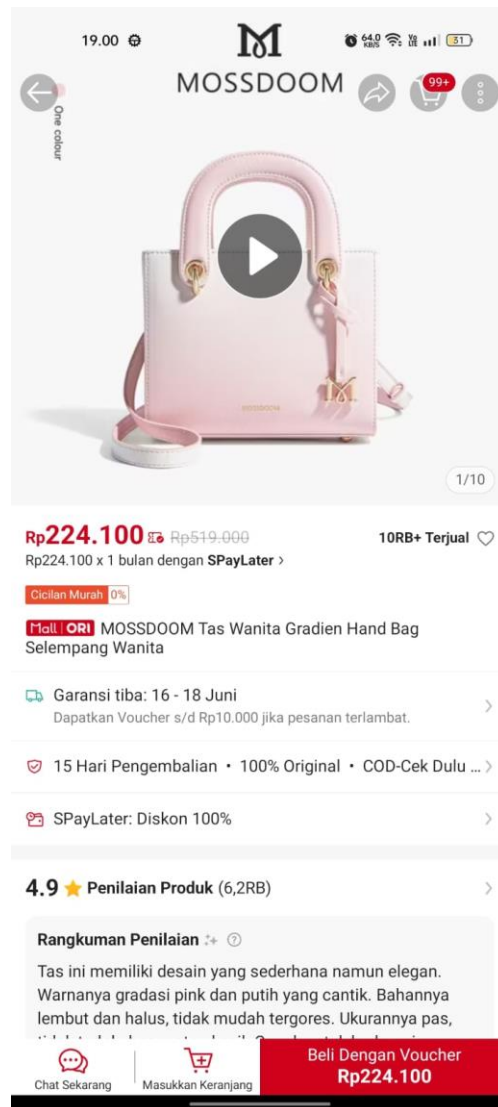
B. Output Program



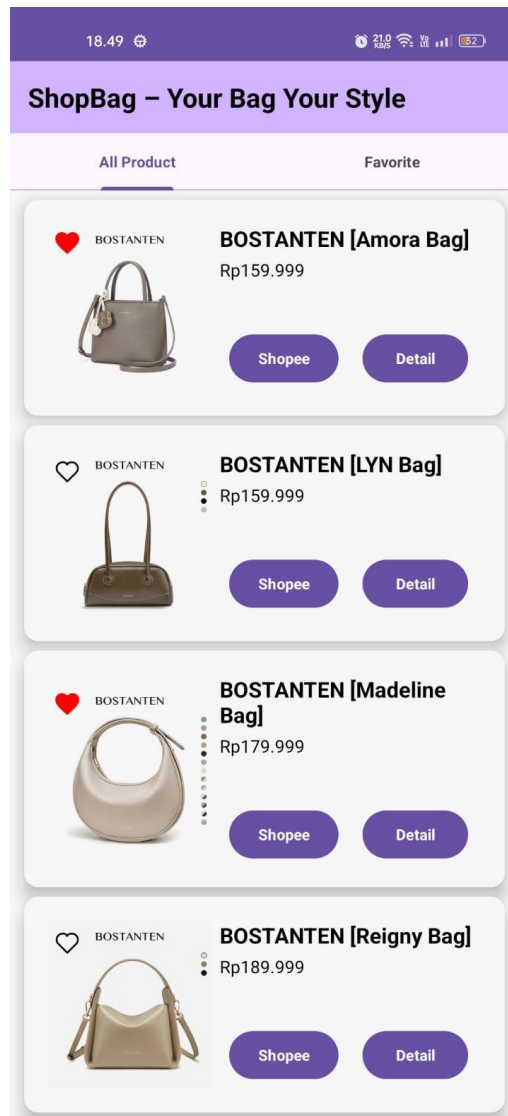
Gambar 33. Screenshot Hasil UI List Soal 1 Modul 5



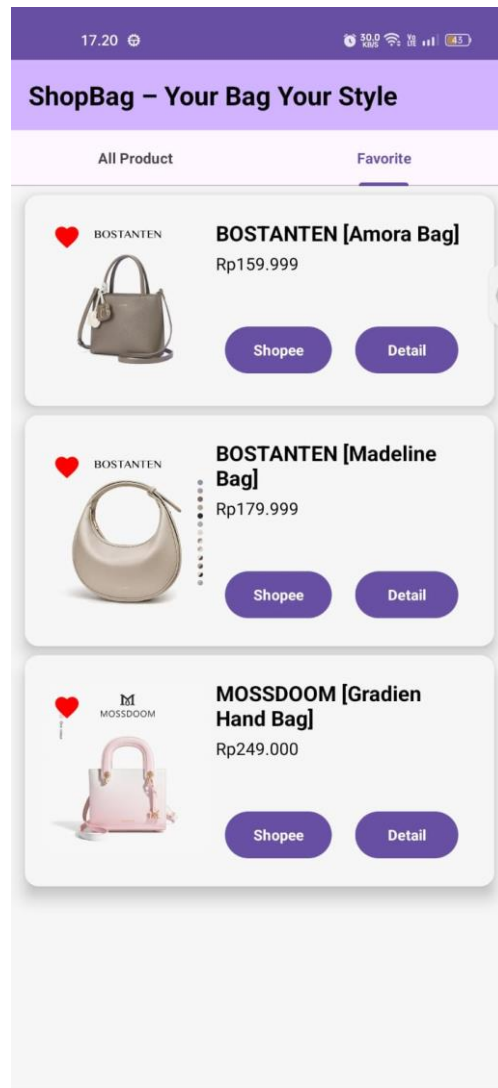
Gambar 34. Screenshot Hasil UI Detail Saat Menekan Tombol Detail Soal 1 Modul 5



Gambar 35. Screenshot Hasil Berpindah ke Aplikasi Shopee Saat Menekan Tombol Shopee Soal 1 Modul 5



Gambar 36. Screenshot Hasil UI List Tambah Favorit Soal 1



Gambar 37. Screenshot Hasil UI Halaman Favorit Soal 1 Modul 5

C. Pembahasan

1. ApiClient.kt

Pada line 1, package `com.example.modul5.data.api` digunakan untuk menandakan file ini berada di dalam package `data.api` dalam modul `modul5`. Kemudian, pada line 3 sampai 6, mengimpor konverter untuk `kotlinx.serialization`, konfigurasi format JSON, tipe `MediaType` dari `OkHttp` (digunakan saat menentukan `Content-Type`), dan retrofit builder class `Lalu`, line 8 membuat singleton object `ApiClient` agar hanya ada satu instance `Retrofit` yang digunakan aplikasi. Line 9 terdapat URL dasar API yang akan digunakan `Retrofit`, disini saya menggunakan `Beeceptor` (endpoint <https://modul5.free.beeceptor.com/data>). Line 11 mengkonfigurasi parser JSON dari `kotlinx.serialization`. Opsi `ignoreUnknownKeys = true` berarti kalau ada field tambahan di JSON yang tidak dikenali, diabaikan saja (tidak menyebabkan error). Line 13 sampai 18 membuat instance `Retrofit` dengan menentukan `baseUrl` dari API, menentukan konverter JSON yang akan digunakan oleh `Retrofit` yaitu `kotlinx.serialization`, membangun instance `Retrofit`, dan menghubungkan `Retrofit` dengan interface `TasApiService`, hasil akhirnya adalah objek siap pakai untuk memanggil API `getTas()`.

2. TasApiService.kt

Pada line 1, package `com.example.modul5.data.api` digunakan untuk menandakan bahwa file ini berada di package `data.api`. Kemudian, pada line 3 dan 4, mengimpor anotasi `Retrofit @GET` dan model `Tas`. Lalu, line 6 mendefinisikan interface API. Ini akan digunakan oleh `Retrofit` untuk memanggil endpoint. Line 7 sampai 9 terdapat anotasi `Retrofit` yang artinya saat fungsi `getTas()` dipanggil, akan melakukan GET ke `BASE_URL + "data"` (<https://modul5.free.beeceptor.com/data>). Juga terdapat fungsi bersifat `suspend` (dipanggil dari `coroutine`). Ini akan

memanggil endpoint API dan mengembalikan list of tas dalam bentuk TasDto.

3. TasDto.kt

Pada line 1, package `com.example.modul5.data.api` digunakan untuk menandakan file ini berada di package yang sama dengan service. Line 3 mengimpor anotasi `@Serializable` agar Kotlin bisa melakukan parsing JSON menggunakan `kotlinx.serialization`. Line 5, `@Serializable` menandai bahwa kelas `TasDto` bisa dikonversi dari dan ke JSON. Line 6 sampai 13 adalah struktur data tas dari API. Properti ini sesuai dengan isi dari JSON response yang disimpan di Beeceptor.

4. TasDao.kt

DAO (*Data Access Object*), yaitu interface untuk akses database. Line 10 dan 11 `@Dao interface TasDao {` berguna untuk memberi tahu Room bahwa ini adalah interface DAO yang akan digunakan untuk akses data Tas. Line 12 dan 13 mengambil semua data dari tabel tas sebagai Flow, artinya data bisa terus mengalir (reactive) dan ini cocok dipakai di `ViewModel`. Line 15 dan 16, memasukkan banyak data ke tabel tas. Jika id bentrok, akan menggantikan data lama (REPLACE). Line 18 sampai 20, memperbarui status `isFavorite` berdasarkan id tas tertentu.

5. TasDatabase.kt

Kelas `TasDatabase` ini ialah kelas abstrak yang menjadi basis Room database. Line 8 dan 9 menandakan ini adalah Room Database. `Entities` ialah daftar tabel yang digunakan (`TasEntity`), `version = 1` itu versi database, dan `exportSchema = false` artinya tidak mengeksport struktur database ke file `.json`. Line 10 terdapat fungsi abstrak untuk mendapatkan DAO

(TasDao), inilah yang digunakan di Repository nantinya. Line 12 sampai 14, INSTANCE dibuat @Volatile supaya perubahan langsung terbaca oleh thread lain. Line 16 sampai 28, memastikan hanya satu instance database yang dibuat, sehingga aman dari multi-threading. Juga membuat instance Room database bernama "tas_db". Lalu, Simpan instance dan kembalikan ke pemanggil.

6. TasEntity.kt

Kelas TasEntity ini ialah data model yang disimpan di database. Line 6 dan 7 menandakan ini adalah entitas/tabel bernama tas dalam database Room. Line 8, kolom id adalah primary key. Line 9 sampai 15 terdapat field untuk menyimpan informasi tas dan juga menyimpan status favorite untuk masing-masing tas.

7. TasMapper.kt

File TasMapper berisi tiga fungsi ekstensi penting, yaitu TasDto.toEntity() digunakan untuk mengubah objek dari API (TasDto) ke entitas database (TasEntity) dengan default isFavorite = false. TasEntity.toDomain() digunakan untuk mengubah data dari Room menjadi objek domain Tas, yang bisa digunakan oleh ViewModel dan ditampilkan di UI. Fungsi opsional (yang tidak ditulis di sini tapi bisa ditambahkan) seperti Tas.toEntity() kadang digunakan jika aplikasi ingin menyimpan perubahan dari UI ke database.

8. TasRepository.kt

Kelas ini berfungsi sebagai sumber data tunggal untuk ViewModel. Dengan menggabungkan data dari remote (API) dan local (Room). Fungsi refreshFromApi() hanya mengambil data dari server jika Room masih kosong, untuk menghindari hilangnya data lokal seperti favorit dan

`toggleFavorite()` digunakan untuk mengubah status favorit suatu tas di database.

9. Tas.kt

Kelas ini adalah model utama yang digunakan di seluruh aplikasi, termasuk UI. Ditandai dengan `@Parcelize` agar bisa dikirim antar-fragment atau Activity. Penggunaan `@Transient` ialah untuk mencegah `isFavorite` ikut diserialisasi saat parsing JSON dari API karena nilai ini hanya disimpan lokal (Room). Ini juga berfungsi sebagai bentuk final data sebelum ditampilkan ke pengguna.

10. AdapterTas.kt

AdapterTas adalah kelas `RecyclerView.Adapter` yang digunakan untuk menampilkan daftar produk tas di aplikasi. Adapter ini menerima tiga parameter berupa lambda function, masing-masing untuk menangani klik pada tombol detail (`onDetailClick`), tombol Shopee (`onShopeeClick`), dan tombol favorit (`onFavoriteClick`). Adapter menyimpan daftar data tas dalam properti `tasList`, yang awalnya kosong dan dapat diperbarui melalui fungsi `updateList`, lalu memanggil `notifyDataSetChanged()` agar daftar diperbarui di tampilan.

Di dalamnya terdapat `TasViewHolder`, inner class yang menyimpan binding terhadap layout `item_tas.xml` melalui `ViewBinding` (`ItemTasBinding`). Fungsi `onCreateViewHolder` bertugas meng-inflate layout item dan mengembalikan instance `TasViewHolder`. Saat setiap item di-bind ke tampilan melalui `onBindViewHolder`, data dari objek Tas ditampilkan ke `TextView` dan `ImageView` yang sesuai, termasuk memuat gambar dari URL menggunakan library Coil. Tombol love (favorit) akan menampilkan ikon berbeda tergantung status `isFavorite`, dan tiap tombol disambungkan ke callback fungsinya masing-masing.

Terakhir, `getItemCount` mengembalikan jumlah item yang akan ditampilkan oleh `RecyclerView`, yaitu panjang dari `tasList`. Keseluruhan kelas ini berfungsi sebagai penghubung antara data model tas dan tampilan daftar produk, serta menyediakan interaksi pengguna secara fleksibel dengan memanfaatkan callback dari luar..

11. HomePagerAdapter.kt

Kelas ini merupakan implementasi dari sebuah adapter bernama `HomePagerAdapter` yang digunakan untuk mengelola tampilan tab atau halaman pada aplikasi Android menggunakan `ViewPager2`. Adapter ini mewarisi kelas `FragmentStateAdapter`, yang memungkinkan fragment ditampilkan dan dikelola secara efisien dalam `ViewPager2`. Konstruktor dari adapter ini menerima sebuah objek `Fragment` sebagai parameter konteks utama, biasanya fragment induk tempat `ViewPager2` berada. Metode `getItemCount()` mengembalikan jumlah total halaman, yaitu 2, yang berarti terdapat dua tab yang akan ditampilkan. Metode `createFragment(position: Int)` bertanggung jawab untuk mengembalikan fragment yang sesuai berdasarkan posisi halaman. Jika posisi adalah 0, maka akan ditampilkan `AllProductFragment`, dan jika posisi adalah 1, maka akan ditampilkan `FavoriteFragment`. Apabila posisi yang diberikan tidak sesuai dengan rentang yang diharapkan (selain 0 atau 1), maka akan dilemparkan pengecualian `IllegalArgumentException` untuk menandakan bahwa posisi tersebut tidak valid.

12. HomeFragment.kt

Kelas ini adalah implementasi dari sebuah fragment bernama `HomeFragment` dalam aplikasi Android berbasis arsitektur MVVM (Model-View-ViewModel). Fragment ini bertanggung jawab sebagai tampilan utama yang menampilkan dua tab: "All Product" dan "Favorite", dengan

menggunakan ViewPager2 dan TabLayout. Pada bagian line awal, fragment mengambil instance dari TasViewModel menggunakan delegation by activityViewModels yang dibantu oleh TasViewModelFactory untuk menyediakan context. Binding view diatur menggunakan FragmentHomeBinding, yang mempermudah akses ke komponen UI yang didefinisikan dalam layout XML terkait.

Dalam metode onCreateView, layout fragment di-*inflate* dan disimpan ke dalam `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama fragment. Kemudian, di `onViewCreated`, adapter `HomePagerAdapter` diinisialisasi dengan referensi `this` (fragment saat ini), dan dipasangkan ke ViewPager untuk mengatur halaman/tab. `TabLayoutMediator` digunakan untuk menghubungkan TabLayout dengan ViewPager, dan memberi label pada masing-masing tab sesuai dengan posisi “All Product” di posisi 0 dan “Favorite” di posisi 1.

Selanjutnya, terdapat blok `lifecycleScope.launch` yang menjalankan coroutine untuk mengamati `navigateToDetail`, sebuah Flow yang dipancarkan oleh `viewModel`. Ketika `navigateToDetail` memancarkan nilai, fragment akan memeriksa apakah ada tas yang dipilih (`selectedTas`). Jika ada, maka navigasi akan dilakukan ke `DetailFragment` menggunakan `findNavController()`. Jika tidak ada tas yang dipilih, maka akan dicetak log error.

Terakhir, di metode `onDestroyView`, binding di-*null*-kan untuk menghindari memory leak ketika fragment dihancurkan. Jadi, fragment ini menangani tampilan dua kategori produk dan memungkinkan navigasi ke detail produk melalui ViewModel secara reaktif.

13. AllProductFragment.kt

Kelas ini adalah implementasi dari `AllProductFragment`, yaitu salah satu tab pada halaman utama aplikasi yang berfungsi untuk menampilkan

daftar semua produk tas. Fragment ini menggunakan arsitektur MVVM dengan bantuan ViewModel (TasViewModel) dan LiveData berbasis Flow untuk memantau data produk secara reaktif. Binding view dilakukan menggunakan FragmentAllProductBinding, yang secara otomatis menghubungkan komponen UI dari layout XML ke kode Kotlin. ViewModel diperoleh menggunakan delegation by `activityViewModels` agar dapat berbagi data dengan fragment lainnya dalam satu aktivitas.

Pada `onCreateView`, layout di-inflate dan disimpan dalam `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama fragment. Di `onViewCreated`, adapter `AdapterTas` diinisialisasi dengan tiga parameter fungsi lambda, diantaranya yaitu `onDetailClick` digunakan untuk memanggil `selectTas(it)` dari `viewModel` ketika pengguna memilih produk untuk melihat detailnya, `onShopeeClick` digunakan untuk membuka link Shopee menggunakan `Intent.ACTION_VIEW` ke browser atau aplikasi Shopee, berdasarkan `shopeeLink` dari produk, dan `onFavoriteClick` untuk memanggil `toggleFavorite(it)` untuk menandai atau menghapus tanda favorit pada produk tersebut. `RecyclerView` diatur menggunakan `LinearLayoutManager` agar menampilkan daftar secara vertikal, dan adapter dipasangkan ke `recyclerViewAll`.

Terakhir, di dalam `lifecycleScope.launch`, fragment mengamati `tasList`, yaitu aliran data dari ViewModel. Ketika data produk berubah, adapter akan diperbarui dengan `adapter.updateList(list)` untuk menampilkan daftar terbaru di layar. Saat fragment dihancurkan, binding di-set ke `null` di `onDestroyView` untuk mencegah memory leak. Secara keseluruhan, fragment ini menangani logika tampilan dan interaksi untuk semua produk, termasuk menampilkan daftar, membuka tautan eksternal, dan mengelola status favorit.

14. FavoriteFragment.kt

Kelas ini adalah implementasi dari `FavoriteFragment`, yaitu salah satu tab dalam tampilan utama aplikasi yang secara khusus menampilkan daftar produk tas yang telah ditandai sebagai favorit oleh pengguna. Fragment ini juga mengikuti arsitektur MVVM dan memanfaatkan `TasViewModel` sebagai sumber data yang dibagikan antar fragment. Fragment ini menggunakan view binding (`FragmentFavoriteBinding`) untuk mengakses elemen UI dari layout XML-nya. `ViewModel` diperoleh melalui delegation by `activityViewModels` dengan `TasViewModelFactory` sebagai penyedia context, sehingga `FavoriteFragment` berbagi `ViewModel` yang sama dengan fragment lainnya seperti `AllProductFragment`.

Di dalam metode `onCreateView`, layout di-inflate dan root view dikembalikan. Kemudian, pada `onViewCreated`, adapter `AdapterTas` diinisialisasi dengan tiga aksi, diantaranya yaitu `onDetailClick` digunakan saat item produk diklik, fragment akan memanggil `viewModel.selectTas(it)` untuk memilih produk tersebut agar nanti bisa diproses navigasi ke halaman detail, `onShopeeClick` digunakan ketika pengguna menekan tombol beli, aplikasi membuka tautan Shopee dari produk tersebut menggunakan `Intent.ACTION_VIEW`, dan `onFavoriteClick` untuk menandai atau membatalkan status favorit pada item produk dengan memanggil `viewModel.toggleFavorite(it)`. `RecyclerView` pada layout (`recyclerViewFavorite`) dikonfigurasi dengan `LinearLayoutManager` agar menampilkan daftar produk secara vertikal, dan adapter yang telah dibuat sebelumnya dipasangkan ke dalamnya.

Selanjutnya, fragment menggunakan coroutine di dalam `viewLifecycleOwner.lifecycleScope.launch` untuk mengamati `tasList`, yaitu daftar produk dari `ViewModel`. Namun, berbeda dari `AllProductFragment`, di sini hanya produk yang memiliki atribut `isFavorite = true` yang akan ditampilkan. Proses ini dilakukan dengan menyaring daftar menggunakan `.filter { it.isFavorite }`, lalu

memperbarui adapter dengan `adapter.updateList(favorites)`. Secara keseluruhan, fragment ini menangani logika tampilan untuk daftar produk favorit dan mendukung interaksi seperti membuka tautan Shopee, melihat detail produk, dan mengelola status favorit.

15. DetailFragment.kt

Kelas ini adalah implementasi dari `DetailFragment`, yaitu tampilan detail dalam aplikasi yang menampilkan informasi lengkap mengenai salah satu produk tas yang dipilih oleh pengguna. Fragment ini menggunakan arsitektur MVVM, dengan `TasViewModel` sebagai `ViewModel` yang mengelola data produk yang sedang dipilih (`selectedTas`). Pada awalnya, `ViewModel` diakses melalui `delegation by activityViewModels` menggunakan `TasViewModelFactory` agar `ViewModel` yang digunakan sama dengan fragment lain dalam aktivitas yang sama. Binding ke layout dilakukan menggunakan `FragmentDetailBinding`.

Di dalam metode `onCreateView`, layout di-inflate dan view root dikembalikan untuk ditampilkan. Kemudian, di `onViewCreated`, toolbar diatur sebagai `ActionBar` menggunakan `setSupportActionBar`. Judul `ActionBar` diset menjadi "ShopBag – Your Bag Your Style", dan tombol navigasi balik (back) diaktifkan. Saat tombol tersebut ditekan, pengguna akan dinavigasikan kembali ke tampilan sebelumnya menggunakan `findNavController().navigateUp()`. Bagian inti dari fragment ini adalah coroutine dalam `lifecycleScope.launch` yang mengamati aliran data `selectedTas` dari `ViewModel` menggunakan `collectLatest`. Ketika data tas tersedia (tidak null), maka elemen UI seperti `TextView` dan `ImageView` akan diisi dengan informasi dari objek `tas`, yaitu nama, harga, deskripsi, dan gambar. Untuk memuat gambar, digunakan library `Coil`, yang menyediakan placeholder saat loading dan ikon error jika gambar gagal dimuat.

Jika `selectedTas` ternyata null (misalnya karena belum dipilih atau telah dihapus), pengguna secara otomatis dikembalikan ke halaman sebelumnya dengan `navigateUp()`, sebagai mekanisme perlindungan dari tampilan kosong atau kesalahan akses data. Terakhir, pada `onDestroyView`, method `clearSelectedTas()` dipanggil untuk menghapus data tas yang sedang dipilih dari `ViewModel`. Binding juga di-null-kan untuk menghindari memory leak. Sehingga, `DetailFragment` berfungsi untuk menampilkan detail produk secara dinamis berdasarkan pilihan pengguna, sekaligus menangani navigasi dan pengelolaan memori secara baik.

16. TasViewModel.kt

Kelas ini adalah implementasi dari `TasViewModel`, sebuah kelas yang berperan sebagai jembatan antara data (`TasRepository`) dan UI dalam arsitektur MVVM pada aplikasi Android. `ViewModel` ini digunakan untuk mengelola data tas, menangani aksi pengguna, dan menyimpan status sementara yang dibutuhkan antar fragment.

`val tasList: StateFlow<List<Tas>>` adalah aliran data utama yang berisi daftar produk tas. Nilai ini diperoleh dari `TasRepository`, lalu dikonversi menjadi `StateFlow` menggunakan `.stateIn()` agar data bisa diamati secara berkelanjutan di UI. Fungsi `.onEach` digunakan di sini untuk mencetak jumlah data dari Room setiap kali data diperbarui, sebagai debug log. `_selectedTas` adalah `MutableStateFlow` yang menyimpan satu tas yang sedang dipilih oleh pengguna (misalnya untuk ditampilkan di `DetailFragment`). Properti publik `selectedTas` adalah versi hanya-baca dari nilai ini agar tidak bisa dimodifikasi langsung oleh kelas lain. `_navigateToDetail` adalah `MutableSharedFlow` yang digunakan untuk memicu event navigasi ke halaman detail. `SharedFlow` cocok untuk satu-kali event (seperti navigasi), karena tidak menyimpan nilai terakhir. `navigateToDetail` adalah versi publiknya.

Fungsi-fungsi penting seperti `init { refreshFromApi() }` ialah ketika `ViewModel` dibuat, ia langsung memanggil `refreshFromApi()` untuk mengambil data terbaru dari API dan menyimpannya ke database lokal. Fungsi `refreshFromApi()` ini memanggil repository untuk memuat ulang data dari API dalam coroutine. Jika terjadi error, pengecualian ditangani secara diam-diam. Fungsi `toggleFavorite(tas: Tas)` ini digunakan untuk menandai atau menghapus status favorit suatu produk tas berdasarkan `id` dan `isFavorite`-nya. `selectTas(item: Tas?)` berguna untuk menyimpan tas yang dipilih ke dalam `_selectedTas`, dan jika nilai tidak null, maka memicu navigasi ke halaman detail dengan memancarkan event `navigateToDetail.clearSelectedTas()` berguna untuk menghapus tas yang sedang dipilih dengan menyetel `_selectedTas` ke null. Biasanya dipanggil saat `DetailFragment` dihancurkan.

17. TasViewModelFactory.kt

Kelas ini adalah implementasi dari `TasViewModelFactory`, yaitu custom `ViewModelProvider.Factory` yang bertugas untuk membuat instance `TasViewModel` dengan parameter tambahan yang tidak bisa disediakan oleh konstruktor default, yaitu `Context`. `TasViewModelFactory` menerima `Context` dalam konstruktor. `ViewModelProvider.Factory` adalah antarmuka yang digunakan oleh `ViewModelProvider` untuk membuat `ViewModel` secara manual. Fungsi `create` dipanggil saat framework (seperti `activityViewModels` atau `viewModels`) membutuhkan instance `ViewModel`, lalu memeriksa apakah `modelClass` adalah `TasViewModel`.

Line 12 sampai 18 adanya pembuatan logika `ViewModel`, yang dimana jika ya, maka mendapatkan instance database lokal (`TasDatabase`), lalu mengambil DAO-nya (`tasDao()`), mengakses API Retrofit melalui `ApiClient`, membuat objek `TasRepository` yang menjadi jembatan

antara API dan database, dan mengembalikan instance `TasViewModel` dengan `repository` sebagai parameter. Jika kelas yang diminta bukan `TasViewModel`, maka `Exception` dilempar untuk mencegah pembuatan `ViewModel` yang tidak dikenal.

18. `MainActivity.kt`

`MainActivity` adalah aktivitas utama dalam aplikasi Android yang berfungsi sebagai titik masuk pertama saat aplikasi dijalankan. Kelas ini merupakan turunan dari `AppCompatActivity`, yang menyediakan dukungan kompatibilitas ke belakang untuk fitur-fitur Android modern. Dalam kelas ini digunakan `View Binding` untuk menghubungkan antara layout XML dan kode Kotlin secara lebih aman dan efisien tanpa perlu menggunakan `findViewById`. Properti binding dideklarasikan dengan `lateinit`, yang memungkinkan inisialisasi dilakukan nanti di dalam fungsi `onCreate()`. Di dalam `onCreate()`, layout utama aplikasi (`activity_main.xml`) di-inflate menggunakan `ActivityMainBinding.inflate(layoutInflater)`, kemudian ditampilkan melalui `setContentView(binding.root)`. Sehingga, seluruh komponen UI dalam layout tersebut bisa langsung diakses melalui objek binding, menjadikan kode aman dari `NullPointerException`, dan lebih mudah dibaca. Secara keseluruhan, `MainActivity` bertindak sebagai wadah awal bagi fragment-fragment yang ditampilkan melalui navigasi aplikasi.

19. `ic_love_border.xml`

XML ini ialah sebuah file vektor drawable yang digunakan untuk menggambarkan ikon berbentuk hati (love) dalam format gambar vektor pada Android. Vektor ini memiliki ukuran lebar dan tinggi masing-masing 24dp, serta menggunakan `viewportWidth` dan `viewportHeight` yang juga bernilai 24, yang artinya sistem koordinat internalnya dibuat sebanding dengan

ukuran tampilannya. Di dalam elemen `<path>`, terdapat atribut `android:pathData` yang mendefinisikan bentuk dari ikon hati melalui serangkaian perintah vektor. Atribut `strokeColor="#000000"` untuk memberikan garis tepi berwarna hitam pada ikon, dan `strokeWidth="1.5"` digunakan untuk menetapkan ketebalan garis sebesar 1.5 satuan. Selain itu, `fillColor="@android:color/transparent"` membuat bagian dalam ikon transparan, sehingga hanya garis tepinya saja yang terlihat. Dengan konfigurasi ini, ikon hati akan tampil sebagai outline (garis luar) berbentuk hati tanpa isi warna di bagian dalamnya. Vektor ini digunakan untuk produk yang ingin dijadikan barang favoritnya.

20. ic_love_filled.xml

XML ini ialah sebuah file vector drawable pada Android yang menampilkan ikon hati (love) berwarna merah penuh. Vektor ini memiliki ukuran tampilan `24dp × 24dp` dan sistem koordinat internal (`viewportWidth` dan `viewportHeight`) sebesar 24, yang umum digunakan untuk ikon standar. Di dalamnya terdapat elemen `<path>` yang mendefinisikan bentuk hati melalui atribut `android:pathData`, yaitu serangkaian perintah kurva dan garis untuk menggambar bentuk. Ikon ini memiliki warna isi merah solid (`android:fillColor="#FF0000"`) tanpa garis tepi, sehingga tampil sebagai gambar hati penuh. Ikon seperti ini umumnya digunakan untuk menyimbolkan item favorit atau disukai dalam antarmuka pengguna Android, dan berfungsi baik di berbagai ukuran layar karena menggunakan format vektor yang bersifat skalabel tanpa kehilangan kualitas.

21. fragment_home.xml

XML ini merupakan sebuah layout UI Android yang menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal, artinya

elemen-elemen di dalamnya akan ditampilkan dari atas ke bawah. Layout ini terdiri dari tiga komponen utama. Pertama, terdapat `TextView` yang berfungsi sebagai judul aplikasi dengan teks “ShopBag – Your Bag Your Style”. Komponen ini memiliki lebar penuh (`match_parent`), tinggi menyesuaikan isi (`wrap_content`), latar belakang berwarna ungu muda (`#D1B3FF`), padding sebesar 15dp agar teks tidak mepet ke pinggir, serta teks berwarna hitam, berukuran 25sp, dan bergaya tebal. Kedua, terdapat `TabLayout` dari pustaka Material Design yang akan digunakan untuk menampilkan tab navigasi, seperti “All Product” dan “Favorite”. Komponen ini biasanya digunakan bersama `ViewPager2` untuk membuat antarmuka tab yang interaktif.

Ketiga, ada `ViewPager2` yang merupakan komponen penampil halaman dengan scroll horizontal. `ViewPager` ini memiliki tinggi 0dp dan `layout_weight=1`, yang artinya ia akan mengisi sisa ruang yang tersedia di layar setelah `TextView` dan `TabLayout` ditampilkan. `ViewPager` ini akan menampung dan menampilkan fragment berdasarkan tab yang dipilih di `TabLayout`. Secara keseluruhan, layout ini dirancang untuk menampilkan halaman utama aplikasi dengan tampilan header, tab navigasi, dan konten yang dapat digeser antar tab.

22. `fragment_all_product.xml`

XML ini adalah layout Android yang menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna abu-abu terang (`#F6F6F6`). Di dalamnya hanya terdapat satu komponen utama, yaitu `RecyclerView`, yang diberi ID `recyclerViewAll`. `RecyclerView` ini berfungsi untuk menampilkan daftar item secara scrollable, seperti daftar produk pada toko online. Komponen ini mengisi seluruh lebar dan tinggi layar (`match_parent`), memiliki padding sebesar 4dp di sekelilingnya, dan atribut `clipToPadding="false"` memungkinkan isi

RecyclerView untuk tetap terlihat saat menyentuh batas padding, misalnya untuk efek animasi atau shadow.

23. fragment_favorite.xml

XML ini adalah layout untuk tampilan Android yang menggunakan `LinearLayout` dengan orientasi vertikal dan latar belakang berwarna abu-abu terang (`#F6F6F6`). Di dalamnya terdapat sebuah `RecyclerView` dengan ID `recyclerViewFavorite`, yang digunakan untuk menampilkan daftar item favorit secara scrollable. `RecyclerView` ini mengisi seluruh ruang layar baik secara lebar maupun tinggi (`match_parent`), memiliki padding sebesar 4dp di sekelilingnya untuk memberikan jarak antar item dengan tepi layar, dan `clipToPadding="false"` agar item tetap bisa terlihat sampai ke area padding, misalnya untuk efek visual seperti bayangan atau animasi scroll. Layout ini digunakan sebagai bagian dari tab "Favorite" pada aplikasi yang menampilkan daftar produk favorit pengguna.

24. item_tas.xml

XML ini adalah layout item untuk sebuah daftar produk yang ditampilkan di dalam `RecyclerView`, menggunakan `CardView` sebagai kontainer utamanya. `CardView` memberikan efek bayangan (`elevation`) dan sudut melengkung (`cardCornerRadius`) agar tampilan produk terlihat seperti kartu yang elegan. Di dalam `CardView`, terdapat `ConstraintLayout` sebagai root view untuk mengatur posisi elemen secara fleksibel. Layout ini mencakup elemen-elemen `ImageView` (`img_item_photo`) untuk menampilkan gambar produk dengan ukuran tetap dan posisi di sisi kiri kartu. Gambar dipangkas (`centerCrop`) agar pas di dalam kotak. `ImageButton` (`btn_favorite`) digunakan untuk tombol berbentuk hati di pojok kiri atas gambar, yang berfungsi untuk menandai produk sebagai favorit. `TextView` (`tv_item_name`) untuk menampilkan nama produk di sebelah kanan

gambar, menggunakan teks besar dan tebal. `TextView (tv_item_price)` untuk menampilkan harga produk di bawah nama produk. `Button (button_shopee)` ialah tombol yang mengarahkan pengguna ke tautan Shopee untuk membeli produk. `Button (button_detail)` ialah tombol untuk melihat detail lengkap produk.

25. fragment_detail.xml

XML ini merupakan tampilan antarmuka untuk halaman detail produk dalam aplikasi Android. Layout ini menggunakan struktur `LinearLayout` vertikal sebagai root, dengan latar belakang berwarna abu-abu muda (`#F6F6F6`). Di bagian atas, terdapat `Toolbar` yang berfungsi sebagai action bar dan menampilkan judul aplikasi “ShopBag – Your Bag Your Style” dengan latar ungu muda (`#D1B3FF`). Bagian isi utama dibungkus dalam `ScrollView`, yang memungkinkan konten digulir jika melebihi tinggi layar. Di dalam `ScrollView`, digunakan `ConstraintLayout` untuk mengatur posisi komponen secara presisi. Komponen utama meliputi `ImageView (img_item_photo)` digunakan untuk menampilkan gambar produk dengan ukuran besar (`350x400dp`) dan mode `centerCrop` agar gambar terisi penuh dan proporsional. `TextView (tv_name)` untuk menampilkan nama produk di bawah gambar, dengan huruf tebal dan ukuran besar (`25sp`). `TextView (tv_harga)` untuk menampilkan harga produk dengan warna merah gelap dan ukuran menonjol (`18sp`). `TextView (tv_deskripsi)` digunakan untuk menampilkan deskripsi produk dengan teks rata tengah, spasi baris ekstra, dan padding agar mudah dibaca.

26. nav_graph.xml

Pada line 1 sampai 5, menentukan bahwa fragment pertama kali yang ditampilkan saat app berjalan adalah `HomeFragment`. Line 7 sampai 11, mewakili untuk tampilan utama (daftar tas). Fragment ini berada di package

com.example.modul3. Layout yang digunakan ialah `fragment_home.xml`. Line 12 sampai 15, mendefinisikan rute navigasi dari `HomeFragment` ke `DetailFragment`, hal ini akan dipicu saat user klik tombol “Detail” pada item tas. Line 17 sampai 22 adalah fragment untuk menampilkan detail dari item tas yang dipilih, dengan menggunakan layout `fragment_detail.xml`.

27. strings.xml

File XML tersebut adalah berkas `strings.xml` yang berfungsi untuk menyimpan resource string dalam aplikasi Android. File ini memungkinkan pengelolaan teks secara terpusat dan memudahkan penerjemahan (lokalisasi) ke berbagai bahasa. `<string name="app_name">MODUL5</string>` ialah endefinisikan nama aplikasi yang akan ditampilkan di launcher dan toolbar. `<string name="label_harga">Harga</string>` adalah label teks untuk menampilkan harga produk. `<string name="label_deskripsi">Deskripsi</string>` adalah label teks untuk bagian deskripsi produk. `<string name="btn_detail">Detail</string>` ialah teks pada tombol yang mengarahkan ke halaman detail produk. `<string name="btn_shopee">Shopee</string>` ialah teks pada tombol yang mengarahkan ke halaman Shopee.

28. styles.xml

File XML tersebut adalah file `styles.xml`, yang digunakan untuk mendefinisikan gaya (style) kustom dalam aplikasi Android. Style ini mempermudah penerapan tampilan yang konsisten di berbagai komponen UI. `<style name="ToolbarStyle" parent="ThemeOverlay.AppCompat.Light">` Style ini digunakan untuk mendesain tampilan Toolbar. Style ini mewarisi dari tema bawaan

AppCompatActivity.Light, yang merupakan gaya terang standar dari Android. `<item name="android:textColorPrimary">@color/black</item>` digunakan untuk mengatur warna utama teks dalam toolbar menjadi hitam. `<item name="android:textStyle">bold</item>` untuk menjadikan teks dalam toolbar tampil tebal (bold). `<item name="titleTextAppearance">@style/ToolbarTitleText</item>` ialah menentukan bahwa gaya teks judul toolbar akan menggunakan style ToolbarTitleText. `<style name="ToolbarTitleText" Style ini digunakan secara spesifik untuk tampilan teks judul toolbar. <item name="android:textColor">@color/black</item>` untuk warna teks judul diset ke hitam. `<item name="android:textStyle">bold</item>` untuk teks judul ditampilkan dalam gaya tebal. `<item name="android:textSize">25sp</item>` ialah ukuran teks judul dengan ditetapkan sebesar 25sp (scale-independent pixels), memastikan keterbacaan pada berbagai ukuran layar.

29. activity_main.xml

Pada line 1 sampai 9, ConstraintLayout adalah layout utama yang digunakan di sini untuk mengatur posisi elemen UI di dalamnya. ConstraintLayout memungkinkan penataan elemen UI dengan fleksibilitas tinggi melalui constraint (penghubung). `xmlns:android` adalah deklarasi namespace untuk atribut Android standar. `xmlns:tools` digunakan oleh Android Studio untuk memberikan data atau tips visual saat pengembangan tanpa mempengaruhi aplikasi yang sebenarnya. `tools:context=".MainActivity"` memberikan konteks di mana layout ini digunakan (dalam hal ini, MainActivity). `tools:ignore="ExtraText"` memberikan petunjuk kepada Android

Studio untuk mengabaikan peringatan tertentu yang tidak penting (misalnya teks ekstra).

Kemudian, line 11 sampai 18 terdapat `FragmentContainerView`, yang di mana komponen ini digunakan untuk menampilkan dan mengelola fragment di dalam activity. `FragmentContainerView` menggantikan penggunaan `FrameLayout` untuk penempatan fragment dan lebih terintegrasi dengan `Navigation Component` `Android`. `android:id="@+id/nav_host_fragme"` memberikan ID untuk `FragmentContainerView`, yang nantinya bisa digunakan untuk referensi dalam kode. `android:name="androidx.navigation.fragment.NavHostFragment"` untuk menyatakan bahwa fragment yang akan ditampilkan di dalam `FragmentContainerView` adalah `NavHostFragment`, yang merupakan bagian dari `Navigation Component`. Fragment ini bertugas mengelola navigasi antar fragment. `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk membuat `FragmentContainerView` mengambil ruang penuh dari layar, dengan lebar dan tinggi `match_parent`, sehingga mengisi seluruh layout. `app:navGraph="@navigation/nav_graph"` untuk menyediakan `NavGraph` yang berisi alur navigasi antar fragment. `@navigation/nav_graph` adalah referensi ke file `nav_graph.xml`, yang mendefinisikan destinasi dan aksi antar fragment dalam aplikasi. `app:defaultNavHost="true"` digunakan untuk menandakan bahwa `NavHostFragment` ini adalah host utama untuk navigasi di activity ini. Artinya, fragment ini akan menangani back stack navigasi, dan tombol back perangkat akan berfungsi untuk kembali ke fragment sebelumnya (jika ada).

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/nrhikmahhh99/Pemrograman-Mobile>